

Méthodes numériques en optimisation

Cours complet en français

Zakari HACHEMI

Basé sur le cours Pierre Gruet - Numerical Methods - MMMEF
2025-2026

Table des matières

1	Introduction aux méthodes numériques pour l'optimisation	1
1.1	Motivations et exemple introductif	1
1.2	Formulation générale d'un problème d'optimisation	2
1.3	Classification des problèmes d'optimisation	3
1.3.1	Avec ou sans contraintes	3
1.3.2	Optimisation continue, discrète et stochastique	3
1.3.3	Problèmes linéaires, quadratiques, non linéaires	3
1.3.4	Convexité, concavité et programmes convexes	4
1.3.5	Méthodes de pénalisation	4
1.4	Principe général des algorithmes d'optimisation	4
1.5	Propriétés souhaitables des méthodes numériques	5
1.6	Critères d'arrêt	5
2	Théorie de l'optimisation sans contrainte	6
2.1	Introduction	6
2.1.1	Problème général	6
2.1.2	Reformulation	6
2.2	Identification des minimiseurs	7
2.2.1	Théorème de Taylor	7
2.2.2	Condition nécessaire d'ordre 1	8
2.2.3	Conditions d'ordre 2 : nécessaire	9
2.2.4	Condition d'ordre 2 : suffisante	9
2.2.5	Cas convexe	10
2.3	Stratégies d'itération	10
2.3.1	Méthodes de région de confiance	11
2.3.2	Méthodes de recherche linéaire	11
2.4	Directions de recherche pour la recherche linéaire	15
2.4.1	Descente de gradient (plus forte pente)	15
2.4.2	Direction de Newton	17
2.4.3	Caractérisation de la convergence	17
2.4.4	Vitesse de convergence de la descente de gradient	19
2.4.5	Méthode de Newton : convergence quadratique	21
2.4.6	Modification de la Hessienne pour Newton	22
3	Méthodes du gradient conjugué	25
3.1	Gradient conjugué linéaire	25
3.1.1	Problème quadratique associé	25
3.1.2	Directions A -conjuguées	26
3.1.3	Méthode des directions conjuguées	26
3.1.4	Algorithme du gradient conjugué linéaire	26
3.1.5	Borne d'erreur et conditionnement	27

3.2	Interprétation géométrique	27
3.2.1	Niveaux d'une fonction quadratique	27
3.3	Comparaison avec la descente de gradient	28
3.4	Préconditionnement	28
3.5	Gradient conjugué non linéaire	28
3.6	Résumé	29
3.6.1	Propriétés des itérés et sous-espaces de Krylov	29
3.6.2	Dérivation de la version efficace (Standard CG)	29
3.7	Vitesse de convergence	30
3.7.1	Illustration : Effet de la distribution des valeurs propres	30
3.7.2	Comparaison des taux de convergence	31
3.8	Préconditionnement	31
3.8.1	Idée générale	31
3.8.2	Choix de la matrice de preconditionnement	31
3.8.3	Mise en œuvre pratique	32
3.8.4	Exemple : Préconditionnement de Cholesky Incomplet	32
4	Méthodes du gradient conjugué non linéaire	33
4.1	Introduction	33
4.2	Méthode de Fletcher et Reeves	33
4.2.1	1. Recherche linéaire (Line Search)	33
4.2.2	2. Substitution du résidu par le gradient	33
4.2.3	Formule du β_{FR}	33
4.2.4	Illustration sur la fonction de Rosenbrock	34
4.2.5	Propriété de descente globale	35
4.3	Méthode de Polak et Ribière	36
4.3.1	Définition et propriétés	37
4.3.2	Variante β^{PR+}	37
4.3.3	Remarques pratiques et vitesse de convergence	37
4.4	Analyse de la convergence globale	37
4.4.1	Hypothèses (Assumptions A)	37
4.4.2	Théorème d'Al-Baali	38
5	Méthodes Quasi-Newton	39
5.1	Introduction	39
5.2	La méthode BFGS	39
5.3	La formule de mise à jour de Davidon, Fletcher, Powell (DFP)	39
5.3.1	L'équation sécante	40
5.3.2	Condition de courbure	41
5.3.3	Construction de la mise à jour DFP	41
5.3.4	Expression de l'inverse (Formule de Sherman-Morrison-Woodbury)	42
5.4	L'approche BFGS	42
5.5	Méthodes Quasi-Newton à mémoire limitée (L-BFGS)	43
5.6	Remarques et Propriétés	44
5.7	La classe de Broyden	44
5.8	Propriétés sur les fonctions quadratiques	45
5.9	Analyse de convergence	45
5.9.1	Convergence globale	45
5.9.2	Convergence super-linéaire de BFGS	46

6	Problèmes d'optimisation sans contrainte à grande échelle	47
6.1	Introduction	47
6.2	Méthodes de Newton Inexactes	47
6.2.1	Convergence locale	47
6.2.2	Taux de convergence	48
6.2.3	Méthodes Newton-CG avec recherche linéaire	48
6.3	Méthodes Quasi-Newton à mémoire limitée (L-BFGS)	48
6.3.1	Rappels sur BFGS	49
6.3.2	Principes de L-BFGS	49
7	Optimisation sans dérivées	51
7.1	Introduction	51
7.2	Quelques solutions possibles	51
7.3	Le problème du bruit	51
7.3.1	Niveau de bruit et erreur d'approximation	52
7.3.2	Exemple : Bruit stochastique	52
7.4	Méthodes basées sur un modèle (Model-based methods)	53
7.4.1	Idée générale	53
7.4.2	Construction du modèle	53
7.4.3	Mise à jour	54
7.4.4	Initialisation et géométrie	54
7.5	Méthodes de recherche par coordonnées ou par motifs (Coordinate or pattern-search methods)	55
7.5.1	Recherche par coordonnées (Coordinate search)	55
7.5.2	Méthodes de recherche par motifs (Pattern-search methods)	56
7.6	Méthode du simplexe de Nelder-Mead	58
7.6.1	Principe général	58
7.6.2	Notation et ordonnancement	58
7.6.3	Centroïde	59
7.6.4	Points candidats	59
7.6.5	Remarques	60
7.6.6	Algorithme 15 : Méthode de Nelder-Mead	61
7.6.7	Illustration sur la fonction de Rosenbrock	62
8	Théorie de l'optimisation sous contraintes	63
8.1	Problème général	63
8.2	Ensemble réalisable	63
8.3	Solutions locales	63
8.4	Rappel : Équivalence problème lisse/non-lisse	64
8.5	Programmation linéaire : la méthode du simplexe	65
8.5.1	Formulation standard	65
8.5.2	Variables de base et hors-base	65
8.5.3	Algorithme 16 : Une étape de la méthode du simplexe	66
9	Conditions d'optimalité	67
9.1	Introduction et définitions	67
9.2	Exemples	67
9.2.1	Une contrainte d'égalité	67
9.2.2	Une contrainte d'inégalité	70
9.2.3	Deux contraintes d'inégalité	73
9.3	Cône tangent et qualifications des contraintes	75
9.3.1	Définitions	75

9.3.2	Qualification des contraintes (LICQ)	77
9.4	Conditions d'optimalité	78
9.4.1	La fonction Lagrangienne	78
9.4.2	Conditions KKT	78
9.4.3	Conditions du second ordre	79
9.5	Dualité	81
9.5.1	Problème Primal	81
9.5.2	Fonction Duale et Problème Dual	81
9.5.3	Exemple	82
9.5.4	Propriétés et Théorèmes de Dualité	82
9.5.5	Exemple : Dualité en Programmation Linéaire	83
10	Programmation linéaire : la méthode du simplexe	85
10.1	Optimalité et dualité	85
10.1.1	Optimalité du problème primal	85
10.1.2	Le problème dual	85
10.2	Géométrie de l'ensemble réalisable	86
10.3	Aperçu de la méthode du simplexe	86
11	Programmation quadratique	87
11.1	Introduction	87
11.2	QP avec contraintes d'égalité	87
11.2.1	Propriétés	87
11.2.2	Résolution directe du système KKT	89
11.2.3	Résolution itérative du système KKT	92
11.3	Problèmes avec contraintes d'inégalité	93
11.3.1	Propriétés	93
11.3.2	La méthode du gradient projeté	93
12	Méthodes de pénalité et Lagrangien augmenté	94
12.1	La méthode de pénalité quadratique	94
12.1.1	Motivation	94
12.1.2	Convergence de la méthode (avec contraintes d'égalité)	94
12.2	Fonctions de pénalité non lisses	94
12.2.1	Propriétés	94
12.2.2	Méthode de pénalité ℓ^1 pratique	95
12.3	Méthode du Lagrangien augmenté	95
12.3.1	Propriétés du cas avec contraintes d'égalité	95
12.3.2	Méthodes pratiques du Lagrangien augmenté	96

Chapitre 1

Introduction aux méthodes numériques pour l'optimisation

1.1 Motivations et exemple introductif

L'*optimisation* consiste à choisir, parmi un ensemble de décisions possibles, celle qui minimise (ou maximise) une quantité appelée *fonction objectif*. Dans ce cours, nous nous concentrons principalement sur les problèmes de *minimisation*, bien que les problèmes de maximisation puissent être traités de manière équivalente en considérant l'opposé de la fonction.

Les problèmes d'optimisation rencontrés en pratique peuvent être :

- **continus**, lorsque les variables prennent des valeurs dans $\mathbb{R}^n$;
- **discrets**, lorsque certaines variables doivent être entières (programmation en nombres entiers) ;
- **mixtes** (MIP : Mixed Integer Programming), lorsque certaines variables sont continues et d'autres discrètes.

Les problèmes discrets ou mixtes sont en général plus difficiles à résoudre que les problèmes continus et ne seront pas étudiés en détail dans ce cours.

On rencontre également des problèmes où des paramètres sont aléatoires. Cela conduit à :

- **l'optimisation stochastique**, où l'on minimise une espérance ;
- les **contraintes de chance** (chance-constrained), où certaines contraintes doivent être satisfaites avec une probabilité élevée ;
- **l'optimisation robuste**, où les contraintes doivent être satisfaites pour toutes les réalisations admissibles d'un ensemble d'incertitude.

Ces cadres sont essentiels dans des domaines tels que la finance, la gestion de l'énergie ou l'ingénierie des systèmes, mais ils dépassent le périmètre immédiat de ce chapitre.

Exemple 1.1 (Problème quadratique avec contraintes linéaires). On considère le problème

$$\min_{x \in \mathbb{R}^2} f(x) := (x_1 - 1)^2 + 2(x_2 + 3)^2 \quad (1.1)$$

sous les contraintes

$$x_1 \geq 2, \quad x_1 - x_2 = 4. \quad (1.2)$$

La fonction objectif f mesure la « mauvaise qualité » d'un vecteur $x = (x_1, x_2)$. Plus $f(x)$ est petit, plus la solution est satisfaisante. Les contraintes imposent que les solutions admissibles vivent sur une droite et dans un demi-plan. Les courbes de niveau de f sont des ellipses centrées en $(1, -3)$.

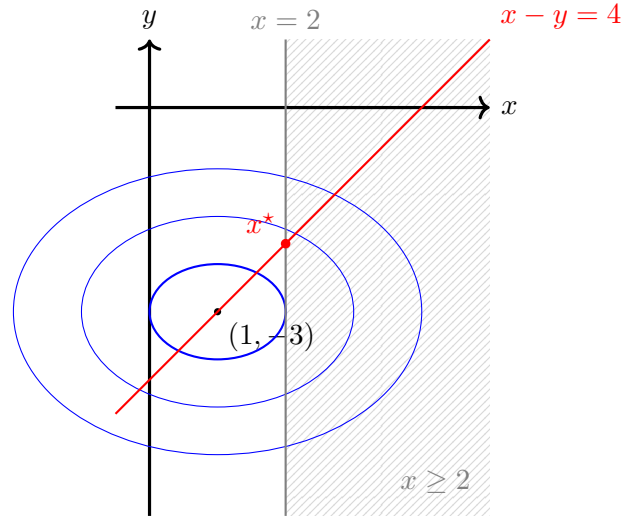


FIGURE 1.1 – Illustration du problème quadratique d'introduction.

Sur la figure 1.1, les **ellipses bleues** représentent les courbes de niveau de la fonction objectif f : plus l'ellipse est petite, plus la valeur de f est faible. Le **centre** des ellipses, situé en $(1, -3)$, correspond au minimum non contraint de f . La **zone hachurée** représente la région admissible $x \geq 2$. La **droite grise verticale** $x = 2$ matérialise la frontière de cette contrainte, et la **droite rouge** représente la contrainte d'égalité $x - y = 4$. Le point x^* , intersection de la droite rouge avec la frontière de la région admissible, est la **solution optimale** du problème contraint.

Ce problème illustre trois composantes essentielles :

- la **fonction objectif** $f : \mathbb{R}^n \rightarrow \mathbb{R}$;
- l'**ensemble des contraintes**, définissant les points admissibles ;
- la **solution optimale** x^* , minimisant f parmi les points admissibles.

1.2 Formulation générale d'un problème d'optimisation

Définition 1.2 (Problème d'optimisation). On appelle *problème d'optimisation* la recherche d'un vecteur $x \in \mathbb{R}^n$ minimisant une fonction $f : \mathbb{R}^n \rightarrow \mathbb{R}$ sous des contraintes d'égalité et/ou d'inégalité, c'est-à-dire :

$$\begin{aligned} & \min_{x \in \mathbb{R}^n} f(x) \\ & \text{sous les contraintes} \quad c_i(x) \geq 0, \quad i \in I, \\ & \quad \quad \quad c_j(x) = 0, \quad j \in E. \end{aligned} \tag{1.3}$$

Définition 1.3 (Ensemble admissible). L'ensemble admissible est

$$\mathcal{X} = \{x \in \mathbb{R}^n : c_i(x) \geq 0 \ \forall i \in I, \ c_j(x) = 0 \ \forall j \in E\}.$$

Tout point de $\mathcal{X}$ est dit *admissible*.

Définition 1.4 (Solution optimale). Un point $x^* \in \mathcal{X}$ est une *solution optimale globale* si

$$f(x^*) \leq f(x), \quad \forall x \in \mathcal{X}.$$

On parle de *minimum local* si cette propriété ne vaut que dans un voisinage de x^* .

Remarque 1.5. En pratique, il est rare de déterminer x^* exactement. Les méthodes numériques produisent une suite (x_k) dont les valeurs du gradient et/ou de la fonction décroissent vers zéro.

1.3 Classification des problèmes d'optimisation

1.3.1 Avec ou sans contraintes

- **Optimisation sans contraintes :**

$$\min_{x \in \mathbb{R}^n} f(x).$$

- **Optimisation avec contraintes :**

$$x \in \mathcal{X} \subseteq \mathbb{R}^n.$$

Un cas particulier courant est celui des contraintes de boîte :

$$\ell_i \leq x_i \leq u_i, \quad i = 1, \dots, n.$$

1.3.2 Optimisation continue, discrète et stochastique

Programmation linéaire en nombres entiers. Lorsque certaines variables x_i doivent être entières, le problème devient *discret* (NP-difficile en général).

Problèmes mixtes (MIP). Combinaison de variables continues et discrètes.

Optimisation stochastique. Présence de paramètres aléatoires :

- minimisation d'une espérance ;
- contraintes satisfaites avec probabilité $\geq p$ (chance-constrained) ;
- satisfaction pour tous les scénarios possibles (optimisation robuste).

1.3.3 Problèmes linéaires, quadratiques, non linéaires

- **Programmation linéaire (PL) :**

$$f(x) = c^\top x, \quad c_i(x) = a_i^\top x - b_i.$$

- **Programmation quadratique (PQ) :**

$$f(x) = \frac{1}{2}x^\top Qx + c^\top x,$$

où Q est symétrique.

- **Programmation non linéaire (PNL) :** f et/ou les contraintes sont non linéaires.

1.3.4 Convexité, concavité et programmes convexes

Définition 1.6 (Ensemble convexe). Un ensemble $\mathcal{C}$ est convexe si

$$\theta x + (1 - \theta)y \in \mathcal{C}, \quad \forall x, y \in \mathcal{C}, \theta \in [0, 1].$$

Définition 1.7 (Fonction convexe). Une fonction f est convexe si

$$f(\theta x + (1 - \theta)y) \leq \theta f(x) + (1 - \theta)f(y).$$

Elle est *strictement convexe* si l'inégalité est stricte pour $x \neq y$. Elle est *concave* si $-f$ est convexe.

Définition 1.8 (Programme convexe). Un problème d'optimisation est dit *convexe* lorsque :

- la fonction objectif f est convexe ;
- les contraintes d'égalité sont linéaires ;
- les contraintes d'inégalité $c_i(x) \geq 0$ sont concaves.

Dans ce cadre, tout minimum local est automatiquement un minimum global.

1.3.5 Méthodes de pénalisation

Certaines contraintes peuvent être intégrées à la fonction objectif. Par exemple :

$$\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 + 3)^2 \quad \text{sous} \quad x_1 + x_2 = 5$$

peut être reformulé en problème sans contrainte :

$$\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 + 3)^2 + \frac{\eta}{2} (x_1 + x_2 - 5)^2, \quad \eta > 0.$$

On en reparlera dans le dernier chapitre sur la pénalisation, mais ce deuxième problème n'est équivalent au premier que pour $\eta \rightarrow \infty$. Précisément, sa solution est

$$x_1 = \frac{9\eta + 2}{2\eta + 2}, \quad x_2 = \frac{\eta - 6}{2\eta + 2}.$$

Alternativement, on peut s'intéresser au problème

$$\min (x_1 - 1)^2 + (x_2 + 3)^2 + \eta |x_1 + x_2 - 5|,$$

pour lequel on indiquera dans le dernier chapitre que la solution coïncide avec celle du problème contraint initial dès lors que $\eta > 7$.

1.4 Principe général des algorithmes d'optimisation

Les méthodes numériques d'optimisation sont *itératives* : elles construisent une suite (x_k) convergeant vers une solution optimale (ou stationnaire). Le schéma général est :

$$x_{k+1} = x_k + \alpha_k p_k,$$

où p_k est une direction de recherche et $\alpha_k > 0$ un pas.

Entrée : un point initial x_0 .

Pour $k = 0, 1, 2, \dots$ **tant que** le critère d'arrêt n'est pas satisfait :

1. Choisir une direction p_k (par exemple $p_k = -\nabla f(x_k)$).
2. Choisir une longueur de pas $\alpha_k > 0$ (recherche linéaire, région de confiance, etc.).
3. Mettre à jour $x_{k+1} = x_k + \alpha_k p_k$.

Fin Pour

Pour les méthodes non contraintes, l'algorithme s'appuie sur les informations suivantes :

- uniquement $f(x_k)$ (méthodes “boîte noire”);
- le gradient $\nabla f(x_k)$ (méthodes de gradient);
- le Hessien $\nabla^2 f(x_k)$ (méthodes de type Newton);
- des approximations du Hessien (méthodes quasi-Newton).

1.5 Propriétés souhaitables des méthodes numériques

Une méthode d'optimisation efficace doit satisfaire plusieurs critères :

- **Robustesse** : bon comportement pour différents problèmes et points initiaux;
- **Efficacité** : coût raisonnable en temps et en mémoire;
- **Précision** : faible sensibilité aux erreurs d'arrondi et aux imprécisions de calcul;
- **Stabilité numérique** : éviter l'amplification des erreurs;
- **Fiabilité** : éviter les stagnations ou comportements chaotiques.

1.6 Critères d'arrêt

Un algorithme d'optimisation s'arrête généralement lorsque l'un des critères suivants est atteint :

- **norme du gradient petite** :

$$\|\nabla f(x_k)\| \leq \varepsilon_{\text{grad}};$$

- **variation faible de la solution** :

$$\|x_{k+1} - x_k\| \leq \varepsilon_{\text{step}};$$

- **variation faible de la fonction** :

$$|f(x_{k+1}) - f(x_k)| \leq \varepsilon_f;$$

- **nombre maximal d'itérations atteint.**

Ce premier chapitre établit le cadre général de l'optimisation numérique : formulation, classification, convexité, typologie large des problèmes, structure des algorithmes et propriétés essentielles. Les chapitres suivants seront consacrés à l'étude détaillée des méthodes de résolution, en commençant par l'optimisation sans contrainte.

Chapitre 2

Théorie de l'optimisation sans contrainte

2.1 Introduction

2.1.1 Problème général

On considère le problème d'optimisation non contraint

$$\min_{x \in \mathbb{R}^n} f(x), \quad (2.1)$$

où $n \geq 1$ et où $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est supposée suffisamment régulière (typiquement $\mathcal{C}^1$ ou $\mathcal{C}^2$).

Remarque 2.1. Nous laissons de côté l'optimisation non lisse : il peut être difficile d'analyser le comportement local de f au voisinage de x_k . Pour une présentation générale, voir par exemple Hiriart-Urruty et Lemaréchal.

2.1.2 Reformulation

Il est parfois possible de reformuler un problème non lisse en un problème lisse en augmentant la dimension.

Soient $f_1, \dots, f_m : \mathbb{R}^n \rightarrow \mathbb{R}$ des fonctions régulières (lisses), et définissons

$$f(x) = \max_{j=1, \dots, m} f_j(x). \quad (2.2)$$

Alors le problème

$$(P) : \min_{x \in \mathbb{R}^n} f(x) = \min_{x \in \mathbb{R}^n} \max_{1 \leq j \leq m} f_j(x) \quad (2.3)$$

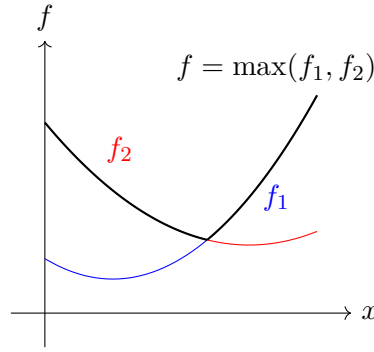
est un problème non lisse construit à partir de fonctions régulières.

Ce problème peut se réécrire

$$\min_{(x,t) \in \mathbb{R}^{n+1}} t \quad \text{sous les contraintes} \quad t - f_j(x) \geq 0, \quad j = 1, \dots, m, \quad (2.4)$$

la dernière condition signifiant

$$f_j(x) \leq t, \quad j = 1, \dots, m. \quad (2.5)$$

FIGURE 2.1 – Exemple de fonction non lisse $f(x) = \max(f_1(x), f_2(x))$.

Sur la figure 2.1, la courbe épaisse noire représente la fonction $f(x) = \max(f_1(x), f_2(x))$. Elle suit la portion supérieure de chaque courbe : f_2 (rouge) lorsque x est petit, et f_1 (bleue) lorsque x est grand. Le point de croisement marque une brisure de la pente, rendant f non différentiable en ce point (fonction non lisse).

2.2 Identification des minimiseurs

Définitions

Définition 2.2. Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$. Un point $x^* \in \mathbb{R}^n$ est

- un **minimiseur global** de f si

$$f(x^*) \leq f(x), \quad \forall x \in \mathbb{R}^n;$$

- un **minimiseur local (faible)** s'il existe un voisinage V de x^* tel que

$$f(x^*) \leq f(x), \quad \forall x \in V;$$

- un **minimiseur local strict** s'il existe un voisinage V de x^* tel que

$$f(x^*) < f(x), \quad \forall x \in V \setminus \{x^*\};$$

- un **minimiseur local isolé** s'il existe un voisinage V de x^* tel que x^* soit l'unique minimiseur local de f dans V .

2.2.1 Théorème de Taylor

Théorème 2.3 (Théorème de la valeur moyenne). *Supposons que $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est de classe $\mathcal{C}^1$. Alors pour tout $x \in \mathbb{R}^n$ et tout $\eta \in \mathbb{R}^n$, il existe $t \in (0, 1)$ tel que*

$$f(x + \eta) = f(x) + \nabla f(x + t\eta)^\top \eta. \quad (2.6)$$

Théorème 2.4 (Développements de Taylor). *Si $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est de classe $\mathcal{C}^2$, alors pour tout $x, \eta \in \mathbb{R}^n$:*

1. *Il existe $t_1 \in (0, 1)$ tel que*

$$\nabla f(x + \eta) = \nabla f(x) + \nabla^2 f(x + t_1 \eta) \eta.$$

2. *Il existe $t_2 \in (0, 1)$ tel que*

$$f(x + \eta) = f(x) + \nabla f(x)^\top \eta + \frac{1}{2} \eta^\top \nabla^2 f(x + t_2 \eta) \eta.$$

Gradient et Hessienne. Pour $f : \mathbb{R}^n \rightarrow \mathbb{R}$, le gradient en x est

$$\nabla f(x) = \begin{pmatrix} \frac{\partial f}{\partial x_1}(x) \\ \vdots \\ \frac{\partial f}{\partial x_n}(x) \end{pmatrix} \in \mathbb{R}^n.$$

La matrice Hessienne de f en $x \in \mathbb{R}^n$ est

$$\nabla^2 f(x) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2}(x) & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n}(x) \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1}(x) & \cdots & \frac{\partial^2 f}{\partial x_n^2}(x) \end{pmatrix} \in \mathbb{R}^{n \times n}.$$

Pour $x \in \mathbb{R}^n$, on note aussi $x^\top = (x_1, \dots, x_n)$.

2.2.2 Condition nécessaire d'ordre 1

Théorème 2.5 (Condition nécessaire du premier ordre). *Supposons que f est de classe $\mathcal{C}^1$ dans un voisinage de x^* , et que x^* est un minimiseur local de f . Alors*

$$\nabla f(x^*) = 0. \quad (2.7)$$

Démonstration. Raisonnons par l'absurde. Supposons que $\nabla f(x^*) \neq 0$ et posons $\eta = -\nabla f(x^*) \neq 0$. Comme f est $\mathcal{C}^1$ au voisinage de x^* , on a

$$\eta^\top \nabla f(x^*) = -\|\nabla f(x^*)\|^2 < 0.$$

Par le théorème de la valeur moyenne, il existe $t \in (0, T)$ tel que

$$f(x^* + t\eta) = f(x^*) + t \nabla f(x^* + t'\eta)^\top \eta$$

pour un certain $t' \in (0, t)$. Pour $t > 0$ assez petit, le produit $\nabla f(x^* + t'\eta)^\top \eta$ reste strictement négatif par continuité. On obtient alors

$$f(x^* + t\eta) < f(x^*),$$

et ceci pour tout $t \in [0, T]$: on a bien capté une direction de descente, et pas seulement un point isolé. Cela contredit le fait que x^* est un minimiseur local. On doit donc avoir $\nabla f(x^*) = 0$. $\square$

2.2.3 Conditions d'ordre 2 : nécessaire

Définition 2.6. Un point x^* est un *point stationnaire* de f si

$$\nabla f(x^*) = 0.$$

Le théorème 2.5 affirme que tout minimiseur local est un point stationnaire.

Théorème 2.7 (Condition nécessaire du second ordre). *Supposons que f est de classe $\mathcal{C}^2$ dans un voisinage de x^* , et que x^* est un minimiseur local de f . Alors*

$$\nabla f(x^*) = 0 \quad \text{et} \quad \nabla^2 f(x^*) \succeq 0,$$

c'est-à-dire

$$\forall \eta \in \mathbb{R}^n, \quad \eta^\top \nabla^2 f(x^*) \eta \geq 0.$$

Démonstration. On sait déjà par [Théorème 2.5](#) que $\nabla f(x^*) = 0$. Raisonnons par l'absurde pour la Hessienne. Supposons que $\nabla^2 f(x^*)$ n'est pas définie positive au sens large. Alors il existe $\eta \in \mathbb{R}^n$ tel que

$$\eta^\top \nabla^2 f(x^*) \eta < 0.$$

Par continuité de $\nabla^2 f$ au voisinage de x^* , il existe $T > 0$ tel que

$$\forall t \in [0, T], \quad \eta^\top \nabla^2 f(x^* + t\eta) \eta < 0.$$

En particulier, pour $t \in (0, T)$, le développement de Taylor donne

$$f(x^* + t\eta) = f(x^*) + t \nabla f(x^*)^\top \eta + \frac{t^2}{2} \eta^\top \nabla^2 f(x^* + \theta t\eta) \eta$$

pour un certain $\theta \in (0, 1)$. Comme $\nabla f(x^*) = 0$ et $\eta^\top \nabla^2 f(x^* + \theta t\eta) \eta < 0$, on obtient

$$f(x^* + t\eta) < f(x^*),$$

et ceci pour tout $t \in [0, T]$: on a une direction de descente et pas un point isolé. Cela contredit le fait que x^* est un minimiseur local. Donc $\nabla^2 f(x^*)$ doit être définie positive au sens large. $\square$

2.2.4 Condition d'ordre 2 : suffisante

Théorème 2.8 (Condition suffisante du second ordre). *Supposons que f est de classe $\mathcal{C}^2$ sur un voisinage ouvert de x^* , et que*

$$\nabla f(x^*) = 0, \quad \nabla^2 f(x^*) \succ 0,$$

c'est-à-dire

$$\forall \eta \in \mathbb{R}^n \setminus \{0\}, \quad \eta^\top \nabla^2 f(x^*) \eta > 0.$$

Alors x^ est un minimiseur local strict de f .*

Démonstration. La Hessienne est continue et définie positive en x^* . Il existe donc $r > 0$ tel que $\nabla^2 f(x) \succ 0$ pour tout x dans la boule

$$D = \{x \in \mathbb{R}^n \mid \|x - x^*\| < r\}.$$

Soit $\eta \in \mathbb{R}^n$ tel que $\|\eta\| \leq r$. Par le théorème de Taylor 2.4, il existe $t \in (0, 1)$ tel que

$$f(x^* + \eta) = f(x^*) + \eta^\top \nabla f(x^*) + \frac{1}{2} \eta^\top \nabla^2 f(x^* + t\eta) \eta.$$

Comme $\nabla f(x^*) = 0$ et $\nabla^2 f(x^* + t\eta) \succ 0$, on en déduit

$$f(x^* + \eta) > f(x^*)$$

dès que $\eta \neq 0$ et $\|\eta\|$ suffisamment petit. Donc x^* est un minimiseur local strict. $\square$

2.2.5 Cas convexe

Théorème 2.9 (Cas convexe). *Soit $f : \mathbb{R}^n \rightarrow \mathbb{R}$ une fonction convexe.*

1. *Tout minimiseur local de f est également un minimiseur global.*
2. *Si f est en plus différentiable, alors tout point stationnaire ($\nabla f(x^*) = 0$) est un minimiseur global.*

Preuve du point (1). Supposons que x^* est un minimiseur local mais non global. Alors il existe $y \in \mathbb{R}^n$ tel que

$$f(y) < f(x^*).$$

Par convexité de f , pour tout $\lambda \in [0, 1]$ on a

$$f(\lambda x^* + (1 - \lambda)y) \leq \lambda f(x^*) + (1 - \lambda)f(y) < f(x^*) \quad (\lambda \neq 1).$$

Soit V un voisinage de x^* . Pour $\lambda \in (0, 1)$ suffisamment petit, le point $z = \lambda x^* + (1 - \lambda)y$ appartient à V et vérifie $f(z) < f(x^*)$, ce qui contredit la minimalité locale de x^* . $\square$

2.3 Stratégies d'itération

Les algorithmes d'optimisation construisent une suite $(x_k)_{k \geq 0}$ telle que

$$x_{k+1} = x_k + \alpha_k p_k,$$

où p_k est une direction de recherche (en général de descente) et $\alpha_k > 0$ un pas (longueur de déplacement).

En pratique :

- on choisit un point de départ x_0 ,
- à l'itération k , on utilise l'information locale (valeurs de f , de ∇f , voire de $\nabla^2 f$) pour définir p_k et α_k ,
- on cherche typiquement à assurer

$$f(x_{k+1}) < f(x_k),$$

ou au moins une décroissance après plusieurs itérations :

$$f(x_{k+m}) < f(x_k).$$

Deux grandes familles de stratégies existent :

- méthodes de **région de confiance**,
- méthodes de **recherche linéaire**.

2.3.1 Méthodes de région de confiance

À l'itération k , on fixe un rayon $\Delta_k > 0$ définissant la région de confiance autour de x_k :

$$\{x \in \mathbb{R}^n \mid \|x - x_k\| \leq \Delta_k\}.$$

En utilisant la formule de Taylor, on approche f par un modèle quadratique

$$m_k(x) = f(x_k) + (x - x_k)^\top \nabla f(x_k) + \frac{1}{2}(x - x_k)^\top \nabla^2 f(x_k) (x - x_k). \quad (2.8)$$

On choisit alors x_{k+1} comme solution (ou approximation de la solution) du problème de région de confiance

$$x_{k+1} = \arg \min_x m_k(x) \quad \text{s.c. } \|x - x_k\| \leq \Delta_k. \quad (2.9)$$

En posant $p_k = x_{k+1} - x_k$, cela revient à minimiser $m_k(p)$ sous la contrainte $\|p\| \leq \Delta_k$:

$$p_k = \arg \min_{\|p\| \leq \Delta_k} \left(f(x_k) + \nabla f(x_k)^\top p + \frac{1}{2} p^\top \nabla^2 f(x_k) p \right). \quad (2.10)$$

On introduit le ratio

$$\rho_k = \frac{f(x_k) - f(x_k + p_k)}{m_k(0) - m_k(p_k)}. \quad (2.11)$$

Les règles d'acceptation typiques sont :

- si $\rho_k < 0$: la nouvelle itération est rejetée, et on réduit Δ_k ;
- si $0 < \rho_k$ est petit : on accepte x_{k+1} mais on garde Δ_k à peu près inchangé ;
- si $\rho_k \approx 1$: on accepte x_{k+1} et on augmente Δ_k .

2.3.2 Méthodes de recherche linéaire

Les méthodes de recherche linéaire ont la forme

$$x_{k+1} = x_k + \alpha_k p_k,$$

où p_k est une direction de descente et α_k un pas choisi de façon adaptative.

Un choix fréquent pour la direction p_k est

$$p_k = -B_k \nabla f_k, \quad (2.12)$$

où B_k est une matrice symétrique définie positive (ou du moins inversible) et

$$f_k = f(x_k), \quad \nabla f_k = \nabla f(x_k), \quad \nabla^2 f_k = \nabla^2 f(x_k).$$

Dans ce cas, on a

$$p_k^\top \nabla f_k = -\nabla f_k^\top B_k \nabla f_k < 0, \quad (2.13)$$

puisque B_k est définie positive et $\nabla f_k \neq 0$. Ainsi p_k est bien une direction de descente. De plus

$$p_k^\top \nabla f_k = \|p_k\| \|\nabla f_k\| \cos(\angle(p_k, \nabla f_k)) < 0,$$

ce qui implique que l'angle entre p_k et ∇f_k est strictement supérieur à $\frac{\pi}{2}$.

L'idée de base est de choisir α_k comme

$$\alpha_k = \arg \min_{\alpha > 0} \varphi(\alpha), \quad \varphi(\alpha) = f(x_k + \alpha p_k),$$

mais ce problème unidimensionnel peut être coûteux à résoudre exactement. On recherche donc une valeur α_k *acceptable*, satisfaisant des conditions suffisantes de décroissance.

Condition de décroissance suffisante (Armijo). On impose

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f_k^\top p_k, \quad (2.14)$$

où $c_1 \in (0, 1)$ est un paramètre (typiquement $c_1 = 10^{-4}$).

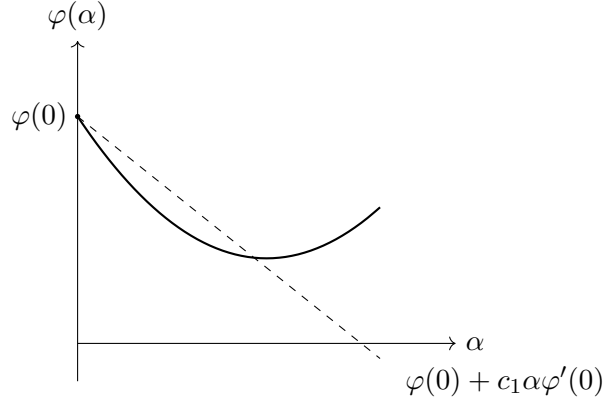


FIGURE 2.2 – Condition de décroissance suffisante (Armijo).

La figure 2.2 illustre la condition de décroissance suffisante : la courbe de la fonction $\varphi(\alpha)$ doit se situer en dessous de la droite pointillée d'équation $\varphi(0) + c_1 \alpha \varphi'(0)$. Cela garantit que la fonction décroît suffisamment par rapport à sa pente initiale.

Remarque 2.10. La condition d'Armijo est toujours satisfaite pour α suffisamment petit. Elle assure que la décroissance de f est proportionnelle à α et à la dérivée directionnelle $\nabla f_k^\top p_k$.

Définition 2.11 (Conditions de Wolfe). On dit que α satisfait les *conditions de Wolfe* si

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha \nabla f_k^\top p_k, \quad (2.15)$$

$$\nabla f(x_k + \alpha p_k)^\top p_k \geq c_2 \nabla f_k^\top p_k, \quad (2.16)$$

pour des constantes $0 < c_1 < c_2 < 1$. En notant $\varphi(\alpha) = f(x_k + \alpha p_k)$, la deuxième condition s'écrit aussi $\varphi'(\alpha) \geq c_2 \varphi'(0)$.

Définition 2.12 (Conditions de Wolfe fortes). Les *conditions de Wolfe fortes* sont obtenues en remplaçant la condition de courbure précédente par

$$|\nabla f(x_k + \alpha p_k)^\top p_k| \leq -c_2 \nabla f_k^\top p_k, \quad (2.17)$$

c'est-à-dire

$$|\varphi'(\alpha)| \leq -c_2 \varphi'(0).$$

Cela exclut les valeurs de α pour lesquelles la pente est trop positive.

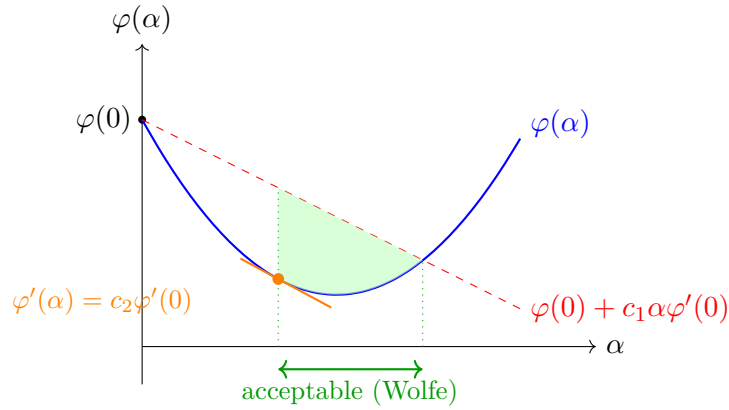


FIGURE 2.3 – Conditions de Wolfe : la zone acceptable (verte) satisfait à la fois la décroissance suffisante (Armijo) et la condition de courbure.

Sur la figure 2.3, la **zone acceptable** (en vert) est l'intervalle des pas α qui satisfont simultanément les deux conditions de Wolfe : 1. La condition d'Armijo (décroissance suffisante) : la courbe bleue est sous la droite rouge. 2. La condition de courbure : la pente de la courbe (tangente orange) est supérieure ou égale à c_2 fois la pente initiale.

Remarque 2.13. Pour des méthodes de type Newton ou quasi-Newton, on choisit souvent $c_2 \approx 0,9$, tandis que pour des méthodes de gradient conjugué non linéaire, on prend plutôt $c_2 \approx 0,1$.

Lemme 2.14. *Supposons que f est de classe $\mathcal{C}^1$ et que p_k est une direction de descente. Supposons que f est bornée inférieurement le long du rayon*

$$\{x_k + \alpha p_k \mid \alpha > 0\}.$$

Alors, pour tout $0 < c_1 < c_2 < 1$, il existe des intervalles de valeurs de α satisfaisant les conditions de Wolfe (simples ou fortes).

Démonstration. Notons $\varphi(\alpha) = f(x_k + \alpha p_k)$, et $\varphi'(0) = \nabla f_k^\top p_k < 0$. Comme f est bornée inférieurement sur le rayon, φ l'est aussi.

Pour $c_1 \in (0, 1)$, considérons la fonction affine

$$\ell(\alpha) = f(x_k) + c_1 \alpha \nabla f_k^\top p_k.$$

Comme $\ell(\alpha) \rightarrow -\infty$ quand $\alpha \rightarrow +\infty$, les graphes de φ et de ℓ se coupent. Soit $\hat{\alpha} > 0$ le plus petit point d'intersection :

$$\varphi(\hat{\alpha}) = f(x_k + \hat{\alpha} p_k) = f(x_k) + c_1 \hat{\alpha} \nabla f_k^\top p_k.$$

La condition d'Armijo est donc vérifiée strictement pour tout $\alpha \in (0, \hat{\alpha})$.

Par le théorème de la valeur moyenne, il existe $\bar{\alpha} \in (0, \hat{\alpha})$ tel que

$$\varphi(\hat{\alpha}) - \varphi(0) = \hat{\alpha} \varphi'(\bar{\alpha}).$$

En combinant avec la définition de $\hat{\alpha}$, on obtient

$$c_1 \nabla f_k^\top p_k = \varphi'(\bar{\alpha}).$$

Comme $c_1 < c_2$ et $\nabla f_k^\top p_k < 0$, il vient

$$c_2 \nabla f_k^\top p_k < \varphi'(\bar{\alpha}),$$

soit la condition de Wolfe

$$\nabla f(x_k + \bar{\alpha}p_k)^\top p_k > c_2 \nabla f_k^\top p_k.$$

Par continuité de φ' et de φ , cette condition et la condition d'Armijo restent valides dans un petit intervalle autour de $\bar{\alpha}$, ce qui donne un intervalle d' α satisfaisant les conditions de Wolfe. De plus, comme $\varphi'(\bar{\alpha}) < 0$, on peut également satisfaire la condition de Wolfe forte sur un intervalle voisin. $\square$

Recherche linéaire par *backtracking*. Une façon simple d'assurer la condition d'Armijo est la *recherche linéaire par backtracking*.

Algorithm 1: Recherche linéaire avec retour en arrière (Backtracking)

- 1 Choose $\bar{\alpha} > 0$, $\rho \in (0, 1)$, $c \in (0, 1)$; Set $\alpha \leftarrow \bar{\alpha}$;
 - 2 **repeat**
 - 3 | $\alpha \leftarrow \rho\alpha$;
 - 4 **until** $f(x_k + \alpha p_k) \leq f(x_k) + c\alpha \nabla f_k^\top p_k$;
 - 5 Terminate with $\alpha_k = \alpha$.
-

Remarque 2.15. En pratique, on commence presque toujours la recherche linéaire avec $\bar{\alpha} = 1$.

Algorithme de recherche linéaire avec zoom. Pour satisfaire les conditions de Wolfe, on utilise un algorithme plus sophistiqué qui fait appel à une procédure de *zoom*.

Algorithm 2: Algorithme de recherche linéaire

- 1 Set $\alpha_0 \leftarrow 0$, choose $\alpha_{\max} > 0$ and $\alpha_1 \in (0, \alpha_{\max})$;
 - 2 $i \leftarrow 1$;
 - 3 **repeat**
 - 4 | Evaluate $\phi(\alpha_i)$;
 - 5 | **if** $\phi(\alpha_i) > \phi(0) + c_1\alpha_i\phi'(0)$ **or** $[\phi(\alpha_i) \geq \phi(\alpha_{i-1})$ **and** $i > 1]$ **then**
 - 6 | | $\alpha_* \leftarrow \text{zoom}(\alpha_{i-1}, \alpha_i)$ **and stop** ;
 - 7 | Evaluate $\phi'(\alpha_i)$;
 - 8 | **if** $|\phi'(\alpha_i)| \leq -c_2\phi'(0)$ **then**
 - 9 | | set $\alpha_* \leftarrow \alpha_i$ **and stop** ;
 - 10 | **if** $\phi'(\alpha_i) \geq 0$ **then**
 - 11 | | set $\alpha_* \leftarrow \text{zoom}(\alpha_i, \alpha_{i-1})$ **and stop** ;
 - 12 | Choose $\alpha_{i+1} \in (\alpha_i, \alpha_{\max})$;
 - 13 | $i \leftarrow i + 1$;
 - 14 **until**;
-

Algorithm 3: Procédure zoom

```

1 repeat
2   Interpolate to find a trial step length  $\alpha_j$  between  $\alpha_{lo}$  and  $\alpha_{hi}$ ;
3   Evaluate  $\phi(\alpha_j)$ ;
4   if  $\phi(\alpha_j) > \phi(0) + c_1\alpha_j\phi'(0)$  or  $\phi(\alpha_j) \geq \phi(\alpha_{lo})$  then
5      $\alpha_{hi} \leftarrow \alpha_j$ ;
6   else
7     Evaluate  $\phi'(\alpha_j)$ ;
8     if  $|\phi'(\alpha_j)| \leq -c_2\phi'(0)$  then
9       Set  $\alpha_* \leftarrow \alpha_j$  and stop ;
10    if  $\phi'(\alpha_j)(\alpha_{hi} - \alpha_{lo}) \geq 0$  then
11       $\alpha_{hi} \leftarrow \alpha_{lo}$ ;
12     $\alpha_{lo} \leftarrow \alpha_j$ ;
13 until;
```

Remarque 2.16. La fonction $\phi(\alpha) = f(x_k + \alpha p_k)$ et $\phi'(\alpha) = \nabla f(x_k + \alpha p_k)^\top p_k$. Les constantes c_1 et c_2 sont celles des conditions de Wolfe, avec typiquement $c_1 = 10^{-4}$ et $c_2 = 0.9$.

2.4 Directions de recherche pour la recherche linéaire

2.4.1 Descente de gradient (plus forte pente)

Soit une direction p et un pas $\alpha > 0$. Le développement de Taylor donne

$$f(x_k + \alpha p) = f(x_k) + \alpha p^\top \nabla f(x_k) + \frac{\alpha^2}{2} p^\top \nabla^2 f(x_k + t p) p$$

pour un certain $t \in (0, \alpha)$. Le taux de variation linéaire de f le long de p est le coefficient de α :

$$p^\top \nabla f(x_k).$$

La direction de plus forte décroissance (de norme 1) est obtenue en résolvant

$$\min_{\|p\|=1} p^\top \nabla f(x_k).$$

En utilisant

$$p^\top \nabla f(x_k) = \|p\| \|\nabla f(x_k)\| \cos(\angle(p, \nabla f(x_k))) = \|\nabla f(x_k)\| \cos(\angle(p, \nabla f(x_k))),$$

on voit que ce minimum est atteint lorsque $\cos(\angle(p, \nabla f(x_k))) = -1$, c'est-à-dire lorsque

$$p = -\frac{\nabla f(x_k)}{\|\nabla f(x_k)\|}.$$

La direction de plus forte pente est donc

$$p_k = -\frac{\nabla f_k}{\|\nabla f_k\|}. \quad (2.18)$$

Elle est orthogonale aux lignes de niveau de f et pointe dans la direction de décroissance maximale instantanée.

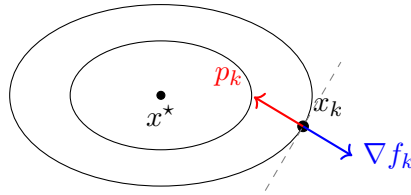


FIGURE 2.4 – Direction de plus forte pente.

La figure 2.4 montre que le gradient ∇f_k est orthogonal (perpendiculaire) à la ligne de niveau de f passant par x_k et pointe vers l'extérieur (croissance de la fonction). La direction de plus forte pente $p_k = -\nabla f_k$ est donc également orthogonale à la ligne de niveau, mais pointe vers l'intérieur (décroissance maximale locale).

Remarque 2.17. Pour la descente de gradient :

- on ne requiert que le gradient ∇f ;
- la direction est toujours de descente :

$$p_k^\top \nabla f_k = -\frac{1}{\|\nabla f_k\|} \|\nabla f_k\|^2 < 0;$$

- la méthode peut cependant être très lente, en particulier sur des problèmes mal conditionnés.

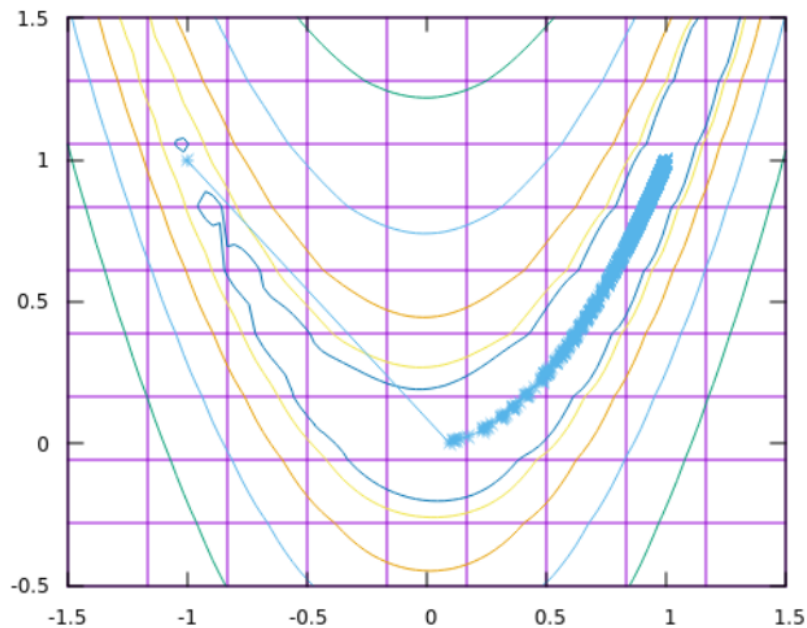


FIGURE 2.5 – Itérations de la méthode de descente du gradient (steepest descent) sur les courbes de niveau de la fonction de Rosenbrock.

Points clés à observer :

- **Trajectoire en zigzag :** La méthode oscille perpendiculairement à la vallée, car le gradient pointe toujours orthogonalement aux courbes de niveau.
- **Convergence lente :** Malgré de nombreuses itérations, la progression vers le minimum $(1, 1)$ est très lente le long de la vallée.
- **Limitation fondamentale :** Sans information de courbure (Hessienne), la méthode ne peut pas « redresser » sa direction vers le fond de la vallée.

Cette convergence lente motive l'utilisation de méthodes plus sophistiquées (Newton, BFGS, gradient conjugué).

2.4.2 Direction de Newton

Considérons l'approximation de second ordre

$$f(x_k + \eta) = f(x_k) + \eta^\top \nabla f(x_k) + \frac{1}{2} \eta^\top \nabla^2 f(x_k + t\eta) \eta,$$

pour un $t \in (0, 1)$. On approche f par le modèle quadratique

$$m_k(\eta) = f(x_k) + \eta^\top \nabla f(x_k) + \frac{1}{2} \eta^\top \nabla^2 f(x_k) \eta.$$

Si la Hessienne $\nabla^2 f_k$ est définie positive, minimiser m_k en η conduit à la condition d'optimalité

$$\nabla f_k + \nabla^2 f_k \eta_k^N = 0,$$

soit

$$\eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k. \quad (2.19)$$

La direction de Newton est donc $p_k^N = \eta_k^N$.

Si $\nabla^2 f_k \succ 0$, on a

$$\nabla f_k^\top p_k^N = -(p_k^N)^\top \nabla^2 f_k p_k^N < 0,$$

donc p_k^N est une direction de descente.

De plus, si $\nabla^2 f$ est suffisamment régulière, l'erreur de l'approximation de Taylor est d'ordre

$$f(x_k + \eta) - m_k(\eta) = \mathcal{O}(\|\eta\|^3).$$

Remarque 2.18. Le pas naturel en méthode de Newton est $\alpha = 1$:

$$x_{k+1} = x_k + p_k^N = x_k - (\nabla^2 f_k)^{-1} \nabla f_k.$$

En pratique, on effectue souvent une recherche linéaire (par exemple avec les conditions de Wolfe), en commençant par $\alpha = 1$.

2.4.3 Caractérisation de la convergence

Considérons un algorithme de type recherche linéaire

$$x_{k+1} = x_k + \alpha_k p_k,$$

où p_k est une direction de descente et α_k satisfait les conditions de Wolfe.

Théorème 2.19 (Zoutendijk). *Supposons que*

- $f : \mathbb{R}^n \rightarrow \mathbb{R}$ est bornée inférieurement,
- f est de classe $\mathcal{C}^1$ sur un ouvert $N \subset \mathbb{R}^n$ contenant l'ensemble de niveau

$$\{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\},$$

- le gradient ∇f est Lipschitzien sur N , i.e. il existe $L > 0$ tel que

$$\|\nabla f(x) - \nabla f(y)\| \leq L\|x - y\|, \quad \forall x, y \in N.$$

Alors

$$\sum_{k=0}^{\infty} \cos^2(\theta_k) \|\nabla f_k\|^2 < +\infty, \quad (2.20)$$

où θ_k est l'angle entre $-\nabla f_k$ et la direction p_k , défini par

$$\cos(\theta_k) = \frac{-\nabla f_k^\top p_k}{\|\nabla f_k\| \|p_k\|}.$$

Remarque 2.20. Une preuve peut être trouvée dans [10].

Illustration 1. Comme conséquence, on obtient

$$\cos^2(\theta_k) \|\nabla f_k\|^2 \xrightarrow[k \rightarrow \infty]{} 0.$$

Si l'on suppose qu'il existe $\delta > 0$ tel que

$$\cos(\theta_k) \geq \delta > 0, \quad \forall k,$$

alors nécessairement

$$\|\nabla f_k\| \xrightarrow[k \rightarrow \infty]{} 0.$$

Définition 2.21. On dit qu'un algorithme est *globalement convergent* s'il vérifie

$$\|\nabla f_k\| \rightarrow 0$$

pour toute suite d'itérés (x_k) produite à partir d'un point de départ x_0 donné. En l'absence d'information sur la Hessienne, c'est le meilleur résultat global qu'on puisse espérer : on garantit la convergence vers un *point stationnaire*, pas nécessairement un minimiseur.

Illustration 2 : cas de Newton. On rappelle la direction de Newton

$$d_k = \eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k,$$

en supposant $\nabla^2 f_k \succ 0$. Supposons que le conditionnement de la Hessienne est uniformément borné, c'est-à-dire qu'il existe $M > 0$ tel que

$$\|\nabla^2 f_k\| \cdot \|(\nabla^2 f_k)^{-1}\| \leq M, \quad \forall k.$$

Alors on peut montrer que $\cos(\theta_k)$ est uniformément borné en dessous par une constante strictement positive.

En effet,

$$\cos(\theta_k) = \frac{-\nabla f_k^\top d_k}{\|\nabla f_k\| \|d_k\|} = \frac{\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k}{\|\nabla f_k\| \|(\nabla^2 f_k)^{-1} \nabla f_k\|}.$$

On a

$$\|(\nabla^2 f_k)^{-1} \nabla f_k\| \leq \|(\nabla^2 f_k)^{-1}\| \|\nabla f_k\|.$$

D'où

$$\cos(\theta_k) \geq \frac{\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k}{\|\nabla f_k\|^2 \|(\nabla^2 f_k)^{-1}\|}.$$

En utilisant la décomposition de Cholesky $\nabla^2 f_k = LL^\top$, on obtient

$$\nabla f_k^\top (\nabla^2 f_k)^{-1} \nabla f_k = \|L^{-1} \nabla f_k\|^2 \geq \frac{\|\nabla f_k\|^2}{\|L\|^2}.$$

Par ailleurs, pour la norme spectrale, $\|L\|^2 = \|\nabla^2 f_k\|$. On déduit alors

$$\cos(\theta_k) \geq \frac{1}{\|\nabla^2 f_k\| \|(\nabla^2 f_k)^{-1}\|} \geq \frac{1}{M} > 0.$$

Par le théorème de Zoutendijk, on en conclut que $\|\nabla f_k\| \rightarrow 0$.

Remarque 2.22. Pour certains algorithmes, on ne prouve que

$$\liminf_{k \rightarrow \infty} \|\nabla f_k\| = 0,$$

ce qui signifie qu'au moins une sous-suite des itérés converge vers un point stationnaire.

2.4.4 Vitesse de convergence de la descente de gradient

Cas quadratique. Considérons

$$f(x) = \frac{1}{2} x^\top Q x - b^\top x,$$

où Q est symétrique et définie positive.

Théorème 2.23 (Luenberger). *Soit (x_k) la suite générée par la méthode de plus forte pente (gradient) avec recherche linéaire exacte appliquée à f . Alors*

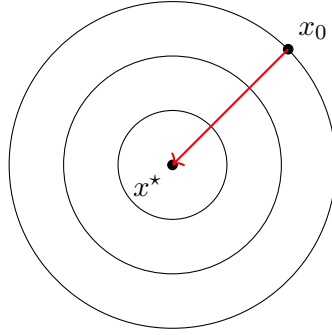
$$\|x_{k+1} - x^*\|_Q \leq \left(\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} \right) \|x_k - x^*\|_Q,$$

où x^* est le minimiseur de f , $\|x\|_Q = \sqrt{x^\top Q x}$, et $\lambda_{\min} \leq \dots \leq \lambda_{\max}$ sont les valeurs propres de Q .

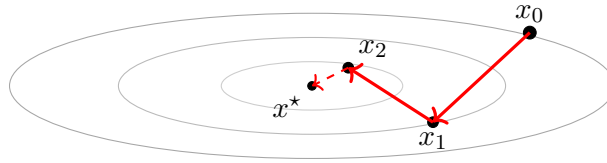
Exemple 2.24. Si $Q = \lambda I$ pour un certain $\lambda > 0$, alors

$$\frac{\lambda_{\max} - \lambda_{\min}}{\lambda_{\max} + \lambda_{\min}} = 0,$$

et la convergence se fait en une seule itération.



(a) Valeurs propres égales ($Q = \lambda I$) : convergence directe en une itération.



(b) Valeurs propres très différentes ($\kappa \gg 1$) : convergence lente en zigzag.

FIGURE 2.6 – Effet du conditionnement de Q sur la descente de gradient.

Cas non linéaire :

Théorème 2.25. Supposons que f est deux fois différentiable et que les itérés générés par la méthode de plus forte pente convergent vers un point x^* tel que la Hessienne $\nabla^2 f(x^*)$ soit définie positive. Soient $\lambda_1 \leq \dots \leq \lambda_m$ les valeurs propres de $\nabla^2 f(x^*)$ et soit

$$\omega \in \left(\frac{\lambda_m - \lambda_1}{\lambda_m + \lambda_1}, 1 \right).$$

Alors, pour k suffisamment grand,

$$f(x_{k+1}) - f(x^*) \leq \omega^2 (f(x_k) - f(x^*)).$$

Exemple 2.26. Supposons que

$$f(x_0) = 1, \quad f(x^*) = 0,$$

et que

$$\kappa(\nabla^2 f(x^*)) = \|\nabla^2 f(x^*)\| \|(\nabla^2 f(x^*))^{-1}\| = \frac{\lambda_m}{\lambda_1} = 800.$$

Alors la constante de convergence linéaire est

$$\omega = \frac{800 - 1}{800 + 1}.$$

Après 1000 itérations,

$$f(x_{1001}) \approx \omega^{2000} f(x_0) \approx 0,082.$$

Définition 2.27 (Vitesse de convergence). Soit (x_k) une suite convergeant vers x^* .

- On dit que (x_k) converge *Q-linéairement* vers x^* s'il existe $\omega \in (0, 1)$ et k_0 tels que

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \leq \omega, \quad \forall k \geq k_0.$$

- La convergence est *Q-superlinéaire* si

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|} \xrightarrow[k \rightarrow \infty]{} 0.$$

- La convergence est *Q-quadratique* s'il existe $M > 0$ et k_0 tels que

$$\frac{\|x_{k+1} - x^*\|}{\|x_k - x^*\|^2} \leq M, \quad \forall k \geq k_0.$$

2.4.5 Méthode de Newton : convergence quadratique

Théorème 2.28 (Convergence quadratique de Newton). *Supposons que f est de classe $\mathcal{C}^2$ et que $\nabla^2 f$ est Lipschitzienne dans un voisinage d'une solution x^* vérifiant les conditions suffisantes du second ordre (cf. [Théorème 2.8](#)). Considérons la méthode de Newton*

$$x_{k+1} = x_k - (\nabla^2 f(x_k))^{-1} \nabla f(x_k).$$

Alors :

1. si x_0 est assez proche de x^* , la suite (x_k) converge vers x^* ;
2. la convergence est quadratique :

$$\|x_{k+1} - x^*\| \leq C \|x_k - x^*\|^2$$

pour une constante $C > 0$ et k assez grand ;

3. les normes de gradient satisfont également une convergence quadratique :

$$\|\nabla f(x_{k+1})\| \leq C' \|\nabla f(x_k)\|^2.$$

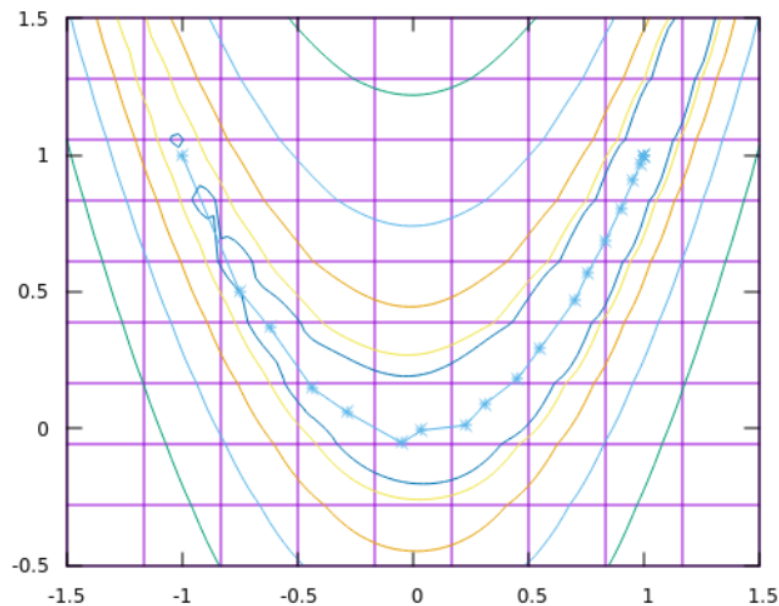


FIGURE 2.7 – Itérations de la méthode de Newton sur les courbes de niveau de la fonction de Rosenbrock.

Points clés à observer :

- **Trajectoire directe** : Contrairement à la descente de gradient (Figure 2.5), Newton atteint le minimum en très peu d'itérations.
- **Utilisation de la Hessienne** : L'information de courbure permet de « corriger » la direction du gradient pour suivre le fond de la vallée.
- **Convergence quadratique** : Proche du minimum, chaque itération double le nombre de chiffres significatifs corrects (théorème 2.28).
- **Coût** : Chaque itération nécessite le calcul et l'inversion de la Hessienne, ce qui peut être prohibitif en grande dimension.

2.4.6 Modification de la Hessienne pour Newton

L'efficacité de la méthode de Newton repose sur la positivité de la Hessienne. Loin de la solution, il peut arriver que $\nabla^2 f_k$ ne soit pas définie positive, de sorte que la direction

$$\eta_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k$$

ne soit pas une direction de descente.

On introduit alors une matrice B_k qui joue le rôle de Hessienne modifiée, et on résout

$$B_k \eta_k = -\nabla f_k.$$

Algorithm 4: Newton avec recherche linéaire et Hessienne modifiée**Input:** an initial point x_0

- 1 **for** $k = 0, 1, \dots$ **do**
- 2 Factorize the matrix $B_k = \nabla^2 f(x_k) + E_k$, where $E_k = 0$ if $\nabla^2 f(x_k)$ is sufficiently positive definite. Otherwise, E_k is chosen to ensure that B_k is sufficiently positive definite;
- 3 Solve $B_k p_k = -\nabla f(x_k)$;
- 4 Set $x_{k+1} \leftarrow x_k + \alpha_k p_k$, where α_k satisfies the Wolfe, Goldstein, or Armijo backtracking conditions;

Définition 2.29 (Propriété de factorisation modifiée bornée (BMFP)). On dit que la suite (B_k) satisfait la *propriété de factorisation modifiée bornée* (BMFP) s'il existe $C > 0$ tel que

$$\kappa(B_k) = \|B_k\| \cdot \|B_k^{-1}\| \leq C, \quad \forall k.$$

Théorème 2.30 (Moré–Sorensen). *Supposons que f est de classe $\mathcal{C}^2$ sur un ouvert $D \subset \mathbb{R}^n$. Supposons que l'ensemble de niveau*

$$\{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$$

est compact. Si la suite (B_k) utilisée dans l'algorithme de Newton modifié satisfait la propriété BMFP, alors

$$\|\nabla f_k\| \xrightarrow[k \rightarrow \infty]{} 0.$$

Exemples de constructions de B_k .

- **Décomposition spectrale.** On peut diagonaliser $\nabla^2 f_k$, remplacer les valeurs propres non positives par une petite constante $\delta > 0$ (ou éventuellement renverser le signe des valeurs propres négatives), puis reconstruire la matrice.
- **Ajout d'un multiple de l'identité.** On cherche $\tau > 0$ tel que $\nabla^2 f_k + \tau I$ soit définie positive.

Algorithm 5: Cholesky avec ajout d'un multiple de l'identité

- 1 Choose $\beta > 0$;
- 2 **if** $\min_i(a_{i,i}) > 0$ **then**
- 3 Set $\tau_0 \leftarrow 0$;
- 4 **else**
- 5 $\tau_0 \leftarrow -\min_i(a_{i,i}) + \beta$;
- 6 **for** $k = 0, 1, \dots$ **do**
- 7 Attempt to apply the Cholesky algorithm to obtain $LL' = A + \tau_k I$;
- 8 **if** the factorization is completed successfully **then**
- 9 **stop and return** L ;
- 10 **else**
- 11 $\tau_{k+1} \leftarrow \max(2\tau_k, \beta)$;

Factorisation de Cholesky modifiée. Toute matrice symétrique définie positive A peut s'écrire $A = LDL^\top$ où D est diagonale et L est triangulaire inférieure à diagonale unité. On peut modifier l'algorithme de Cholesky pour assurer que les éléments de D restent strictement

positifs, ce qui fournit une méthode stable de construction de matrices B_k définies positives. Voir par exemple [2] ou [8] pour plus de détails.

Chapitre 3

Méthodes du gradient conjugué

Dans ce chapitre, nous présentons les *méthodes du gradient conjugué*, qui constituent une famille d'algorithmes très efficaces pour résoudre des problèmes quadratiques de grande dimension, ainsi que leurs extensions non linéaires.

Les méthodes du gradient conjugué ont été introduites dans les années 1950 par Hestenes et Stiefel [3] pour résoudre des systèmes linéaires

$$Ax = b,$$

où $A \in \mathbb{R}^{n \times n}$ est symétrique définie positive. Plus tard, Fletcher et Reeves [1] ont proposé une extension *non linéaire* pour la minimisation d'une fonction générale $f : \mathbb{R}^n \rightarrow \mathbb{R}$.

Par rapport à la descente de gradient :

- elles ne nécessitent pas de stocker la matrice A ;
- elles convergent beaucoup plus rapidement sur les problèmes quadratiques bien conditionnés ;
- elles sont particulièrement adaptées aux grandes dimensions.

3.1 Gradient conjugué linéaire

3.1.1 Problème quadratique associé

Considérons le système linéaire

$$Ax = b,$$

où A est symétrique définie positive. Ce problème est équivalent à la minimisation de

$$q(x) = \frac{1}{2}x^\top Ax - b^\top x. \quad (3.1)$$

Son gradient vaut

$$\nabla q(x) = Ax - b,$$

et donc x^* minimise q si et seulement si $Ax^* = b$.

Pour une direction p_k , on minimise $q(x_k + \alpha p_k)$:

$$\psi(\alpha) = q(x_k + \alpha p_k).$$

La condition $\psi'(\alpha) = 0$ donne

$$p_k^\top A(x_k + \alpha p_k) - p_k^\top b = 0.$$

Comme $r_k = Ax_k - b$:

$$\alpha_k = -\frac{p_k^\top r_k}{p_k^\top A p_k}.$$

3.1.2 Directions A -conjuguées

Définition 3.1. Deux vecteurs p_i, p_j non nuls sont A -conjugués si

$$p_i^\top A p_j = 0.$$

Une famille $(p_0, \dots, p_m)$ est A -conjuguée si tous ses vecteurs sont deux à deux A -conjugués.

Une famille A -conjuguée constitue une base dans laquelle la minimisation devient indépendante entre composantes : une correction dans une direction ne dégrade pas celles déjà faites.

3.1.3 Méthode des directions conjuguées

On suppose données des directions A -conjuguées $(\eta_0, \dots, \eta_{n-1})$. On pose :

$$x_{k+1} = x_k + \alpha_k \eta_k, \quad \alpha_k = -\frac{\eta_k^\top r_k}{\eta_k^\top A \eta_k}.$$

Théorème 3.2 (Minimisation sur sous-espaces croissants). Pour $k \geq 1$:

$$r_k^\top \eta_i = 0 \quad (i < k),$$

et

$$x_k = \arg \min_{x \in x_0 + \text{span}(\eta_0, \dots, \eta_{k-1})} q(x).$$

3.1.4 Algorithme du gradient conjugué linéaire

On note $r_k = Ax_k - b$.

On définit les directions par

$$p_0 = -r_0, \quad p_k = -r_k + \beta_k p_{k-1}.$$

La condition A -conjugaison $p_k^\top A p_{k-1} = 0$ impose

$$\beta_k = \frac{r_k^\top A p_{k-1}}{p_{k-1}^\top A p_{k-1}} \quad (\text{Hestenes-Stiefel [3] linéaire}).$$

Dans la pratique, on utilise souvent la forme équivalente

$$\beta_k = \frac{r_k^\top r_k}{r_{k-1}^\top r_{k-1}}.$$

Algorithm 6: Version préliminaire du Gradient Conjugué

```

1 Given  $x_0$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ,  $p_0 \leftarrow -r_0$ ,  $k \leftarrow 0$ ;
3 while  $r_k \neq 0$  do
4    $\alpha_k \leftarrow -\frac{r_k^\top p_k}{p_k^\top A p_k}$ ;
5    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
6    $r_{k+1} \leftarrow Ax_{k+1} - b$ ;
7    $\beta_{k+1} \leftarrow \frac{r_{k+1}^\top A p_k}{p_k^\top A p_k}$ ;
8    $p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Algorithm 7: Gradient Conjugué (version pratique)

```

1 Given  $x_0$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ,  $p_0 \leftarrow -r_0$ ,  $k \leftarrow 0$ ;
3 while  $r_k \neq 0$  do
4    $\alpha_k \leftarrow \frac{r'_k r_k}{p'_k A p_k}$ ;
5    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
6    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ ;
7    $\beta_{k+1} \leftarrow \frac{r'_{k+1} r_{k+1}}{r'_k r_k}$ ;
8    $p_{k+1} \leftarrow -r_{k+1} + \beta_{k+1} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Remarque 3.3. La version pratique évite de recalculer Ax_{k+1} en utilisant la relation de récurrence $r_{k+1} = r_k + \alpha_k A p_k$. De plus, les formules pour α_k et β_{k+1} ne nécessitent que des produits scalaires.

Les propriétés suivantes sont vérifiées :

$$r_i^\top r_j = 0, \quad p_i^\top A p_j = 0 \quad (i \neq j).$$

3.1.5 Borne d'erreur et conditionnement

Si $\lambda_{\min}$ et $\lambda_{\max}$ sont les valeurs propres extrêmes de A , alors

$$\frac{\|x_k - x^*\|_A}{\|x_0 - x^*\|_A} \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k, \quad \kappa(A) = \frac{\lambda_{\max}}{\lambda_{\min}}.$$

3.2 Interprétation géométrique

3.2.1 Niveaux d'une fonction quadratique

Les ensembles de niveau de (3.1) sont des ellipses centrées en x^* . Le gradient conjugué suit des directions successivement A -conjuguées, ce qui permet de “corriger” l'erreur dans chacune des directions de manière indépendante.

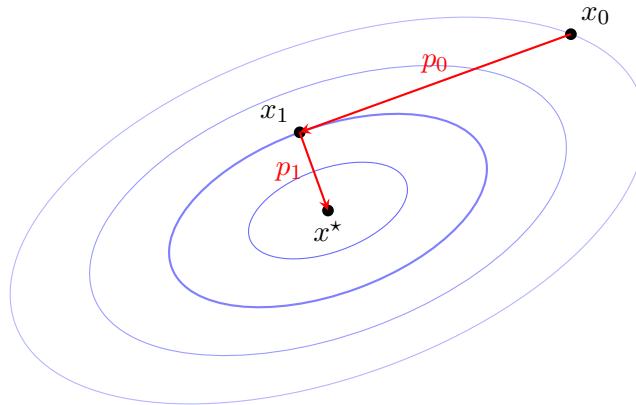


FIGURE 3.1 – Interprétation géométrique : niveaux de q et directions conjuguées.

En dimension deux, le gradient conjugué atteint exactement x^* en deux itérations.

3.3 Comparaison avec la descente de gradient

La descente de gradient zigzague sur des ellipses allongées, alors que le gradient conjugué choisit des directions qui “diagonaliseront” progressivement la matrice A .

3.4 Préconditionnement

Lorsque A est mal conditionnée, on introduit une matrice SPD M approchant A ou A^{-1} . On considère le système équivalent :

$$M^{-1}Ax = M^{-1}b.$$

On introduit $z_k = M^{-1}r_k$.

Algorithm 8: Gradient Conjugué Préconditionné

```

1 Given  $x_0$ , preconditioner  $M$ ;
2 Set  $r_0 \leftarrow Ax_0 - b$ ;
3 Solve  $My_0 = r_0$  for  $y_0$ ;
4 Set  $p_0 \leftarrow -y_0$ ,  $k \leftarrow 0$ ;
5 while  $r_k \neq 0$  do
6    $\alpha_k \leftarrow \frac{r'_k y_k}{p'_k A p_k}$ ;
7    $x_{k+1} \leftarrow x_k + \alpha_k p_k$ ;
8    $r_{k+1} \leftarrow r_k + \alpha_k A p_k$ ;
9   Solve  $My_{k+1} = r_{k+1}$ ;
10   $\beta_{k+1} \leftarrow \frac{r'_{k+1} y_{k+1}}{r'_k y_k}$ ;
11   $p_{k+1} \leftarrow -y_{k+1} + \beta_{k+1} p_k$ ;
12   $k \leftarrow k + 1$ ;

```

3.5 Gradient conjugué non linéaire

On cherche à minimiser $f(x)$, supposée $\mathcal{C}^1$. On note $g_k = \nabla f(x_k)$.

Algorithm 9: Méthode de Fletcher-Reeves

```

1 Given  $x_0$ ;
2 Evaluate  $f_0 = f(x_0)$ ,  $\nabla f_0 = \nabla f(x_0)$ ;
3 Set  $p_0 = -\nabla f_0$ ,  $k \leftarrow 0$ ;
4 while  $\nabla f_k \neq 0$  do
5   Compute  $\alpha_k$  and set  $x_{k+1} = x_k + \alpha_k p_k$ ;
6   Evaluate  $\nabla f_{k+1}$ ;
7    $\beta_{k+1}^{FR} \leftarrow \frac{\nabla f'_{k+1} \nabla f_{k+1}}{\nabla f'_k \nabla f_k}$ ;
8    $p_{k+1} \leftarrow -\nabla f_{k+1} + \beta_{k+1}^{FR} p_k$ ;
9    $k \leftarrow k + 1$ ;

```

Théorème 3.4. *Si la recherche linéaire satisfait les conditions de Wolfe et si f est régulière, alors*

$$\liminf_{k \rightarrow \infty} \|g_k\| = 0.$$

3.6 Résumé

- Le gradient conjugué linéaire est optimal pour les problèmes quadratiques SPD.
- Les directions sont A -conjuguées et les résidus orthogonaux.
- Le préconditionnement améliore considérablement la convergence.
- Le gradient conjugué non linéaire est efficace et peu coûteux.
- Le choix de β_k influence fortement la performance.

3.6.1 Propriétés des itérés et sous-espaces de Krylov

Les propriétés suivantes justifient l'efficacité théorique de la méthode.

Théorème 3.5. *Supposons que le k -ème itéré n'est pas égal à la solution x^* . Alors :*

(i) *Les résidus sont orthogonaux :*

$$\forall i = 0, \dots, k-1, \quad r_k^\top r_i = 0.$$

(ii) *L'espace engendré par les résidus est un sous-espace de Krylov de degré k pour r_0 :*

$$\text{span}(r_0, \dots, r_k) = \text{span}(r_0, Ar_0, \dots, A^k r_0).$$

(iii) *L'espace engendré par les directions de descente coïncide avec cet espace :*

$$\text{span}(p_0, \dots, p_k) = \text{span}(r_0, Ar_0, \dots, A^k r_0).$$

(iv) *Les directions sont A -conjuguées par rapport aux précédentes :*

$$\forall i = 0, \dots, k-1, \quad p_k^\top A p_i = 0.$$

Preuve. Référence : [10]. □

Conséquences. D'après le point (iv), le gradient conjugué converge vers x^* en au plus n itérations (en arithmétique exacte), car il ne peut y avoir plus de n directions non nulles mutuellement A -conjuguées dans $\mathbb{R}^n$.

Remarque 3.6. Il est essentiel pour ces propriétés que l'initialisation soit $p_0 = -r_0 = -(Ax_0 - b)$.

3.6.2 Dérivation de la version efficace (Standard CG)

Jusqu'à présent, nous avons la formule générale pour α_k . Nous pouvons la simplifier pour réduire le coût de calcul.

On sait que :

$$\alpha_k = \frac{-r_k^\top p_k}{p_k^\top A p_k}.$$

D'après le théorème de minimisation sur sous-espace (ESM), on sait que $r_k^\top p_i = 0$ pour tout $i = 0, \dots, k-1$. Comme $p_k = -r_k + \beta_k p_{k-1}$, on a :

$$-r_k^\top p_k = -r_k^\top (-r_k + \beta_k p_{k-1}) = r_k^\top r_k - \underbrace{\beta_k r_k^\top p_{k-1}}_{=0}.$$

Ainsi, la formule simplifiée pour le pas est :

$$\boxed{\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}}.$$

Ensuite, pour le coefficient β_{k+1} , nous partons de la définition assurant la A -conjugaison :

$$\beta_{k+1} = \frac{r_{k+1}^\top A p_k}{p_k^\top A p_k}.$$

Nous utilisons la relation de récurrence du résidu : $r_{k+1} = r_k + \alpha_k A p_k$, ce qui implique $A p_k = \frac{1}{\alpha_k}(r_{k+1} - r_k)$.

En substituant dans le numérateur :

$$r_{k+1}^\top A p_k = \frac{1}{\alpha_k} r_{k+1}^\top (r_{k+1} - r_k) = \frac{1}{\alpha_k} (r_{k+1}^\top r_{k+1} - \underbrace{r_{k+1}^\top r_k}_{=0}) = \frac{r_{k+1}^\top r_{k+1}}{\alpha_k}.$$

De même pour le dénominateur, on sait que $p_k^\top A p_k = \frac{r_k^\top r_k}{\alpha_k}$.

Ainsi, le rapport se simplifie remarquablement :

$$\beta_{k+1} = \frac{\frac{r_{k+1}^\top r_{k+1}}{\alpha_k}}{\frac{r_k^\top r_k}{\alpha_k}} = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}.$$

Remarques sur l'Algorithme 7 (Standard CG) :

- Nous avons éliminé les produits matrice-vecteur dans le calcul des coefficients scalaires (ils ne restent que dans la mise à jour $A p_k$).
- Nous n'avons besoin de stocker les valeurs de x , p et r que sur deux itérations.

3.7 Vitesse de convergence

Théorème 3.7 ([10]). *Si la matrice A possède seulement r valeurs propres distinctes, alors la convergence se produit en au plus r itérations.*

Une borne plus précise de l'erreur est donnée par le théorème suivant, qui dépend de la distribution des valeurs propres.

Théorème 3.8 ([7]). *Si A a pour valeurs propres $\lambda_1 \leq \dots \leq \lambda_n$, alors :*

$$\|x_{k+1} - x^\star\|_A^2 \leq \left(\frac{\lambda_{n-k} - \lambda_1}{\lambda_{n-k} + \lambda_1} \right)^2 \|x_0 - x^\star\|_A^2.$$

Ce résultat suggère que le gradient conjugué élimine l'influence des valeurs propres extrêmes au fur et à mesure des itérations.

3.7.1 Illustration : Effet de la distribution des valeurs propres

Considérons une matrice A possédant 2 « clusters » (groupes) de valeurs propres :

- Un grand groupe de $n - m$ valeurs propres proches de λ_1 .
- Un petit groupe de m valeurs propres (outliers) situées plus loin.

D'après le théorème de Luenberger, après m itérations, on élimine l'influence des m valeurs propres extrêmes. L'erreur à l'étape $m + 1$ satisfait alors :

$$\|x_{m+1} - x^\star\|_A^2 \leq \left(\frac{\lambda_{n-m} - \lambda_1}{\lambda_{n-m} + \lambda_1} \right)^2 \|x_0 - x^\star\|_A^2.$$

Grossièrement, cela signifie que la convergence effective démarre après avoir traité les quelques valeurs propres isolées.

3.7.2 Comparaison des taux de convergence

Il est instructif de comparer la borne d'erreur du Gradient Conjugué (CG) avec celle de la méthode de la Plus Grande Pente (Steepest Descent, SD).

On note le conditionnement $\kappa(A) = \|A\| \cdot \|A^{-1}\| = \frac{\lambda_n}{\lambda_1}$.

- **Pour le Gradient Conjugué (CG) :**

$$\|x_k - x^*\|_A \leq 2 \left(\frac{\sqrt{\kappa(A)} - 1}{\sqrt{\kappa(A)} + 1} \right)^k \|x_0 - x^*\|_A.$$

- **Pour la Plus Grande Pente (SD) :**

$$\|x_k - x^*\|_A \leq \left(\frac{\kappa(A) - 1}{\kappa(A) + 1} \right)^k \|x_0 - x^*\|_A.$$

Remarque 3.9. Lorsque $\kappa(A)$ est grand (problème mal conditionné), le terme $\frac{\kappa(A)-1}{\kappa(A)+1}$ est très proche de 1, ce qui implique une convergence très lente (du "bruit" numérique). En revanche, le terme avec la racine carrée $\sqrt{\kappa(A)}$ dans le CG est beaucoup plus favorable, assurant une convergence plus rapide.

3.8 Préconditionnement

3.8.1 Idée générale

L'idée du préconditionnement est de « transformer » la distribution des valeurs propres de A pour accélérer la convergence.

Nous effectuons un changement de variable de x vers $\hat{x}$:

$$\hat{x} = Cx \iff x = C^{-1}\hat{x},$$

où C est une matrice inversible (non-singulière).

La fonction objective quadratique $\phi(x)$ devient alors une fonction de $\hat{x}$, notée $\hat{\phi}(\hat{x})$:

$$\hat{\phi}(\hat{x}) = \phi(C^{-1}\hat{x}) = \frac{1}{2}(C^{-1}\hat{x})^\top A(C^{-1}\hat{x}) - b^\top(C^{-1}\hat{x}).$$

En développant, on obtient :

$$\hat{\phi}(\hat{x}) = \frac{1}{2}\hat{x}^\top \left[(C^{-1})^\top A C^{-1} \right] \hat{x} - \left((C^{-1})^\top b \right)^\top \hat{x}.$$

3.8.2 Choix de la matrice de préconditionnement

La fonction $\hat{\phi}$ est toujours quadratique. Si nous utilisons le Gradient Conjugué pour minimiser $\hat{\phi}$, la performance dépendra désormais des valeurs propres de la nouvelle matrice coefficient :

$$\hat{A} = (C^{-1})^\top A C^{-1}.$$

Il faut donc choisir la matrice C telle que :

1. Le conditionnement de $(C^{-1})^\top A C^{-1}$ soit plus faible que celui de A (c'est-à-dire $\kappa(\hat{A}) < \kappa(A)$).
2. Ou que les valeurs propres de $(C^{-1})^\top A C^{-1}$ soient bien groupées (clustered).

Dans la pratique, on ne forme pas explicitement $\hat{A}$. On pose souvent $M = C^\top C$ (le préconditionneur) et on adapte l'algorithme CG pour résoudre le système $M^{-1}Ax = M^{-1}b$ implicitement.

3.8.3 Mise en œuvre pratique

En pratique, nous n'avons besoin d'utiliser la matrice C qu'à travers le préconditionneur M défini par :

$$M = C^\top C.$$

Ceci mène à l'Algorithme 8 (Gradient Conjugué Préconditionné).

Bilan (Balance). L'utilisation du préconditionnement implique un compromis :

(+) Le préconditionnement améliore la distribution des valeurs propres (réduisant le nombre d'itérations).

(-) Il introduit des résolutions de systèmes linéaires $My = r$ à chaque étape.

Le but est donc de choisir M pour que le système $My = r$ soit peu coûteux à résoudre.

3.8.4 Exemple : Préconditionnement de Cholesky Incomplet

Une méthode classique repose sur la décomposition de Cholesky. Pour une matrice A symétrique définie positive, il existe une matrice triangulaire inférieure L telle que $A = LL^\top$.

Idée : On cherche une approximation de L par une matrice *creuse* (sparse) $\tilde{L}$ telle que :

$$A \approx \tilde{L}\tilde{L}^\top.$$

On choisit alors $C = \tilde{L}^\top$. Le préconditionneur devient $M = (\tilde{L}^\top)^\top \tilde{L}^\top = \tilde{L}\tilde{L}^\top$.

La matrice du système transformé est alors :

$$(C^{-1})^\top AC^{-1} = \tilde{L}^{-1}A(\tilde{L}^{-1})^\top \approx I.$$

La distribution des valeurs propres de cette matrice est généralement favorable.

Résolution du système. Avec ce choix, résoudre $My = r$ revient à résoudre $\tilde{L}\tilde{L}^\top y = r$. Cela s'effectue par deux résolutions de systèmes triangulaires consécutives (très rapides) :

1. Résoudre $\tilde{L}z = r$ pour trouver z (descente).
2. Résoudre $\tilde{L}^\top y = z$ pour trouver y (remontée).

Voir [6] pour plus de détails.

Chapitre 4

Méthodes du gradient conjugué non linéaire

4.1 Introduction

Idée : Adapter le gradient conjugué linéaire (efficace pour les problèmes quadratiques) aux fonctions objectives non linéaires générales.

4.2 Méthode de Fletcher et Reeves

Fletcher et Reeves [1] ont proposé deux changements majeurs par rapport à l'algorithme linéaire pour minimiser une fonction f quelconque.

4.2.1 1. Recherche linéaire (Line Search)

Contrairement au cas quadratique où le pas α_k est donné par une formule exacte, ici on substitue la formule :

$$\alpha_k = \frac{r_k^\top r_k}{p_k^\top A p_k}$$

par une **recherche linéaire** pour approximer le minimum de f le long de la direction p_k .

4.2.2 2. Substitution du résidu par le gradient

Dans le cas linéaire, le résidu r_k correspondait au gradient de la fonction quadratique ϕ . Pour le cas général, on substitue le gradient de ϕ par le gradient de f :

$$r_k = \nabla \phi(x_k) \longrightarrow \nabla f_k.$$

4.2.3 Formule du β_{FR}

La formule pour β utilisée dans le cas linéaire ($\beta = \frac{r_{k+1}^\top r_{k+1}}{r_k^\top r_k}$) devient la formule de Fletcher-Reeves :

$$\beta_{FR} = \frac{\nabla f_{k+1}^\top \nabla f_{k+1}}{\nabla f_k^\top \nabla f_k}.$$

Ceci définit l'Algorithme 9.

Remarque 4.1. Il n'y a pas de signe moins dans cette formule.

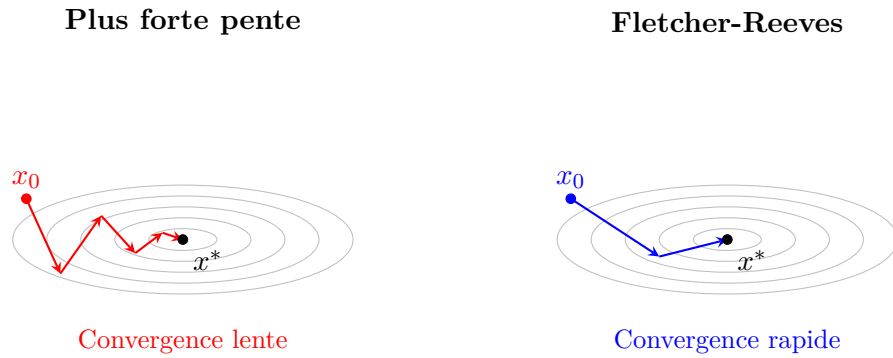


FIGURE 4.1 – Comparaison de la plus forte pente (gauche) et Fletcher-Reeves (droite) sur des courbes de niveau elliptiques. La méthode de Fletcher-Reeves exploite les directions conjuguées pour converger plus rapidement.

La figure 4.1 illustre l'avantage du gradient conjugué non linéaire (Fletcher-Reeves) par rapport à la plus forte pente. Sur une fonction quadratique mal conditionnée (ellipses allongées), la plus forte pente zigzague en alternant entre directions orthogonales, tandis que Fletcher-Reeves exploite les directions conjuguées pour atteindre le minimum en moins d'itérations.

4.2.4 Illustration sur la fonction de Rosenbrock

La **fonction de Rosenbrock** $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$ est un cas test classique pour les algorithmes d'optimisation. Elle possède un minimum unique en $(1, 1)$ mais présente une vallée très étroite et courbée qui rend la convergence difficile.

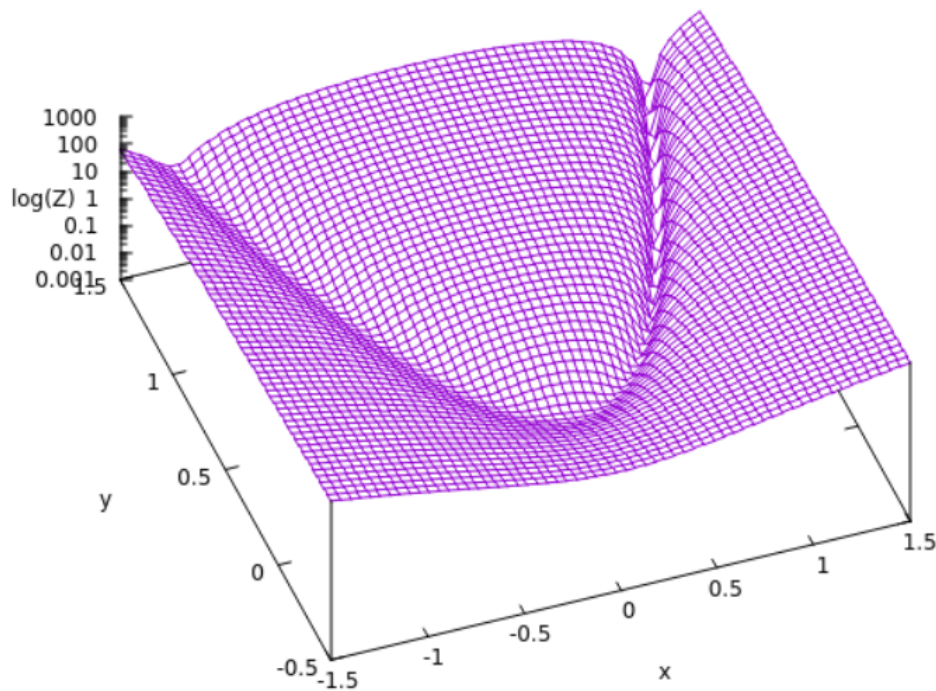


FIGURE 4.2 – La fonction de Rosenbrock $f(x, y) = (1 - x)^2 + 100(y - x^2)^2$, ayant un minimum unique en $(1, 1)$.

Points clés à observer :

- **Forme de la surface :** La vallée parabolique (« banana valley ») est clairement visible. Le minimum global (1, 1) se trouve au fond de cette vallée étroite.
- **Échelle logarithmique :** L'axe vertical utilise $\log(Z)$ pour visualiser les grandes variations de la fonction.
- **Difficulté d'optimisation :** Atteindre le fond de la vallée est facile, mais progresser le long de la vallée vers le minimum est difficile pour de nombreuses méthodes.

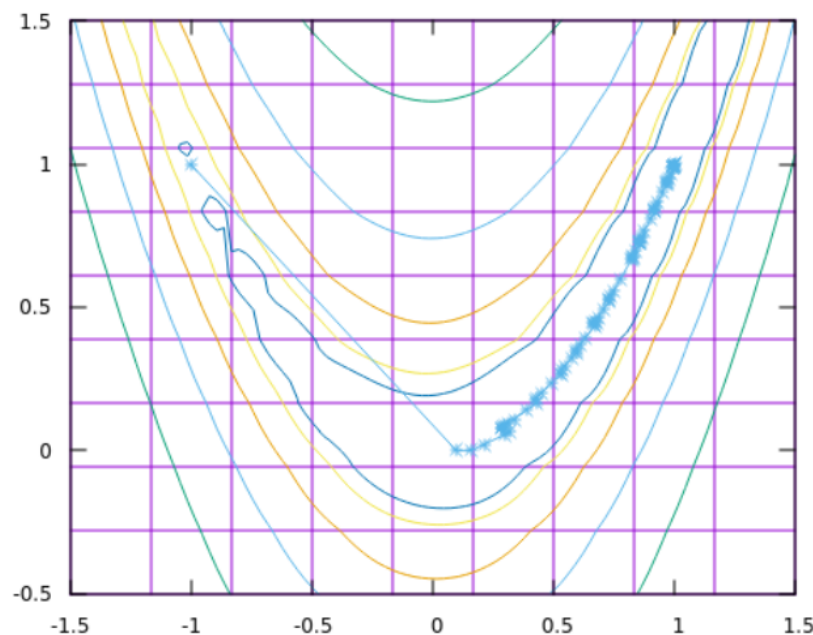


FIGURE 4.3 – Itérations de la méthode Fletcher-Reeves (gradient conjugué non linéaire) sur les courbes de niveau de la fonction de Rosenbrock.

Points clés à observer :

- **Trajectoire efficace :** La méthode converge vers le minimum (1, 1) en exploitant les directions conjuguées, sans utiliser explicitement la Hessienne.
- **Compromis :** Fletcher-Reeves offre une convergence plus rapide que la descente de gradient, tout en évitant le coût de calcul de la Hessienne (nécessaire pour Newton).
- **Adaptation au cas non-linéaire :** Bien que la théorie du gradient conjugué soit optimale pour les fonctions quadratiques, la méthode reste efficace sur cette fonction fortement non-quadratique.

4.2.5 Propriété de descente globale

Contrairement au cas quadratique, la direction p_k générée par Fletcher-Reeves n'est pas automatiquement une direction de descente.

On a la relation :

$$\nabla f_k^\top p_k = -\|\nabla f_k\|^2 + \beta_{FR} \nabla f_k^\top p_{k-1}.$$

Si le second terme est grand, la direction peut remonter. Pour garantir la descente, il faut imposer des conditions strictes sur la recherche linéaire.

Lemme 4.2 (Condition de descente). *Supposons que le pas α_k satisfait les **conditions de Wolfe fortes** avec un paramètre $0 < c_2 < \frac{1}{2}$. Alors, la direction p_k est toujours une direction de descente et satisfait l'encadrement suivant pour tout k :*

$$-\frac{1}{1-c_2} \leq \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2} \leq \frac{2c_2-1}{1-c_2}.$$

Démonstration. Préliminaire. Considérons la fonction $t(\xi) = \frac{2\xi-1}{1-\xi}$. Cette fonction est croissante sur $[0, \frac{1}{2}]$. On a $t(0) = -1$ et $t(1/2) = 0$. Comme $0 < c_2 < 1/2$, on a :

$$-1 < \frac{2c_2-1}{1-c_2} < 0.$$

Initialisation ($k = 0$). On a $p_0 = -\nabla f_0$, donc $\nabla f_0^\top p_0 = -\|\nabla f_0\|^2$. Le rapport vaut :

$$\frac{\nabla f_0^\top p_0}{\|\nabla f_0\|^2} = -1.$$

L'inégalité est satisfaite car :

$$-\frac{1}{1-c_2} \leq -1 < \frac{2c_2-1}{1-c_2} \quad (\text{vérifié car } c_2 < 1/2).$$

Hérédité. Supposons le résultat vrai pour k . Calculons le rapport pour $k+1$:

$$\frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} = \frac{\nabla f_{k+1}^\top (-\nabla f_{k+1} + \beta_{k+1}^{FR} p_k)}{\|\nabla f_{k+1}\|^2} = -1 + \beta_{k+1}^{FR} \frac{\nabla f_{k+1}^\top p_k}{\|\nabla f_{k+1}\|^2}.$$

En utilisant la définition de $\beta_{FR} = \frac{\|\nabla f_{k+1}\|^2}{\|\nabla f_k\|^2}$, on simplifie :

$$\frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} = -1 + \frac{\nabla f_{k+1}^\top p_k}{\|\nabla f_k\|^2}.$$

La condition de courbure (seconde condition de Wolfe forte) impose :

$$|\nabla f_{k+1}^\top p_k| \leq c_2 |\nabla f_k^\top p_k|.$$

Comme p_k est une direction de descente (hypothèse de récurrence), $\nabla f_k^\top p_k < 0$. Ainsi $|\nabla f_k^\top p_k| = -\nabla f_k^\top p_k$. L'inégalité devient :

$$|\nabla f_{k+1}^\top p_k| \leq -c_2 \nabla f_k^\top p_k.$$

Cela implique :

$$c_2 \nabla f_k^\top p_k \leq \nabla f_{k+1}^\top p_k \leq -c_2 \nabla f_k^\top p_k.$$

En divisant par $\|\nabla f_k\|^2$ et en ajoutant -1 , on obtient l'encadrement :

$$-1 + c_2 \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2} \leq \frac{\nabla f_{k+1}^\top p_{k+1}}{\|\nabla f_{k+1}\|^2} \leq -1 - c_2 \frac{\nabla f_k^\top p_k}{\|\nabla f_k\|^2}.$$

En utilisant l'hypothèse de récurrence sur le terme de droite, on montre que l'encadrement est préservé pour $k+1$. $\square$

4.3 Méthode de Polak et Ribière

Une autre variante très populaire, souvent considérée comme plus efficace en pratique, est la méthode de Polak-Ribière (PR).

4.3.1 Définition et propriétés

Polak et Ribière ont proposé la formule suivante pour β :

$$\beta_{k+1}^{PR} = \frac{\nabla f_{k+1}^\top (\nabla f_{k+1} - \nabla f_k)}{\|\nabla f_k\|^2}.$$

Coïncidence avec FR. Si la fonction f est strictement quadratique, alors les gradients successifs sont orthogonaux ($\nabla f_{k+1}^\top \nabla f_k = 0$). Dans ce cas, le terme de soustraction s'annule et :

$$\beta_{k+1}^{PR} = \beta_{k+1}^{FR}.$$

PR coïncide donc avec FR et le gradient conjugué linéaire sur les problèmes quadratiques.

Problème de la descente. Cependant, contrairement à Fletcher-Reeves, même les conditions de Wolfe fortes ne garantissent pas que la direction p_k générée par Polak-Ribière soit une direction de descente.

4.3.2 Variante β^{PR+}

Pour garantir la génération de directions de descente, on utilise souvent la modification suivante :

$$\beta_{k+1}^{PR+} = \max(\beta_{k+1}^{PR}, 0).$$

Cela revient à réinitialiser la direction à celle de la plus grande pente ($p_{k+1} = -\nabla f_{k+1}$) lorsque β_{k+1}^{PR} devient négatif.

4.3.3 Remarques pratiques et vitesse de convergence

- **Performance :** En pratique, la méthode PR est souvent meilleure que FR pour les fonctions non linéaires générales et lorsque la recherche linéaire est inexacte.
- **Redémarrage (Restart) :** Une stratégie courante consiste à "redémarrer" l'algorithme toutes les n itérations en effectuant un pas de plus grande pente (Steepest Descent). Notons que ce pas de descente correspond d'ailleurs à l'initialisation de la méthode du gradient conjugué.
- **Vitesse de convergence :** Avec des redémarrages appropriés, on peut prouver une convergence quadratique en n étapes ("n-step quadratic convergence") :

$$\|x_{k+n} - x^*\| = O(\|x_k - x^*\|^2).$$

4.4 Analyse de la convergence globale

Nous étudions ici les conditions sous lesquelles le gradient descend vers zéro.

4.4.1 Hypothèses (Assumptions A)

Pour établir la convergence, nous posons les hypothèses suivantes sur la fonction f et l'ensemble de niveau $\mathcal{L} = \{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$:

1. L'ensemble de niveau $\mathcal{L}$ est borné.
2. f est continûment différentiable et son gradient est Lipschitzien dans un voisinage de $\mathcal{L}$.

4.4.2 Théorème d'Al-Baali

Ce résultat important établit la convergence globale de la méthode de Fletcher-Reeves sous les conditions de Wolfe fortes avec un paramètre c_2 suffisamment petit.

Théorème 4.3 (Al-Baali). *Sous les hypothèses A, si l'algorithme de Fletcher-Reeves est implémenté avec les conditions de Wolfe fortes et des paramètres satisfaisant :*

$$0 < c_1 < c_2 < \frac{1}{2},$$

alors la suite des gradients converge vers zéro au sens suivant :

$$\liminf_{k \rightarrow \infty} \|\nabla f_k\| = 0.$$

Chapitre 5

Méthodes Quasi-Newton

5.1 Introduction

Principe. L'objectif est d'éviter le calcul de la Hessienne en utilisant à la place une matrice symétrique définie positive.

Domaine d'application. Dans ce chapitre, nous nous concentrons sur des problèmes de taille petite à moyenne. En effet, ces méthodes nécessitent des résolutions de systèmes linéaires ; la question des grands problèmes est reportée au chapitre suivant, où on évitera de telles résolutions.

5.2 La méthode BFGS

La méthode de **Broyden, Fletcher, Goldfarb, Shanno** (BFGS) est la plus populaire des méthodes Quasi-Newton.

Considérons le modèle quadratique suivant de la fonction objectif f à l'itération k :

$$m_k(p) = f_k + \nabla f_k^\top p + \frac{1}{2} p^\top B_k p, \quad (5.1)$$

où B_k est une matrice symétrique définie positive qui est mise à jour à chaque itération.

Notons que le modèle vérifie les propriétés de consistance suivantes en $p = 0$:

- $m_k(0) = f_k = f(x_k)$
- $\nabla m_k(0) = \nabla f_k = \nabla f(x_k)$

Le minimiseur p_k du modèle quadratique m_k est obtenu en annulant son gradient :

$$\nabla m_k(p_k) = \nabla f_k + B_k p_k = 0. \quad (5.2)$$

Ceci implique que la direction de recherche est donnée par :

$$p_k = -B_k^{-1} \nabla f_k. \quad (5.3)$$

Une fois la direction p_k obtenue, nous effectuons une recherche linéaire (line search) pour déterminer le pas α_k .

5.3 La formule de mise à jour de Davidon, Fletcher, Powell (DFP)

Davidon (1959) a proposé de mettre à jour B_k pour prendre en compte le changement de courbure survenu depuis la dernière itération.

Considérons le nouveau modèle affiné en x_{k+1} :

$$m_{k+1}(p) = f_{k+1} + \nabla f_{k+1}^\top p + \frac{1}{2} p^\top B_{k+1} p. \quad (5.4)$$

Quelles sont les exigences raisonnables pour mettre à jour B_k ?

5.3.1 L'équation sécante

Par exemple, le gradient de m_{k+1} doit coïncider avec le vrai gradient de f aux points x_{k+1} et x_k .

1. En x_{k+1} (c'est-à-dire pour $p = 0$) :

$$\nabla m_{k+1}(0) = \nabla f_{k+1}.$$

Cette condition est vérifiée par définition.

2. En x_k (c'est-à-dire pour le déplacement inverse $p = -\alpha_k p_k$) : Nous voulons que :

$$\nabla m_{k+1}(-\alpha_k p_k) = \nabla f_{k+1} + B_{k+1}(-\alpha_k p_k) = \nabla f_k. \quad (5.5)$$

Réarrangeons cette dernière équation :

$$B_{k+1}(\alpha_k p_k) = \nabla f_{k+1} - \nabla f_k.$$

Introduisons les vecteurs s_k (déplacement) et y_k (changement de gradient) :

$$s_k = x_{k+1} - x_k = \alpha_k p_k, \quad (5.6)$$

$$y_k = \nabla f_{k+1} - \nabla f_k. \quad (5.7)$$

On obtient alors l'équation sécante :

$$B_{k+1} s_k = y_k. \quad (5.8)$$

La matrice B_{k+1} doit transformer s_k en y_k .

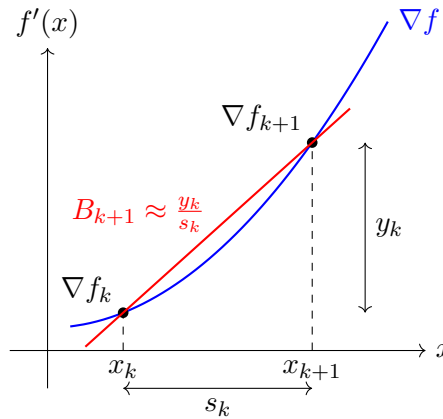


FIGURE 5.1 – Interprétation géométrique de l'équation sécante (cas 1D) : B_{k+1} est la pente de la sécante de la courbe ∇f .

La figure 5.1 illustre le principe de l'équation sécante en dimension 1. La matrice (ici un scalaire) B_{k+1} correspond à la pente de la droite sécante (en rouge) reliant les points $(\nabla f_k, x_k)$ et $(\nabla f_{k+1}, x_{k+1})$ sur la courbe du gradient (en bleu). Cette pente approxime la courbure moyenne de la fonction entre ces deux points.

5.3.2 Condition de courbure

Une condition nécessaire pour que B_{k+1} soit définie positive (comme nous le souhaitons pour avoir une direction de descente) est obtenue en multipliant l'équation sécante par $s_k^\top$ à gauche :

$$s_k^\top B_{k+1} s_k = s_k^\top y_k.$$

Si $B_{k+1} \succ 0$, alors on doit avoir :

$$s_k^\top y_k > 0. \quad (5.9)$$

C'est ce qu'on appelle la **condition de courbure**.

- Cette condition est toujours satisfaite si f est fortement convexe.
- Sinon, nous devons l'imposer en appliquant les **conditions de Wolfe** lors de la recherche linéaire (line search).

Preuve avec les conditions de Wolfe

La seconde condition de Wolfe est :

$$\nabla f_{k+1}^\top p_k \geq c_2 \nabla f_k^\top p_k. \quad (5.10)$$

Comme $s_k = \alpha_k p_k$ avec $\alpha_k > 0$, cela équivaut à :

$$\nabla f_{k+1}^\top s_k \geq c_2 \nabla f_k^\top s_k.$$

Ainsi :

$$\begin{aligned} y_k^\top s_k &= (\nabla f_{k+1} - \nabla f_k)^\top s_k \\ &\geq c_2 \nabla f_k^\top s_k - \nabla f_k^\top s_k \\ &= (c_2 - 1) \nabla f_k^\top s_k \\ &= (c_2 - 1) \alpha_k \nabla f_k^\top p_k. \end{aligned}$$

Comme $c_2 < 1$ (donc $c_2 - 1 < 0$) et que p_k est une direction de descente ($\nabla f_k^\top p_k < 0$), le produit est strictement positif :

$$y_k^\top s_k > 0.$$

5.3.3 Construction de la mise à jour DFP

L'équation sécante $B_{k+1} s_k = y_k$ admet une infinité de solutions (car B_{k+1} a $n(n+1)/2$ degrés de liberté pour seulement n contraintes).

Nous avons besoin d'un autre critère : choisir B_{k+1} le plus « proche » possible de B_k . On résout :

$$B_{k+1} = \arg \min_B \|B - B_k\| \quad \text{t.q. } B = B^\top \text{ et } B s_k = y_k. \quad (5.11)$$

DFP a choisi la norme de Frobenius pondérée :

$$\|A\|_W = \|W^{1/2} A W^{1/2}\|_F, \quad \text{où } \|C\|_F^2 = \sum_{i,j} C_{ij}^2.$$

La matrice de pondération W est choisie telle que $W y_k = s_k$.

Par exemple, on peut choisir $W = (\bar{G}_k)^{-1}$, où $\bar{G}_k$ est la Hessienne moyenne :

$$\bar{G}_k = \int_0^1 \nabla^2 f(x_k + \tau \alpha_k p_k) d\tau.$$

D'après le théorème de Taylor :

$$\nabla f(x_{k+1}) = \nabla f(x_k) + \bar{G}_k(x_{k+1} - x_k),$$

c'est-à-dire $y_k = \bar{G}_k s_k$, ce qui correspond bien au choix de W .

Avec ce choix de W , la solution est donnée par la formule de mise à jour DFP :

$$B_{k+1} = (I - \rho_k y_k s_k^\top) B_k (I - \rho_k s_k y_k^\top) + \rho_k y_k y_k^\top, \quad (5.12)$$

où

$$\rho_k = \frac{1}{y_k^\top s_k}. \quad (5.13)$$

5.3.4 Expression de l'inverse (Formule de Sherman-Morrison-Woodbury)

Si on exprime la mise à jour en termes de l'inverse de la Hessienne $H_k = B_k^{-1}$, comme dans la méthode de Newton où $p_k = -H_k \nabla f_k$, on peut utiliser la formule de Sherman-Morrison-Woodbury.

La formule de mise à jour DFP pour l'inverse est donnée par :

$$H_{k+1}^{DFP} = H_k - \frac{H_k y_k y_k^\top H_k}{y_k^\top H_k y_k} + \frac{s_k s_k^\top}{y_k^\top s_k}. \quad (5.14)$$

On remarque que cette mise à jour correspond à l'ajout de deux matrices de rang 1 (mise à jour de rang 2). De plus, $\text{rang}(H_{k+1} - H_k) \leq 2$.

5.4 L'approche BFGS

La méthode BFGS (Broyden-Fletcher-Goldfarb-Shanno) utilise le même raisonnement que DFP, mais en travaillant directement sur l'inverse de la Hessienne H_k au lieu de B_k .

L'équation sécante pour l'inverse devient :

$$H_{k+1} y_k = s_k. \quad (5.15)$$

On cherche alors H_{k+1} solution de :

$$H_{k+1} = \arg \min_H \|H - H_k\|_W \quad \text{t.q. } H = H^\top \text{ et } H y_k = s_k. \quad (5.16)$$

La solution donne la **formule de mise à jour BFGS** pour l'inverse :

$$H_{k+1}^{BFGS} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top, \quad (5.17)$$

avec $\rho_k = \frac{1}{y_k^\top s_k}$.

Algorithm 10: Méthode BFGS

- 1 Given starting point x_0 , convergence tolerance $\varepsilon > 0$, inverse Hessian approximation H_0 ;
 - 2 $k \leftarrow 0$;
 - 3 **while** $\|\nabla f_k\| > \varepsilon$ **do**
 - 4 Compute search direction $p_k = -H_k \nabla f_k$;
 - 5 Set $x_{k+1} = x_k + \alpha_k p_k$, where α_k is computed from a line search procedure to satisfy the Wolfe conditions;
 - 6 $s_k \leftarrow x_{k+1} - x_k$;
 - 7 $y_k \leftarrow \nabla f_{k+1} - \nabla f_k$;
 - 8 Compute $H_{k+1} = (I - \rho_k s_k y_k^\top) H_k (I - \rho_k y_k s_k^\top) + \rho_k s_k s_k^\top$;
 - 9 $k \leftarrow k + 1$;
-

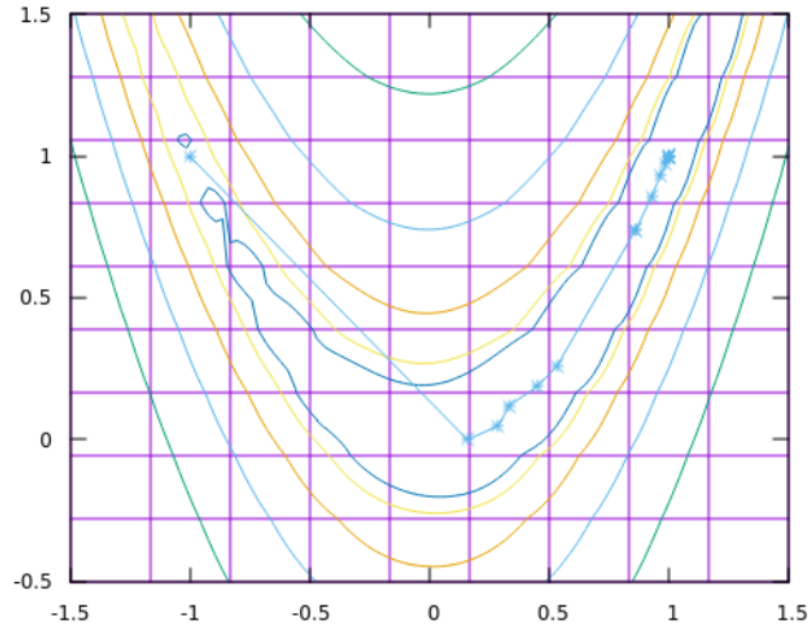


FIGURE 5.2 – Itérations de la méthode BFGS sur les courbes de niveau de la fonction de Rosenbrock.

Points clés à observer :

- **Trajectoire directe :** Contrairement à la descente de gradient qui zigzague, BFGS converge rapidement vers le minimum $(1, 1)$ en utilisant l'approximation de la courbure.
- **Adaptation locale :** La méthode apprend progressivement la forme de la Hessienne au fur et à mesure des itérations, ce qui lui permet de « redresser » ses directions de recherche.
- **Comparaison :** Le nombre d'itérations est bien inférieur à celui de la descente de gradient (Figure correspondante dans le chapitre 2), mais légèrement supérieur à Newton (qui utilise la vraie Hessienne).

5.5 Méthodes Quasi-Newton à mémoire limitée (L-BFGS)

Pour des problèmes de grande dimension, stocker la matrice H_k (de taille $n \times n$) devient prohibitif. La méthode L-BFGS (Limited-memory BFGS) résout ce problème en stockant uniquement les m dernières paires (s_k, y_k) .

Algorithm 11: L-BFGS récursion à deux boucles

```

1  $q \leftarrow \nabla f_k$ ;
2 for  $i = k - 1, k - 2, \dots, k - m$  do
3    $\alpha_i \leftarrow \rho_i s_i' q$ ;
4    $q \leftarrow q - \alpha_i y_i$ ;
5  $r \leftarrow H_k^0 q$ ;
6 for  $i = k - m, k - m + 1, \dots, k - 1$  do
7    $\beta \leftarrow \rho_i y_i' r$ ;
8    $r \leftarrow r + s_i(\alpha_i - \beta)$ ;
9 stop with result  $H_k \nabla f_k = r$ .
```

Algorithm 12: L-BFGS

```

1 Choose starting point  $x_0$ , integer  $m > 0$ ;
2  $k \leftarrow 0$ ;
3 repeat
4   Choose  $H_k^0$ , for instance  $H_k^0 = \frac{s'_{k-1}y_{k-1}}{y'_{k-1}y_{k-1}}I$ ;
5   Compute  $p_k \leftarrow -H_k \nabla f_k$  from Algorithm 11;
6   Compute  $x_{k+1} \leftarrow x_k + \alpha_k p_k$ , where  $\alpha_k$  is chosen to satisfy the Wolfe conditions;
7   if  $k > m$  then
8      $\lfloor$  Discard the vector pair  $\{s_{k-m}, y_{k-m}\}$  from storage;
9   Compute and save  $s_k \leftarrow x_{k+1} - x_k$ ,  $y_k = \nabla f_{k+1} - \nabla f_k$ ;
10   $k \leftarrow k + 1$ ;
11 until convergence;

```

5.6 Remarques et Propriétés

Initialisation. Il n'y a pas d'initialisation optimale universelle pour H_0 . Les possibilités sont :

- Approximer la Hessienne inverse initiale.
- Prendre un multiple de l'identité $H_0 = \gamma I$.

Coût. Le coût de calcul de chaque itération est en $O(n^2)$ (multiplications matrice-vecteur et mises à jour de rang 1/2), contrairement à $O(n^3)$ pour Newton (qui nécessite une inversion de système).

Définie positivité. Si H_k est définie positive, alors H_{k+1} l'est aussi, pourvu que $y_k^\top s_k > 0$. En effet, pour tout $z \neq 0$:

$$z^\top H_{k+1} z = z^\top (I - \rho s y^\top) H (I - \rho y s^\top) z + \rho (z^\top s)^2 \geq 0.$$

En toute rigueur, à ce stade on a seulement ≥ 0 car $(I - \rho y s^\top)z$ et $z^\top s$ pourraient être nuls. Cependant, si $z^\top s = 0$, alors il ne reste que le terme $z^\top (I - \rho s y^\top) H (I - \rho y s^\top) z$, qui est > 0 car H est définie positive et $(I - \rho y s^\top)z \neq 0$ (sinon $z = \rho(y^\top z)s$, ce qui impliquerait $z^\top s = \rho(y^\top z)s^\top s \neq 0$, contradiction). Donc $z^\top H_{k+1} z > 0$.

5.7 La classe de Broyden

Il existe une famille de mises à jour, appelée classe de Broyden, définie par :

$$B_{k+1} = (1 - \phi_k) B_{k+1}^{BFGS} + \phi_k B_{k+1}^{DFP}. \quad (5.18)$$

Ou sous une forme explicite :

$$B_{k+1} = B_k - \frac{B_k s_k s_k^\top B_k}{s_k^\top B_k s_k} + \frac{y_k y_k^\top}{y_k^\top s_k} + \phi_k (s_k^\top B_k s_k) v_k v_k^\top, \quad (5.19)$$

où ϕ_k est un paramètre scalaire et

$$v_k = \frac{y_k}{y_k^\top s_k} - \frac{B_k s_k}{s_k^\top B_k s_k}.$$

Cas particuliers :

- Si $\phi_k = 0$, on retrouve **BFGS**.
- Si $\phi_k = 1$, on retrouve **DFP**.

Classe de Broyden restreinte. La classe de Broyden restreinte correspond au choix $\phi_k \in [0, 1)$. Les méthodes de cette classe sont des combinaisons convexes de BFGS et DFP.

5.8 Propriétés sur les fonctions quadratiques

Considérons une fonction fortement quadratique $f(x) = \frac{1}{2}x^\top Qx - b^\top x$. Supposons que B_0 est symétrique définie positive et que α_k est le pas exact (exact line search).

On a alors les propriétés suivantes (Théorème admis) :

1. Les itérés x_k générés sont indépendants du choix de ϕ_k dans la classe de Broyden.
2. L'équation sécante est satisfaite pour toutes les directions de recherche précédentes :

$$B_k s_j = y_j, \quad \forall j = 0, \dots, k-1. \quad (5.20)$$

3. Si on choisit $B_0 = I$, alors les itérés sont identiques à ceux de la méthode du gradient conjugué, et les directions de recherche sont conjuguées :

$$s_j^\top Q s_k = 0, \quad \forall j \neq k. \quad (5.21)$$

4. Si on effectue n itérations, alors la matrice Hessienne est identifiée exactement :

$$B_n = Q. \quad (5.22)$$

5.9 Analyse de convergence

5.9.1 Convergence globale

Remarque. BFGS et les méthodes de la classe de Broyden restreinte ($\phi_k \in [0, 1)$) sont très robustes en pratique. Cependant, il n'existe pas de résultat de convergence globale pour des fonctions non-linéaires générales (non convexes).

Nous allons énoncer un résultat pour les fonctions convexes.

Hypothèses B.

- La fonction objective f est deux fois continûment différentiable (C^2).
- L'ensemble de niveau $\mathcal{L} = \{x \in \mathbb{R}^n \mid f(x) \leq f(x_0)\}$ est convexe.
- Il existe des constantes $0 < m \leq M$ telles que pour tout $z \in \mathbb{R}^n$ et pour tout $x \in \mathcal{L}$:

$$m\|z\|^2 \leq z^\top \nabla^2 f(x) z \leq M\|z\|^2. \quad (5.23)$$

Cela implique que f est fortement convexe sur $\mathcal{L}$.

Théorème 5.1 (Convergence globale de BFGS). *Soit B_0 une matrice symétrique définie positive quelconque. Soit x_0 un point tel que les Hypothèses B sont vérifiées. Alors la suite des itérés (x_k) générée par l'algorithme BFGS (avec $\epsilon = 0$) converge vers le minimiseur x^* de f (qui est unique d'après les Hypothèses B).*

Preuve. Voir [10].

Remarque. Ce théorème peut être généralisé à toute la classe de Broyden restreinte.

5.9.2 Convergence super-linéaire de BFGS

Il est également possible de prouver que la convergence est suffisamment rapide pour que :

$$\sum_{k=0}^{\infty} \|x_k - x^*\| < \infty.$$

Hypothèse C. La matrice Hessienne $G = \nabla^2 f$ est Lipschitz continue en x^* : il existe $L > 0$ tel que pour x suffisamment proche de x^* :

$$\|\nabla^2 f(x) - \nabla^2 f(x^*)\| \leq L\|x - x^*\|.$$

Théorème 5.2 (Admis). *Supposons que f est deux fois continûment différentiable (C^2). Considérons les itérés $x_{k+1} = x_k + \alpha_k p_k$ où α_k satisfait les conditions de Wolfe avec $c_1 \leq \frac{1}{2}$. Si la suite (x_k) converge vers un point x^* où $\nabla f(x^*) = 0$ et $\nabla^2 f(x^*)$ est définie positive. Et si les directions de recherche satisfont (Condition de Dennis-Moré) :*

$$\lim_{k \rightarrow \infty} \frac{\|\nabla f_k + \nabla^2 f_k p_k\|}{\|p_k\|} = 0, \quad (5.24)$$

alors la valeur $\alpha_k = 1$ est admissible pour tout $k \geq k_0$ (où k_0 est un certain entier), et si on choisit $\alpha_k = 1$ pour $k \geq k_0$, la convergence de (x_k) vers x^ est **super-linéaire**.*

Remarque. Si $p_k = -B_k^{-1} \nabla f_k$ est une direction quasi-Newton, alors la condition ci-dessus se réécrit :

$$\lim_{k \rightarrow \infty} \frac{\|(B_k - \nabla^2 f(x_k))p_k\|}{\|p_k\|} = 0.$$

Interprétation : Cela signifie que B_k approche suffisamment bien la Hessienne le long de la direction p_k . Il suffit que l'approximation soit suffisamment bonne dans cette direction, pas la peine qu'elle le soit dans tout l'espace.

Chapitre 6

Problèmes d'optimisation sans contrainte à grande échelle

6.1 Introduction

Les applications pratiques peuvent comporter des centaines ou des milliers de variables. Avec les techniques des chapitres précédents, nous pouvons être confrontés à des coûts élevés :

- Coûts de calcul (temps CPU).
- Coûts de stockage (mémoire).

Dans ce chapitre, nous présenterons des techniques pour surmonter ces difficultés.

6.2 Méthodes de Newton Inexactes

Dans les méthodes de Newton, on trouve la direction p_k^N en résolvant le système linéaire :

$$\nabla^2 f_k p_k^N = -\nabla f_k.$$

(Formellement, $p_k^N = -(\nabla^2 f_k)^{-1} \nabla f_k$).

Notre objectif est de trouver une approximation p_k de p_k^N qui n'est pas trop coûteuse à calculer : ce sont les méthodes de Newton inexactes. Par exemple, nous pouvons utiliser des méthodes itératives comme le gradient conjugué pour résoudre le système de manière approchée.

La qualité de l'approximation déterminera les propriétés de convergence.

6.2.1 Convergence locale

Il est raisonnable d'arrêter la résolution du système $\nabla^2 f_k p_k^N = -\nabla f_k$ lorsque le résidu

$$r_k = \nabla^2 f_k p_k + \nabla f_k$$

est petit.

Par exemple, en utilisant une **suite forçante** $(\eta_k) \in (0, 1)^{\mathbb{N}}$ et en arrêtant lorsque :

$$\|r_k\| \leq \eta_k \|\nabla f_k\|. \quad (6.1)$$

Les deux théorèmes de cette sous-section s'appliquent à toutes les méthodes de Newton inexactes satisfaisant la condition ci-dessus. (Preuves : *Dembo et al.*)

Théorème 6.1 (Convergence locale). *Supposons que $\nabla^2 f(x)$ existe et est continue dans un voisinage d'un minimiseur x^* , avec $\nabla^2 f(x^*)$ définie positive. Soit $x_{k+1} = x_k + p_k$ (où l'on suppose un pas unitaire $\alpha_k = 1$ près de la solution) où p_k satisfait :*

$$\|r_k\| \leq \eta_k \|\nabla f_k\|.$$

Supposons que $\eta_k \leq \eta$ pour tout k , où $\eta \in [0, 1)$.

Alors, si x_0 est assez proche de x^ , la suite (x_k) converge vers x^* . De plus, il existe un certain $\hat{\eta} \in (\eta, 1)$ tel que :*

$$\|\nabla^2 f(x^*)(x_{k+1} - x^*)\| \leq \hat{\eta} \|\nabla^2 f(x^*)(x_k - x^*)\|.$$

6.2.2 Taux de convergence

Théorème 6.2 (Taux de convergence). *Supposons les conditions du théorème précédent vérifiées, et que (x_k) converge vers x^* .*

- *Le taux de convergence est **super-linéaire** si $\lim_{k \rightarrow \infty} \eta_k = 0$.*
- *Si $\nabla^2 f$ est Lipschitz-continue près de x^* et si $\eta_k = O(\|\nabla f_k\|)$, alors la convergence est **quadratique**.*

Éléments de preuve. On peut montrer que :

$$\frac{\|\nabla f_{k+1}\|}{\|\nabla f_k\|} \leq \eta_k + o(1).$$

- Si $\eta_k \rightarrow 0$, alors le gradient (et donc les itérés) converge super-linéairement.
- Si $\nabla^2 f$ est Lipschitzienne, on a $\nabla f_{k+1} = O(\|\nabla f_k\|^2)$, ce qui implique une convergence quadratique.

Remarques.

- Le choix $\eta_k = \min(0.5, \sqrt{\|\nabla f_k\|})$ donne une convergence **super-linéaire**.
- Le choix $\eta_k = \min(0.5, \|\nabla f_k\|)$ donne une convergence **quadratique**.
- Ces résultats sont locaux : nous avons supposé que x_0 était proche de x^* et que $\alpha_k = 1$.

6.2.3 Méthodes Newton-CG avec recherche linéaire

On résout le système $\nabla^2 f_k p_k = -\nabla f_k$ en utilisant la méthode du gradient conjugué (CG). Nous devons arrêter si nous rencontrons une direction de courbure négative (c'est-à-dire si $\nabla^2 f_k$ n'est pas définie positive).

6.3 Méthodes Quasi-Newton à mémoire limitée (L-BFGS)

Lorsque la Hessienne n'est pas disponible à un coût raisonnable (en temps ou en stockage), les méthodes Quasi-Newton à mémoire limitée sont utiles. Elles remplacent le stockage de matrices $n \times n$ par le stockage d'un nombre limité de vecteurs de taille n . Nous nous concentrons ici sur la méthode **L-BFGS** (Low-storage BFGS), qui stocke l'information sur la courbure des m itérations précédentes. (Dérivation : [9]).

6.3.1 Rappels sur BFGS

Les étapes de BFGS sont $x_{k+1} = x_k - \alpha_k H_k \nabla f_k$, où H_k est mis à jour selon :

$$H_{k+1} = V_k^\top H_k V_k + \rho_k s_k s_k^\top, \quad (6.2)$$

avec $V_k = I - \rho_k y_k s_k^\top$ et $\rho_k = \frac{1}{y_k^\top s_k}$. On note $s_k = x_{k+1} - x_k$ et $y_k = \nabla f_{k+1} - \nabla f_k$.

6.3.2 Principes de L-BFGS

- Au lieu de stocker la matrice dense H_k , nous stockons m paires de vecteurs (s_i, y_i) .
- À chaque itération, nous substituons la paire la plus ancienne (s_{k-m}, y_{k-m}) par la nouvelle (s_k, y_k) .
- En pratique, on choisit m entre 3 et 20.
- À l'itération k , nous disposons de l'itéré courant x_k et des paires (s_i, y_i) pour $i = k - m, \dots, k - 1$.
- On choisit une approximation initiale H_k^0 .

Stockage L-BFGS : fenêtre glissante de m paires

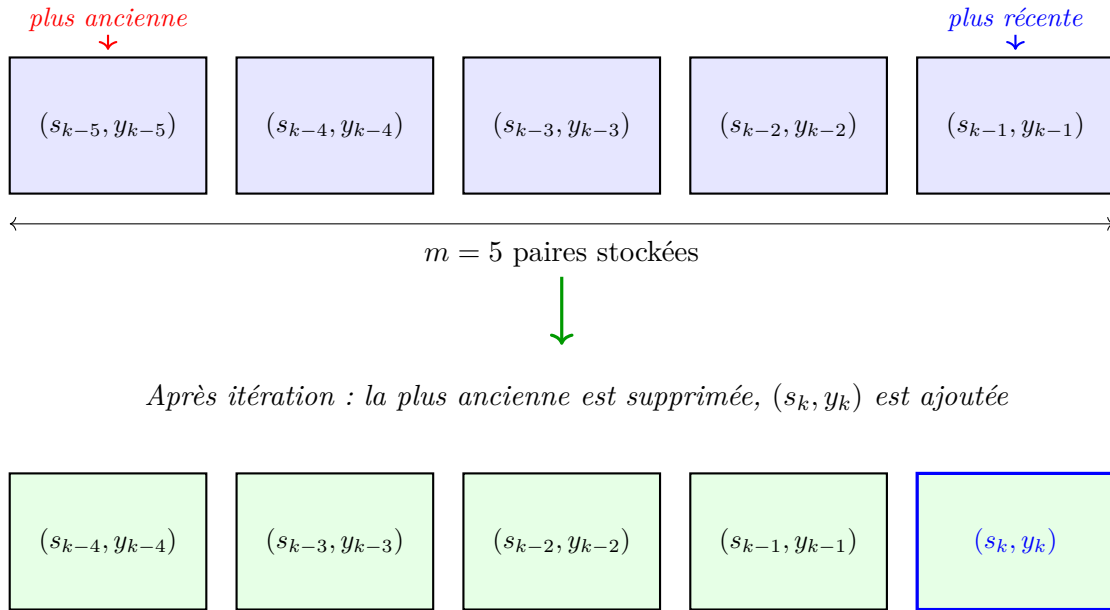


FIGURE 6.1 – Principe du stockage L-BFGS : fenêtre glissante de m paires (s_i, y_i) . À chaque itération, la paire la plus ancienne est éliminée et la nouvelle paire est ajoutée.

La figure 6.1 illustre le principe de stockage de L-BFGS. Au lieu de maintenir la matrice H_k (de taille $n \times n$), on conserve uniquement les m dernières paires de vecteurs (s_i, y_i) . Cette approche réduit le coût de stockage de $O(n^2)$ à $O(mn)$, ce qui est essentiel pour les problèmes de grande dimension.

On peut développer la formule de mise à jour de manière récursive :

$$\begin{aligned}
H_k &= V_{k-1}^\top H_{k-1} V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
&= V_{k-1}^\top (V_{k-2}^\top H_{k-2} V_{k-2} + \rho_{k-2} s_{k-2} s_{k-2}^\top) V_{k-1} + \rho_{k-1} s_{k-1} s_{k-1}^\top \\
&= \dots \\
&= (V_{k-1}^\top \dots V_{k-m}^\top) H_k^0 (V_{k-m} \dots V_{k-1}) \\
&\quad + \rho_{k-m} (V_{k-1}^\top \dots V_{k-m+1}^\top) s_{k-m} s_{k-m}^\top (V_{k-m+1} \dots V_{k-1}) \\
&\quad + \dots + \rho_{k-1} s_{k-1} s_{k-1}^\top.
\end{aligned}$$

Cette formule permet de calculer le produit $H_k \nabla f_k$ sans jamais former la matrice H_k (voir Algorithme 11, *two-loop recursion*).

Complexité. L'Algorithme 11 requiert $4mn$ opérations, plus le coût du calcul de $H_k^0 q$ (n multiplications si H_k^0 est diagonale).

Choix de H_k^0 . En pratique, choisir $H_k^0 = \gamma_k I$ avec :

$$\gamma_k = \frac{s_{k-1}^\top y_{k-1}}{y_{k-1}^\top y_{k-1}}, \quad (6.3)$$

donne des résultats satisfaisants. On peut montrer que γ_k approche une des valeurs propres de la Hessienne.

Recherche linéaire. Comme pour BFGS, on utilise les conditions de Wolfe (fortes) et on initialise la recherche linéaire avec $\alpha_k = 1$.

Chapitre 7

Optimisation sans dérivées

(Brent, 1973)

7.1 Introduction

Souvent, les dérivées sont indisponibles ou difficiles à calculer. Exemples :

- Le résultat d'une mesure expérimentale.
- Le résultat d'une simulation stochastique.
- f est connue mais le calcul des dérivées est trop coûteux.

7.2 Quelques solutions possibles

- **Différentiation automatique** : si un code binaire calcule f , on peut appliquer la différentiation automatique.

Principe. Si $f(x) = g(h(m(x)))$, on note $w = m(x)$, $y = h(w)$, $z = g(y)$. On utilise la règle de la chaîne (*chain rule*) :

$$f'(x) = \frac{\partial g}{\partial y} \Big|_y \times \frac{\partial h}{\partial w} \Big|_w \times \frac{\partial m}{\partial x} \Big|_x.$$

- **Différences finies** : utiliser des différences finies pour approximer ∇f (et peut-être $\nabla^2 f$), puis appliquer les algorithmes des chapitres précédents. **Mais** cela peut être impossible à cause des approximations ou du bruit (*noise*).

7.3 Le problème du bruit

Dans de nombreuses applications, la fonction objectif ressemble à :

$$f(x) = h(x) + \phi(x), \tag{7.1}$$

où h est une fonction régulière (différentiable), et ϕ représente le bruit (possiblement indépendant de x).

L'approximation par différences finies du gradient $\nabla f(x)$ pour $\epsilon > 0$ est donnée par :

$$\nabla_\epsilon f(x) = \left(\frac{f(x + \epsilon e_i) - f(x - \epsilon e_i)}{2\epsilon} \right)_{i=1, \dots, n},$$

où $e_i = (0, \dots, 1, \dots, 0)^\top$ est le i -ème vecteur de la base canonique.

7.3.1 Niveau de bruit et erreur d'approximation

Définissons le niveau de bruit local $\eta(x, \epsilon)$ par :

$$\eta(x, \epsilon) = \sup_{\|z-x\|_\infty < \epsilon} |\phi(z)|. \quad (7.2)$$

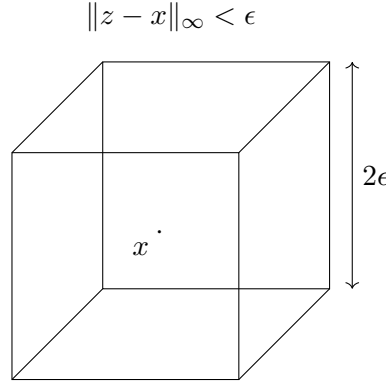


FIGURE 7.1 – Voisinage pris en compte pour le niveau de bruit.

La figure 7.1 représente le voisinage cubique $\|z - x\|_\infty < \epsilon$ autour du point x . Le niveau de bruit local $\eta(x, \epsilon)$ correspond à la plus grande amplitude du bruit $\phi(z)$ rencontrée à l'intérieur de ce cube de côté 2ϵ .

On peut montrer que si $\nabla^2 h$ est L -Lipschitzienne dans le voisinage $\{z \mid \|z - x\|_\infty < \epsilon\}$, alors :

$$\|\nabla_\epsilon f(x) - \nabla h(x)\|_\infty \leq \underbrace{L\epsilon^2}_{\text{différences finies}} + \underbrace{\frac{\eta(x, \epsilon)}{\epsilon}}_{\text{bruit}}. \quad (7.3)$$

On ne peut donc pas espérer une bonne approximation de $\nabla h(x)$ si le bruit est important, car réduire ϵ augmente l'amplification du bruit (η/ϵ).

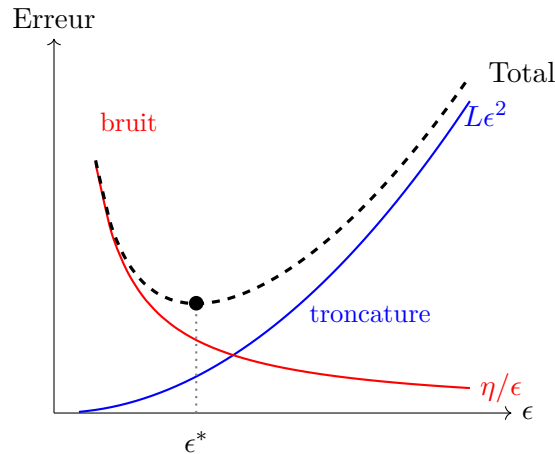


FIGURE 7.2 – Compromis erreur de troncature vs amplification du bruit. L'erreur totale (pointillés) a un minimum en ϵ^* .

7.3.2 Exemple : Bruit stochastique

Soient $X_1, \dots, X_n$ des variables aléatoires i.i.d. (indépendantes et identiquement distribuées) avec une moyenne $h(x)$ et une variance finie σ^2 .

Alors le théorème central limite donne :

$$\sqrt{n} \left(\frac{1}{n} \sum_{i=1}^n X_i - h(x) \right) \xrightarrow{d} \mathcal{N}(0, \sigma^2).$$

Approximativement (« Roughly ») :

$$\frac{1}{n} \sum_{i=1}^n X_i - h(x) \approx \frac{\sigma}{\sqrt{n}} G,$$

où $G \sim \mathcal{N}(0, 1)$. Cela montre que l'erreur décroît seulement en $1/\sqrt{n}$, ce qui peut nécessiter un très grand nombre d'échantillons pour réduire le bruit à un niveau acceptable pour les différences finies.

De sorte que :

$$\underbrace{\frac{1}{n} \sum_{i=1}^n X_i}_{\text{observé}} \simeq \underbrace{h(x)}_{\text{partie utile}} + \underbrace{\frac{\sigma}{\sqrt{n}} G}_{\text{bruit } \phi}.$$

Dans cette situation, ϕ est indépendant de x , et ϕ est même non borné (car la loi normale n'est pas bornée).

Attention : À partir de maintenant, nous supposons que f a des dérivées continues, même si ces dernières ne seront pas utilisées dans les algorithmes.

Nous proposons une sélection non exhaustive de méthodes.

7.4 Méthodes basées sur un modèle (Model-based methods)

7.4.1 Idée générale

L'idée est de construire un modèle m_k (souvent quadratique) qui interpole la fonction f sur un ensemble de points bien choisis.

Supposons qu'à l'étape k , nous disposons de l'ensemble de points interpolation $\mathcal{Y} = \{y_1, \dots, y_q\}$ avec $y_i \in \mathbb{R}^n$ pour $i = 1, \dots, q$, et que l'itéré courant x_k est l'un d'entre eux. De plus, aucun point dans $\mathcal{Y}$ n'a une valeur de fonction inférieure à celle de $f(x_k)$.

7.4.2 Construction du modèle

Notre modèle sera de la forme :

$$m_k(x_k + p) = c + g^\top p + \frac{1}{2} p^\top G p, \quad (7.4)$$

où $c \in \mathbb{R}$, $g \in \mathbb{R}^n$, et $G \in \mathbb{R}^{n \times n}$ est symétrique.

Nous déterminons c , g et G en imposant les conditions d'interpolation :

$$m_k(y_i) = f(y_i), \quad i = 1, \dots, q. \quad (7.5)$$

Il y a $1 + n + \frac{n(n+1)}{2} = \frac{(n+1)(n+2)}{2}$ coefficients à ajuster. Si nous choisissons $q = \frac{(n+1)(n+2)}{2}$, le système admet une solution unique (sous condition que les points soient bien positionnés).

Ensuite, nous calculons le pas p en résolvant le sous-problème de confiance :

$$\min_p m_k(x_k + p) \quad \text{sujet à } \|p\|_2 \leq \Delta, \quad (7.6)$$

pour un certain rayon de région de confiance Δ .

7.4.3 Mise à jour

- Si la décroissance est « importante », nous augmenterons Δ pour l'itération suivante.
- Sinon, nous mettons à jour l'ensemble $\mathcal{Y}$ ou nous réduisons le rayon de la région de confiance Δ , et nous rejetons le pas.

7.4.4 Initialisation et géométrie

Pour l'algorithme complet (Algorithme 13), l'initialisation se fait souvent en prenant les $n+1$ sommets et les $\frac{n(n+1)}{2}$ milieux des arêtes d'un simplexe dans $\mathbb{R}^n$.

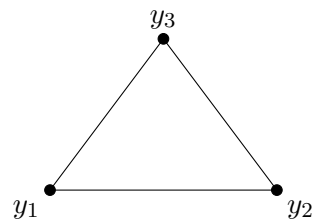
Définition 7.1 (Simplexe). Un simplexe dans $\mathbb{R}^n$ est un ensemble de $n+1$ vecteurs $y_1, \dots, y_{n+1}$ dans $\mathbb{R}^n$.

Le simplexe $\{y_1, \dots, y_{n+1}\}$ est dit **non dégénéré** si la matrice

$$\begin{bmatrix} y_2 - y_1 & y_3 - y_1 & \cdots & y_{n+1} - y_1 \end{bmatrix}$$

est non singulière (c'est-à-dire que les directions sont indépendantes).

non dégénéré (non-degenerate)



Exemple dans $\mathbb{R}^2$

dégénéré (degenerate)

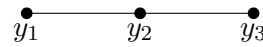


FIGURE 7.3 – Illustration d'un simplexe non dégénéré vs dégénéré dans $\mathbb{R}^2$.

La figure 7.3 illustre la notion de dégénérescence. À gauche, les vecteurs $y_2 - y_1$ et $y_3 - y_1$ sont linéairement indépendants, formant un vrai triangle (volume non nul). À droite, les points sont alignés : les vecteurs sont colinéaires, la matrice associée est singulière, et le simplexe est dégénéré (volume nul).

Algorithm 13: Méthode dérivée-free basée sur un modèle

```

1 Choose an interpolation set  $Y = \{y^2, \dots, y^q\}$  such that the linear system defined by
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$  is nonsingular, and select  $x_0$  as a point in this set such
that  $f(x_0) \leq f(y^i)$  for all  $y^i \in Y$ . Choose an initial trust region radius  $\Delta_0$ , a constant
 $\eta \in (0, 1)$ , and set  $k \leftarrow 0$ ;
2 repeat
3   Form the quadratic model  $m_k(x_k + \cdot)$  that satisfies the interpolation conditions
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$ ;
4   Compute a step  $p$  by approximately solving  $\min_p m_k(x_k + p)$ , subject to  $\|p\|_2 \leq \Delta$ ;
5   Define the trial point  $x_k^+ = x_k + p$ ;
6   Compute the ratio  $\rho = \frac{f(x_k) - f(x_k^+)}{m_k(x_k) - m_k(x_k^+)}$ ;
7   if  $\rho \geq \eta$  then
8     Replace an element of  $Y$  by  $x_k^+$ ;
9     Choose  $\Delta_{k+1} \geq \Delta_k$ ;
10    Set  $x_{k+1} \leftarrow x_k^+$ ;
11    Set  $k \leftarrow k + 1$  and go to the next iteration;
12  else if the set  $Y$  need not be improved then
13    Choose  $\Delta_{k+1} < \Delta_k$ ;
14    Set  $x_{k+1} \leftarrow x_k$ ;
15    Set  $k \leftarrow k + 1$  and go to the next iteration;
16  Invoke a geometry-improving procedure to update  $Y$  : at least one of the points in
 $Y$  is replaced by some other point, with the goal of improving the conditioning of
 $m_k(y^\ell) = f(y^\ell), \ell = 1, \dots, q$ ;
17  Set  $\Delta_{k+1} \leftarrow \Delta_k$ ;
18  Set  $x_k^+ \leftarrow \hat{x}$  and recompute  $\rho$  by  $\rho = \frac{f(x_k) - f(x_k^+)}{m_k(x_k) - m_k(x_k^+)}$ ;
19  if  $\rho \geq \eta$  then
20    Set  $x_{k+1} \leftarrow x_k^+$ ;
21  else
22    Set  $x_{k+1} \leftarrow x_k$ ;
23   $k \leftarrow k + 1$ ;
24 until a convergence test is satisfied;

```

7.5 Méthodes de recherche par coordonnées ou par motifs (Coordinate or pattern-search methods)

Partant de l'itéré courant, on regarde certains points le long de directions spécifiées, et on décide du pas.

7.5.1 Recherche par coordonnées (Coordinate search)

Aussi appelée *coordinate descent*.

Idée. On cycle à travers les n directions de coordonnées $e_1, \dots, e_n$. On minimise f le long de e_i , puis on recommence...

Attention. Cette méthode peut ne jamais approcher un point où le gradient s'annule (*gradient-vanishing point*) ! Elle peut stagner aux coins de lignes de niveau anguleuses si les directions ne sont pas adaptées à la géométrie locale de la fonction.

Lorsque la convergence a lieu, elle peut être plus lente que la plus forte pente (*steepest descent*).

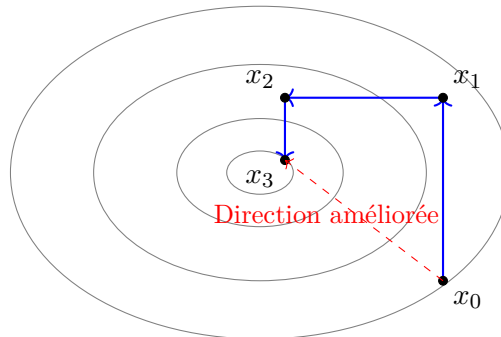


FIGURE 7.4 – Illustration de la recherche par coordonnées (zigzag) et de la lenteur potentielle.

La figure 7.4 montre que la méthode progresse en "escalier" (zig-zag). Les directions de coordonnées ne pointent pas nécessairement vers le minimum, forçant de nombreux petits pas.

Amélioration possible. Effectuer n itérations puis faire une recherche linéaire le long de la direction $(x_k - x_{k-n})$.

7.5.2 Méthodes de recherche par motifs (Pattern-search methods)

Cette classe de méthode généralise l'approche précédente.

Idée. Choisir un ensemble de directions de recherche D_k , et à chaque itération, évaluer f à une longueur de pas donnée γ_k le long de ces directions. Si un point « plus intéressant » est trouvé, il devient le nouvel itéré.

Algorithme 14 (Schéma général)

On peut montrer qu'il existe des points d'accumulation stationnaires. À l'étape k , soit x_k l'itéré courant, D_k l'ensemble courant de directions, et γ_k la longueur de pas (*step length*).

- **Succès :** Si l'une des directions dans D_k produit une décroissance significative, nous nous déplaçons vers ce nouveau point et nous augmentons le pas γ_k .
- **Échec :** Sinon, nous gardons x_k et nous diminuons γ_k .
- Sous certaines conditions, l'ensemble D_k peut être modifié.

Conditions sur D_k

Pour assurer la convergence, l'ensemble D_k doit couvrir l'espace de manière suffisante. Il doit y avoir au moins une direction de descente dans D_k , c'est-à-dire une direction $d \in D_k$ telle que :

$$\cos(\theta) = \frac{-\nabla f_k^\top d}{\|\nabla f_k\| \|d\|} > 0$$

Condition de Zoutendijk (adaptée). Assurons-nous qu'il existe $\delta > 0$ tel que $\cos(\theta) > \delta$ pour au moins une direction. On définit la mesure de qualité de l'ensemble de directions $\kappa(D_k)$ par :

$$\kappa(D_k) = \min_{v \in \mathbb{R}^n} \max_{p \in D_k} \frac{v^\top p}{\|v\| \|p\|} \quad (7.7)$$

La condition est alors : $\kappa(D_k) \geq \delta > 0$.

Interprétation : Cette condition signifie que pour tout vecteur v , il existe une direction $p \in D_k$ qui forme un angle « suffisamment éloigné » de $\pi/2$ avec v . Plus précisément, le cosinus de l'angle entre v et p est borné inférieurement par une constante strictement positive δ .

Bornes sur le pas

Il doit exister des constantes positives $\underline{\beta}, \bar{\beta}$ telles que pour tout $p \in D_k$:

$$\underline{\beta} \leq \|p\|_2 \leq \bar{\beta}$$

Le pas γ_k doit capturer le diamètre effectif de la recherche.

Sous ces deux conditions, pour tout k , il existe un vecteur $p \in D_k$ tel que :

$$-\nabla f_k^\top p \geq \kappa(D_k) \|\nabla f_k\| \|p\| \geq \delta \|\nabla f_k\| \underline{\beta} \quad (7.8)$$

Cela garantit une décroissance suffisante tant que le gradient n'est pas nul.

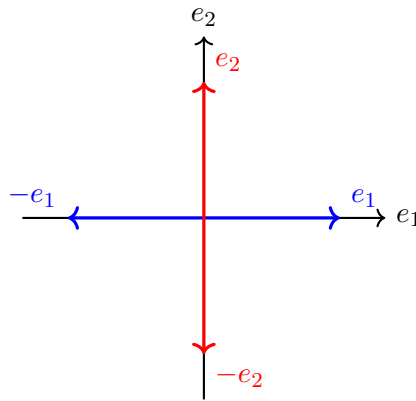
Ensembles D_k possibles satisfaisant ces conditions

Voici quelques exemples d'ensembles D_k qui satisfont les conditions énoncées :

Exemple 1 : Directions canoniques et leurs opposées.

$$D_k = \{e_1, -e_1, e_2, -e_2, \dots, e_n, -e_n\} \quad (7.9)$$

où e_i est le i -ème vecteur de la base canonique.



Cas $n = 2$

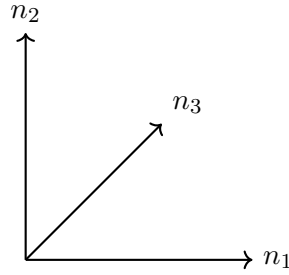
FIGURE 7.5 – Ensemble de directions $D_k = \{e_1, -e_1, e_2, -e_2\}$ dans $\mathbb{R}^2$.

Exemple 2 : Directions du simplexe. Pour $n \geq 1$, on définit :

$$n_i = \frac{1}{2n} e - e_i, \quad i = 1, \dots, n, \quad (7.10)$$

$$n_{n+1} = \frac{1}{2n} e, \quad (7.11)$$

où $e = (1, \dots, 1)^\top \in \mathbb{R}^n$.



Cas $n = 2$

FIGURE 7.6 – Directions du simplexe dans $\mathbb{R}^2$.

Remarque 7.2. • On peut toujours enrichir l'ensemble D_k avec des heuristiques intéressantes.
• Il n'est pas nécessaire d'examiner toutes les directions à chaque étape.

Algorithm 14: Méthode de recherche par motifs

```

1 Given convergence tolerance  $\gamma_{tol}$ , contraction parameter  $\theta_{max}$ , sufficient decrease
  function  $\rho : [0, \infty) \rightarrow \mathbf{R}$  with  $\rho$  an increasing function and  $\frac{\rho(t)}{t} \rightarrow 0$  as  $t \downarrow 0$ ;
2 Choose initial point  $x_0$ , initial step length  $\gamma_0 > \gamma_{tol}$ , initial direction set  $\mathcal{D}_0$ ;
3 for  $k = 1, 2, \dots$  do
4   if  $\gamma_k \leq \gamma_{tol}$  then
5     stop ;
6   if  $f(x_k + \gamma_k p_k) < f(x_k) - \rho(\gamma_k)$  for some  $p_k \in \mathcal{D}_k$  then
7     Set  $x_{k+1} \leftarrow x_k + \gamma_k p_k$  for some such  $p_k$ ;
8     Set  $\gamma_{k+1} \leftarrow \phi_k \gamma_k$  for some  $\phi_k \geq 1$  ; ▷ increase step length
9   else
10    Set  $x_{k+1} \leftarrow x_k$ ;
11    Set  $\gamma_{k+1} \leftarrow \theta_k \gamma_k$ , where  $0 < \theta_k \leq \theta_{max} < 1$ ;

```

7.6 Méthode du simplexe de Nelder-Mead

7.6.1 Principe général

À chaque étape, nous gardons la trace de $n + 1$ points dans $\mathbb{R}^n$, qui forment un simplexe.

À chaque itération :

- On retire un point du simplexe.
- On le remplace par un point situé sur la droite qui joint l'ancien point au centroïde des n meilleurs points.
- Le simplexe sera **étendu** (*expanded*), **réfléchi** (*reflected*), ou **contracté** (*contracted*).

S'il n'existe pas de tel point qui améliore la fonction, on **rétrécit** (*shrinks*) le simplexe vers le meilleur point.

7.6.2 Notation et ordonnancement

À une étape donnée, supposons que les sommets du simplexe sont ordonnés par valeur de fonction croissante :

$$f(x_1) \leq f(x_2) \leq \dots \leq f(x_{n+1}) \quad (7.12)$$

Ainsi, x_1 est le meilleur point (plus petite valeur de f), et x_{n+1} est le pire point.

7.6.3 Centroïde

Le centroïde des n meilleurs points (c'est-à-dire tous sauf x_{n+1}) est :

$$\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i \quad (7.13)$$

7.6.4 Points candidats

Les points sur la droite joignant x_{n+1} et $\bar{x}$ sont donnés par :

$$\bar{x}(t) = \bar{x} + t(x_{n+1} - \bar{x}), \quad t \in \mathbb{R} \quad (7.14)$$

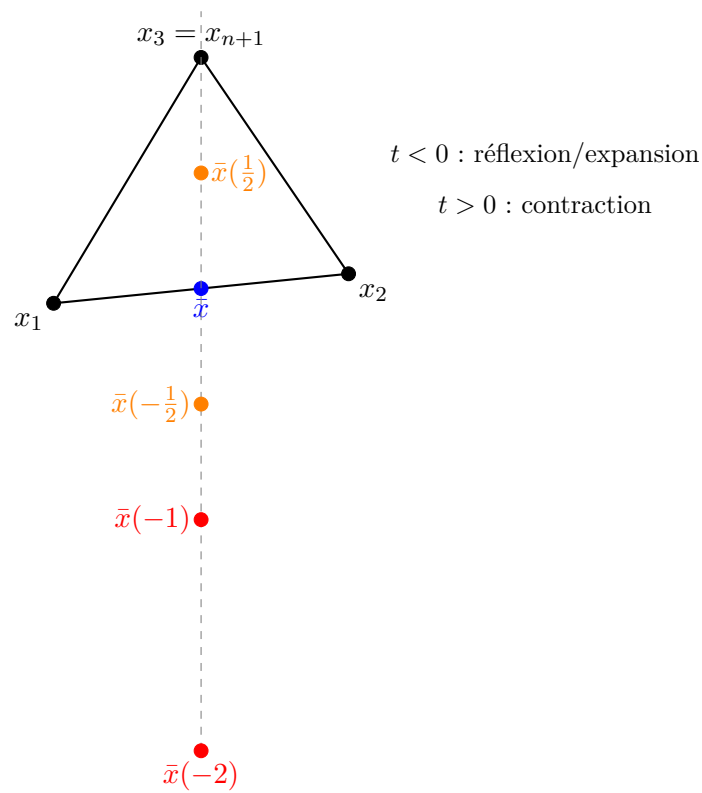
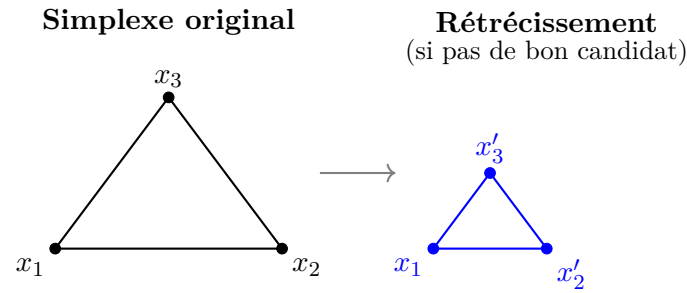


FIGURE 7.7 – Points candidats sur la droite joignant x_{n+1} et le centroïde $\bar{x}$.

En pratique, on évalue :

- $\bar{x}(-2)$: expansion
- $\bar{x}(-1)$: réflexion
- $\bar{x}(-\frac{1}{2})$: contraction extérieure
- $\bar{x}(\frac{1}{2})$: contraction intérieure

FIGURE 7.8 – Opération de rétrécissement vers le meilleur point x_1 .

La figure 7.8 illustre l'opération de rétrécissement (*shrink*). Lorsqu'aucun des points candidats $\bar{x}(-2)$, $\bar{x}(-1)$, $\bar{x}(-\frac{1}{2})$, $\bar{x}(\frac{1}{2})$ n'améliore la fonction, le simplexe est rétréci vers le meilleur sommet x_1 .

7.6.5 Remarques

Remarque 7.3. L'ordre d'évaluation $\bar{x}(\frac{1}{2})$, $\bar{x}(-\frac{1}{2})$, $\bar{x}(-1)$, $\bar{x}(-2)$ est assez standard dans les implémentations.

Remarque 7.4. À moins que l'étape de rétrécissement (*shrinkage step*) ne soit effectuée, la quantité

$$\frac{1}{n+1} \sum_{i=1}^{n+1} f(x_i) \quad (7.15)$$

décroît à chaque étape.

Remarque 7.5. Pour la théorie de convergence, voir [4] ou [5].

7.6.6 Algorithme 15 : Méthode de Nelder-Mead

Algorithm 15: Une étape de la méthode de Nelder-Mead

```

1 Compute the reflection point  $\bar{x}(-1)$  and evaluate  $f_{-1} = f(\bar{x}(-1))$ ;
2 if  $f(x_1) \leq f_{-1} < f(x_n)$  then
    | ;▷ Reflected point is neither best nor worst in the new simplex
3 | Replace  $x_{n+1}$  by  $\bar{x}(-1)$  and go to the next iteration;
4 else if  $f_{-1} < f(x_1)$  then
    | ;▷ Reflected point is better than the current best, try to go further
    |   along this direction
5 | Compute the expansion point  $\bar{x}(-2)$  and evaluate  $f_{-2} = f(\bar{x}(-2))$ ;
6 | if  $f_{-2} < f_{-1}$  then
7 | | Replace  $x_{n+1}$  by  $x_{-2}$  and go to the next iteration;
8 | else
9 | | Replace  $x_{n+1}$  by  $x_{-1}$  and go to the next iteration;
10 else if  $f_{-1} \geq f(x_n)$  then
    | ;▷ Reflected point is still worse than  $x_n$ , contract
11 | if  $f(x_n) \leq f_{-1} < f(x_{n+1})$  then
    | | ;▷ try to perform "outside" contraction
12 | | Evaluate  $f_{-1/2} = f(\bar{x}(-1/2))$ ;
13 | | if  $f_{-1/2} \leq f_{-1}$  then
14 | | | Replace  $x_{n+1}$  by  $x_{-1/2}$  and go to the next iteration;
15 | else
    | | ;▷ Try to perform "inside" contraction
16 | | Evaluate  $f_{1/2} = f(\bar{x}_{1/2})$ ;
17 | | if  $f_{1/2} < f_{n+1}$  then
18 | | | Replace  $x_{n+1}$  by  $x_{1/2}$  and go to the next iteration;
    | ;▷ Neither outside nor inside contraction was acceptable, shrink the
    |   simplex toward  $x_1$ 
19 | Replace  $x_i \leftarrow \frac{1}{2}(x_1 + x_i)$  for  $i = 2, 3, \dots, n+1$ ;

```

7.6.7 Illustration sur la fonction de Rosenbrock

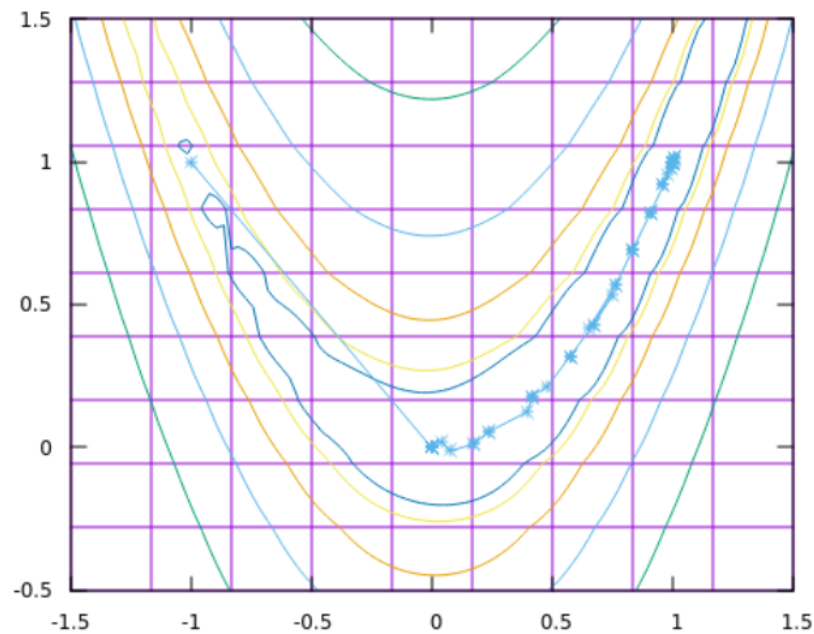


FIGURE 7.9 – Itérations de la méthode du simplexe de Nelder-Mead sur les courbes de niveau de la fonction de Rosenbrock.

Points clés à observer :

- **Méthode sans dérivée :** Nelder-Mead n'utilise aucune information sur le gradient, uniquement des évaluations de f . La trajectoire est donc moins directe que Newton ou BFGS.
- **Progression dans la vallée :** On peut observer comment le simplexe se déforme et s'adapte à la géométrie de la vallée de Rosenbrock par réflexions et contractions successives.
- **Robustesse vs efficacité :** La méthode converge malgré l'absence de dérivées, mais le nombre d'évaluations de f est bien supérieur à la steepest descent (car toutes les autres méthodes utilisent aussi le gradient).
- **Cas d'usage :** Cette méthode est particulièrement utile lorsque les dérivées sont indisponibles ou trop coûteuses à calculer (simulations, mesures expérimentales).

Chapitre 8

Théorie de l'optimisation sous contraintes

8.1 Problème général

On considère le problème d'optimisation sous contraintes suivant :

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{sous les contraintes} \quad \begin{cases} c_i(x) = 0, & i \in \mathcal{E}, \\ c_i(x) \geq 0, & i \in \mathcal{I}, \end{cases} \quad (8.1)$$

où f et les c_i sont des fonctions régulières (*smooth*), et $\mathcal{E}$ et $\mathcal{I}$ sont des ensembles finis d'indices.

- f : fonction objectif (*objective function*).
- $c_i, i \in \mathcal{E}$: contraintes d'égalité (*equality constraints*).
- $c_i, i \in \mathcal{I}$: contraintes d'inégalité (*inequality constraints*).

8.2 Ensemble réalisable

Définition 8.1 (Ensemble réalisable). L'ensemble réalisable (*feasible set*) est l'ensemble des points qui satisfont toutes les contraintes :

$$\Omega = \{x \in \mathbb{R}^n \mid c_i(x) = 0, i \in \mathcal{E}; c_i(x) \geq 0, i \in \mathcal{I}\}. \quad (8.2)$$

Le problème peut alors s'écrire de manière équivalente :

$$\min_{x \in \Omega} f(x). \quad (8.3)$$

Dans ce chapitre, nous étudierons les conditions d'optimalité nécessaires et suffisantes pour ce problème.

8.3 Solutions locales

Définition 8.2 (Solution locale). Un vecteur $x^* \in \mathbb{R}^n$ est une **solution locale** si :

1. $x^* \in \Omega$, et
2. il existe un voisinage $\mathcal{N}$ de x^* tel que :

$$f(x) \geq f(x^*) \quad \text{pour tout } x \in \mathcal{N} \cap \Omega. \quad (8.4)$$

Définition 8.3 (Solution locale stricte). Un vecteur x^* est une **solution locale stricte** si :

1. $x^* \in \Omega$, et
2. il existe un voisinage $\mathcal{N}$ de x^* tel que :

$$f(x) > f(x^*) \quad \text{pour tout } x \in \mathcal{N} \cap \Omega, \quad x \neq x^*. \quad (8.5)$$

Définition 8.4 (Solution locale isolée). Un vecteur x^* est une **solution locale isolée** si :

1. $x^* \in \Omega$, et
2. il existe un voisinage $\mathcal{N}$ de x^* tel que x^* soit la seule solution locale dans $\mathcal{N} \cap \Omega$.

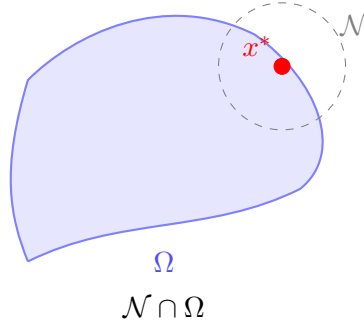


FIGURE 8.1 – Solution locale x^* sur le bord de Ω : le voisinage $\mathcal{N}$ n'est pas nécessairement inclus dans Ω .

8.4 Rappel : Équivalence problème lisse/non-lisse

Remarque 8.5. Si $g_1, \dots, g_m$ sont des fonctions scalaires régulières, alors le problème non-lisse et sans contraintes :

$$\min_{x \in \mathbb{R}^n} \left[\max_{\mu=1, \dots, m} g_\mu(x) \right] \quad (8.6)$$

est équivalent au problème lisse et sous contraintes :

$$\min_{(x,t) \in \mathbb{R}^{n+1}} t \quad \text{sous les contraintes} \quad t - g_\mu(x) \geq 0, \quad \mu = 1, \dots, m. \quad (8.7)$$

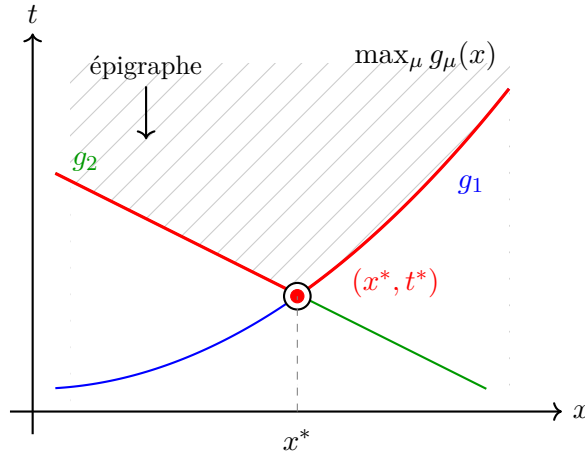


FIGURE 8.2 – Équivalence entre minimisation du max (non-lisse) et problème sous contraintes (lisse). La variable t représente une borne supérieure sur toutes les fonctions g_μ . La région hachurée représente l'épigraphe $\{(x, t) : t \geq \max_\mu g_\mu(x)\}$. Pour $x < x^*$, on a $\max(g_1, g_2) = g_2$; pour $x > x^*$, on a $\max(g_1, g_2) = g_1$.

La figure 8.2 illustre comment le problème de minimisation du maximum de plusieurs fonctions (enveloppe rouge, qui est non-différentiable au point d'intersection) peut être reformulé comme un problème lisse en introduisant une variable auxiliaire t qui majore toutes les fonctions $g_\mu(x)$. La solution optimale (x^*, t^*) se trouve au point le plus bas de l'enveloppe, c'est-à-dire là où les deux fonctions se croisent.

8.5 Programmation linéaire : la méthode du simplexe

Pour le cas particulier de la programmation linéaire, on dispose d'un algorithme efficace : la méthode du simplexe.

8.5.1 Formulation standard

Un problème de programmation linéaire en forme standard s'écrit :

$$\min_{x \in \mathbb{R}^n} c'x \quad \text{s.c.} \quad Ax = b, \quad x \geq 0, \quad (8.8)$$

où $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$.

8.5.2 Variables de base et hors-base

On partitionne les colonnes de A en :

- $\mathcal{B}$: ensemble des indices des variables de base ($|\mathcal{B}| = m$).
- $\mathcal{N}$: ensemble des indices des variables hors-base ($|\mathcal{N}| = n - m$).

On a alors $B = A_{\mathcal{B}}$ (matrice de base, inversible) et $N = A_{\mathcal{N}}$.

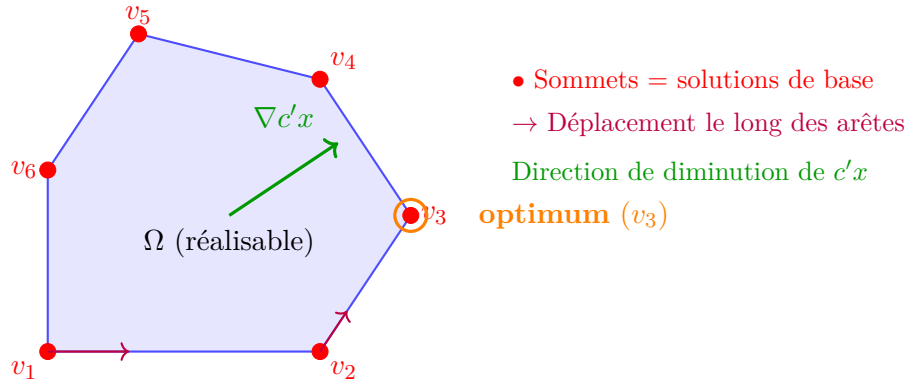


FIGURE 8.3 – Géométrie de la programmation linéaire. L'ensemble réalisable Ω est un polyèdre. Les sommets correspondent aux solutions de base réalisables. La méthode du simplexe se déplace de sommet en sommet le long des arêtes.

La figure 8.3 illustre la géométrie du problème de programmation linéaire. Les solutions de base réalisables correspondent aux sommets du polyèdre. La méthode du simplexe démarre à un sommet et se déplace vers un sommet adjacent qui améliore la fonction objectif, jusqu'à atteindre l'optimum (ici v_3).

8.5.3 Algorithme 16 : Une étape de la méthode du simplexe

Algorithme 16: Une étape de la méthode du Simplexe

- 1 Given $\mathcal{B}, \mathcal{N}, x_B = B^{-1}b \geq 0, x_N = 0$;
 - 2 Solve $B'\lambda = c_B$ for λ ;
 - 3 Compute $s_N = c_N - N'\lambda$;
 - ▷ Pricing
 - 4 **if** $s_N \geq 0$ **then**
 - 5 **stop** ;
 - ▷ optimal point found
 - 6 Select $q \in \mathcal{N}$ with $s_q < 0$ as the entering index;
 - 7 Solve $Bd = A_q$ for d ;
 - 8 **if** $d \leq 0$ **then**
 - 9 **stop** ;
 - ▷ problem is unbounded
 - 10 Calculate $x_q^+ = \min_{i|d_i > 0} \left(\frac{(x_B)_i}{d_i} \right)$, use p to denote the minimizing i ;
 - 11 Update $x_B^+ = x_B - dx_q^+, x_N^+ = (0, \dots, 0, x_q^+, 0, \dots, 0)'$;
 - 12 Change $\mathcal{B}$ by adding q and removing the basic variable corresponding to column p of B ;
-

Chapitre 9

Conditions d'optimalité

9.1 Introduction et définitions

Définition 9.1 (Ensemble actif). Soit $x \in \Omega$ un point réalisable. L'ensemble actif (*active set*), noté $\mathcal{A}(x)$, est constitué des indices des contraintes d'égalité et des indices des contraintes d'inégalité qui sont saturées (égales à 0) en x .

$$\mathcal{A}(x) = \mathcal{E} \cup \{i \in \mathcal{I} \mid c_i(x) = 0\}.$$

- Une contrainte $i \in \mathcal{I}$ est dite **active** si $c_i(x) = 0$ (donc $i \in \mathcal{A}(x)$).
- Elle est dite **inactive** si $c_i(x) > 0$ (donc $i \notin \mathcal{A}(x)$).

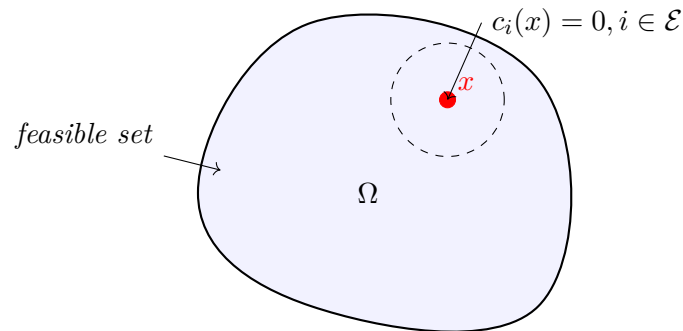


FIGURE 9.1 – Illustration de l'ensemble réalisable Ω et d'un point x sur la frontière où certaines contraintes sont actives.

9.2 Exemples

9.2.1 Une contrainte d'égalité

Considérons le problème de minimisation suivant dans $\mathbb{R}^2$:

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{sous la contrainte} \quad x_1^2 + x_2^2 - 2 = 0. \quad (9.1)$$

On a ici :

$$\begin{aligned} f(x) &= x_1 + x_2, \\ c_1(x) &= x_1^2 + x_2^2 - 2, \\ \mathcal{E} &= \{1\}, \quad \mathcal{I} = \emptyset. \end{aligned}$$

L'ensemble réalisable est le cercle de centre $(0,0)$ et de rayon $\sqrt{2}$. Calculons les gradients :

$$\nabla f(x) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \quad \nabla c_1(x) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix}. \quad (9.2)$$

On remarque que $\nabla f(x)$ est constant (le même en tout point), contrairement à $\nabla c_1(x)$ qui dépend de la position sur le cercle.

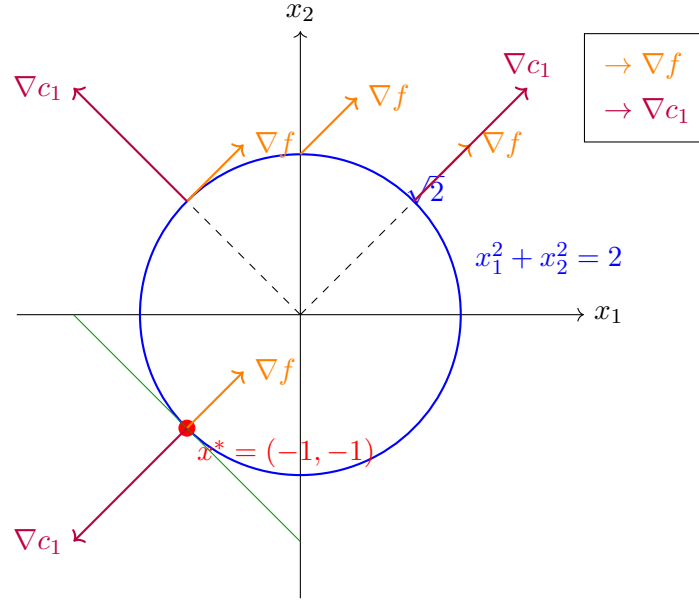


FIGURE 9.2 – Minimisation de $x_1 + x_2$ sur le cercle. Au point optimal $x^* = (-1, -1)$, les gradients ∇f et ∇c_1 sont colinéaires et de sens opposés. Notons que $\|\nabla c_1(x)\| = 2\|\nabla f(x)\|$ sur le cercle.

Commentaires :

- **Variation de la fonction objectif** : Depuis n'importe quel point du cercle (autre que l'optimum), il est possible de se déplacer le long du cercle ("reduce the function while staying on the circle") pour diminuer la valeur de f .
- **Comportement des gradients** : Le gradient de la fonction objectif ∇f est le même en tout point (vecteur constant $(1,1)$), ce qui n'est pas le cas des vecteurs contraintes $\nabla c_1(x)$ qui changent de direction selon le point considéré sur le cercle (vecteurs radiaux).
- **Condition d'optimalité** : À l'optimum $x^* = (-1, -1)$, il n'est plus possible de diminuer f tout en restant sur le cercle. Géométriquement, cela correspond au point où les lignes de niveau de f sont tangentes au cercle. Algébriquement, on observe que $\nabla f(x^*)$ et $\nabla c_1(x^*)$ sont colinéaires.

Remarque 9.2. Au point $x^* = (-1, -1)$, le gradient de la contrainte $\nabla c_1(x^*) = \begin{pmatrix} -2 \\ -2 \end{pmatrix}$ est parallèle au gradient de la fonction objectif $\nabla f(x^*) = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$.

Il existe un scalaire $\lambda_1^* \in \mathbb{R}$ tel que :

$$\nabla f(x^*) = \lambda_1^* \nabla c_1(x^*). \quad (9.3)$$

Dans cet exemple, on trouve $\lambda_1^* = -1/2$.

Dérivation (Justification heuristique) Soit x un point réalisable. Un petit pas s partant de x reste réalisable si, au premier ordre :

$$0 = c_1(x + s) \approx \underbrace{c_1(x)}_{=0} + \nabla c_1(x)^T s. \quad (9.4)$$

Donc, pour rester sur la surface de contrainte (au premier ordre), le pas s doit être orthogonal au gradient de la contrainte : $\nabla c_1(x)^T s = 0$.

De plus, le pas s produit une diminution de la fonction objectif f si :

$$0 > f(x + s) - f(x) \approx \nabla f(x)^T s. \quad (9.5)$$

C'est-à-dire $\nabla f(x)^T s < 0$.

Si x^* est un minimum local, il ne doit pas exister de direction s qui soit à la fois réalisable (tangente à la contrainte) et de descente. Cela implique que le gradient $\nabla f(x^*)$ doit être aligné avec $\nabla c_1(x^*)$. Si $\nabla f(x^*)$ avait une composante non nulle dans l'espace tangent, on pourrait trouver un s réduisant f .

Cela suggère qu'on cherche une direction d (par exemple avec $\|d\| = 1$) telle que :

$$\begin{cases} \nabla c_1(x)^T d = 0 & \text{(tangente à la surface)} \\ \nabla f(x)^T d < 0 & \text{(descente)} \end{cases} \quad (9.6)$$

S'il n'existe pas de tel d , alors il est probable que x soit un minimiseur local.

Cela ne peut être le cas que si $\nabla f(x)$ et $\nabla c_1(x)$ sont parallèles, c'est-à-dire :

$$\nabla f(x) = \lambda_1 \nabla c_1(x) \quad \text{pour un certain } \lambda_1 \in \mathbb{R}. \quad (9.7)$$

Sinon, on peut construire explicitement une direction de descente *projetée* par la formule suivante :

$$\bar{d} = - \left(I - \frac{\nabla c_1(x) \nabla c_1(x)^T}{\|\nabla c_1(x)\|^2} \right) \nabla f(x). \quad (9.8)$$

En normalisant $d = \frac{\bar{d}}{\|\bar{d}\|}$ (si $\bar{d} \neq 0$), cette direction satisfait $\nabla c_1(x)^T d = 0$ et $\nabla f(x)^T d < 0$.

Remarque 9.3 (Projecteur orthogonal). La matrice $P = I - \frac{\nabla c_1(x) \nabla c_1(x)^T}{\|\nabla c_1(x)\|^2}$ est le **projecteur orthogonal** sur le noyau de $\nabla c_1(x)^T$, c'est-à-dire sur l'espace tangent à la surface de contrainte en x . Ainsi, $\bar{d}$ est la projection de $-\nabla f(x)$ (direction de plus forte descente) sur cet espace tangent.

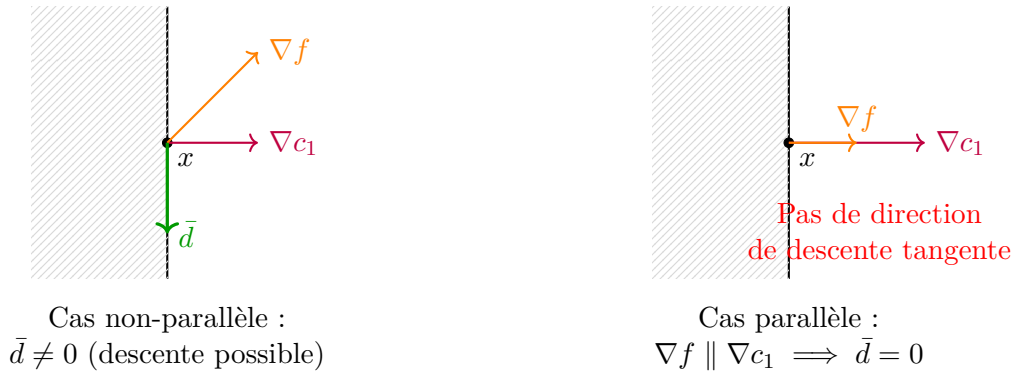


FIGURE 9.3 – Comparaison des cas : À gauche, ∇f n'est pas colinéaire à ∇c_1 , la projection $\bar{d}$ fournit une direction de descente tangente. À droite, les gradients sont colinéaires, aucune descente tangente n'est possible (condition d'optimalité).

Ce qu'il faut observer : La direction projetée $\bar{d}$ n'existe que si ∇f et ∇c_1 ne sont pas colinéaires. Quand ils sont parallèles, toute composante tangente de $-\nabla f$ est nulle : on ne peut pas descendre en restant sur la contrainte.

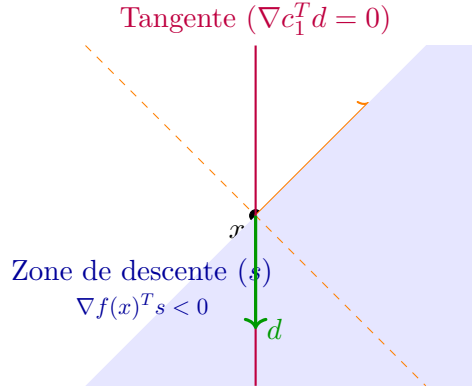


FIGURE 9.4 – Illustration de la recherche de direction. La zone bleue représente l'ensemble des directions de descente (formant un angle obtus avec ∇f). La droite verticale représente les directions tangentes à la contrainte. Ici, une direction d existe (intersection de la droite et de la zone bleue), donc x n'est pas un optimum.

Ce qu'il faut observer : On cherche une direction d telle que $\nabla f^T d < 0$ (descente) et $\nabla c_1^T d = 0$ (tangente). Si la droite tangente passe par la zone bleue, une telle direction existe et x n'est pas optimal.

Fonction Lagrangienne Définissons la fonction Lagrangienne :

$$\mathcal{L}(x, \lambda_1) = f(x) - \lambda_1 c_1(x). \quad (9.9)$$

Son gradient par rapport à x est :

$$\nabla_x \mathcal{L}(x, \lambda_1) = \nabla f(x) - \lambda_1 \nabla c_1(x). \quad (9.10)$$

La condition de parallélisme des gradients peut alors être réécrite ainsi :

$$\text{Il existe } \lambda_1^* \in \mathbb{R} \text{ tel que } \nabla_x \mathcal{L}(x^*, \lambda_1^*) = 0.$$

Cela suggère qu'il faut chercher les points stationnaires de $\mathcal{L}$ pour trouver les minimiseurs de f sous contrainte.

Remarque 9.4. Le point $(1, 1)$ est également un point stationnaire de $\mathcal{L}$ (avec un multiplicateur de Lagrange différent). Il correspond ici au maximum global de f sur le cercle.

9.2.2 Une contrainte d'inégalité

Considérons maintenant le problème avec une contrainte d'inégalité :

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{sous la contrainte} \quad 2 - x_1^2 - x_2^2 \geq 0. \quad (9.11)$$

La contrainte est $c_1(x) = 2 - x_1^2 - x_2^2 \geq 0$. L'ensemble réalisable est le disque fermé de rayon $\sqrt{2}$. La solution est toujours le même point $x^* = (-1, -1)$.

Calculons les gradients en x^* :

$$\begin{aligned}\nabla f(x^*) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2x_1 \\ -2x_2 \end{pmatrix} \implies \nabla c_1(x^*) = \begin{pmatrix} 2 \\ 2 \end{pmatrix}.\end{aligned}$$

La condition de parallélisme :

$$\nabla f(x^*) = \lambda_1^* \nabla c_1(x^*) \quad (9.12)$$

est satisfaite pour $\lambda_1^* = \frac{1}{2}$. Notez que cette fois, $\lambda_1^* > 0$.

Remarque 9.5 (Interprétation du multiplicateur positif). Le fait que $\lambda_1^* > 0$ indique que la contrainte est **bloquante** : relâcher cette contrainte permettrait de diminuer davantage f . En effet, si l'on pouvait "sortir" du disque (i.e. aller dans la direction $-\nabla f$), on pourrait continuer à diminuer la fonction objectif. Le multiplicateur λ^* quantifie cette "pression" de la contrainte sur l'optimum.

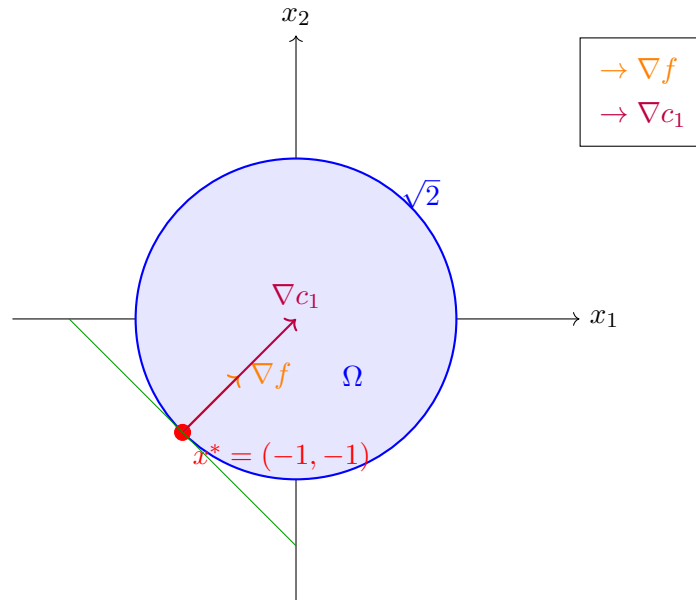


FIGURE 9.5 – Minimisation avec contrainte d'inégalité. Au point optimal, ∇f et ∇c_1 pointent dans la même direction ($\lambda^* > 0$).

Ce qu'il faut observer : Contrairement au cas d'égalité, les gradients pointent dans le **même sens**. Le multiplicateur $\lambda^* > 0$ traduit le fait que la contrainte "pousse" l'optimum : sans elle, on pourrait descendre davantage.

Dérivation (Justification heuristique) Soit x un point réalisable. Un petit pas s partant de x produit une diminution de la fonction objectif si, au premier ordre :

$$\nabla f(x)^T s < 0. \quad (9.13)$$

Ce pas conserve la réalisabilité par rapport à la contrainte c_1 si :

$$0 \leq c_1(x + s) \approx c_1(x) + \nabla c_1(x)^T s. \quad (9.14)$$

Ainsi, au premier ordre, on doit avoir :

$$c_1(x) + \nabla c_1(x)^T s \geq 0. \quad (9.15)$$

On distingue alors deux cas :

1. x est strictement à l'intérieur du domaine (i.e. $c_1(x) > 0$).

Tout pas s suffisamment petit garantira que $c_1(x) + \nabla c_1(x)^T s \geq 0$. Par exemple, on peut choisir la direction de la plus forte pente $s = -\alpha \nabla f(x)$ avec $\alpha > 0$ petit (si $\nabla f(x) \neq 0$). Dans ce cas, la contrainte n'est pas *active* et ne limite pas localement la minimisation.

2. x est sur la frontière de l'ensemble réalisable (i.e. $c_1(x) = 0$).

Dans ce cas, les conditions de diminution de la fonction et de réalisabilité deviennent :

$$\nabla f(x)^T s < 0 \quad \text{et} \quad \nabla c_1(x)^T s \geq 0. \quad (9.16)$$

À l'optimum, il ne doit pas exister de tel s . Cela implique une relation spécifique entre les gradients (voir condition KKT plus loin).

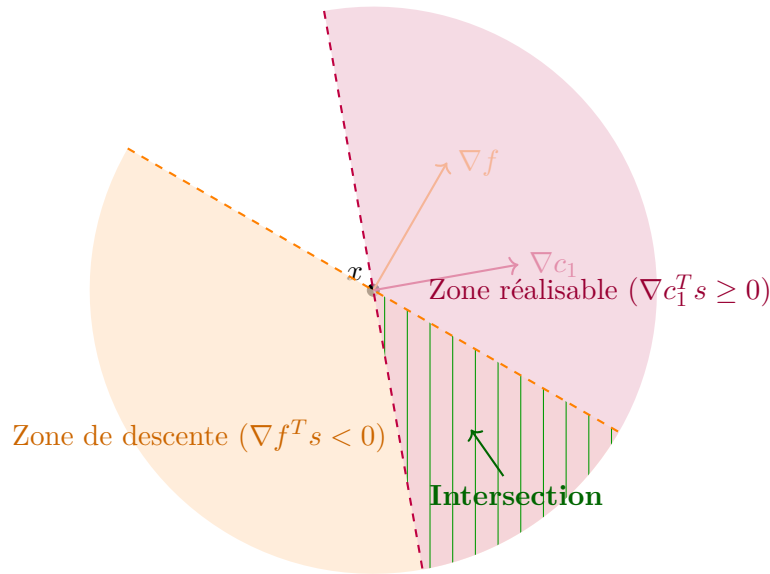
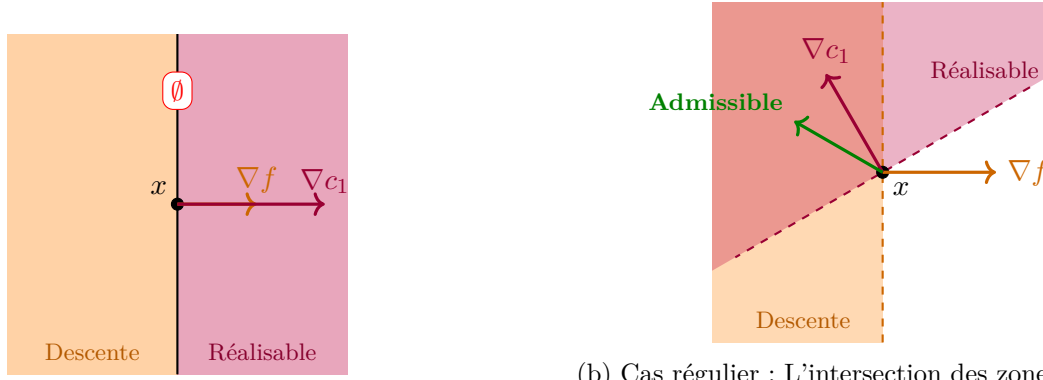


FIGURE 9.6 – Visualisation des directions admissibles. Les deux zones colorées (orange pour la descente, violet pour la réalisabilité) se chevauchent. L'intersection (hachurée en vert) représente l'ensemble des directions admissibles.

La question fondamentale devient alors : **Quand n'y a-t-il aucune direction admissible ?**



(a) Gradients alignés : zones disjointes, aucune direction admissible.

(b) Cas régulier : L'intersection des zones (mélange des couleurs) donne les directions admissibles.

FIGURE 9.7 – Comparaison des configurations "Demi-plans". À gauche, les gradients parallèles de même sens créent une obstruction totale (opposition parfaite des zones).

Ce qu'il faut observer :

- **Cas (a) :** Les zones de descente et de réalisabilité sont disjointes (aucun chevauchement). C'est la condition d'optimalité.
- **Cas (b) :** Les deux zones se chevauchent via la transparence des couleurs. L'intersection (plus foncée) donne les directions admissibles.

On en déduit la conclusion suivante :

Il n'existe aucune direction admissible lorsque les gradients sont parallèles et pointent dans la même direction :

$$\nabla f(x) = \lambda \nabla c_1(x) \quad \text{avec } \lambda > 0. \quad (9.17)$$

Résumé sur les deux cas (Conditions d'optimalité) Lorsqu'il n'existe aucune direction de descente réalisable, on a le système suivant :

$$\begin{cases} \nabla_x \mathcal{L}(x, \lambda_1^*) = 0 & \text{pour un certain } \lambda_1^* \geq 0, \\ \lambda_1^* c_1(x) = 0. \end{cases} \quad (9.18)$$

La seconde équation est appelée **condition de complémentarité** ("Complementarity condition"). Elle signifie que le multiplicateur λ_1^* ne peut être (strictement) positif que si la contrainte c_1 est active ($c_1(x) = 0$).

Rappelons la définition du Lagrangien :

$$\mathcal{L}(x, \lambda_1) = f(x) - \lambda_1 c_1(x). \quad (9.19)$$

Si $\lambda_1 = 0$, alors $\mathcal{L}(x, \lambda_1) = f(x)$ et $\nabla_x \mathcal{L} = \nabla f(x)$. Cela correspond au cas où l'optimum se trouve à l'intérieur du domaine (contrainte inactive), et la condition classique $\nabla f(x) = 0$ est retrouvée.

9.2.3 Deux contraintes d'inégalité

Considérons le problème suivant :

$$\min_{(x_1, x_2) \in \mathbb{R}^2} x_1 + x_2 \quad \text{sous les contraintes} \quad \begin{cases} 2 - x_1^2 - x_2^2 \geq 0, \\ x_2 \geq 0. \end{cases} \quad (9.20)$$

L'ensemble réalisable est la moitié supérieure du disque de rayon $\sqrt{2}$. Le point optimal est $x^* = (-\sqrt{2}, 0)$.

Calculons les gradients en ce point :

$$\begin{aligned}\nabla f(x) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2x_1 \\ -2x_2 \end{pmatrix} \implies \nabla c_1(x^*) = \begin{pmatrix} 2\sqrt{2} \\ 0 \end{pmatrix}, \\ \nabla c_2(x) &= \begin{pmatrix} 0 \\ 1 \end{pmatrix} \implies \nabla c_2(x^*) = \begin{pmatrix} 0 \\ 1 \end{pmatrix}.\end{aligned}$$

Remarquons que les deux contraintes sont actives en x^* ($c_1(x^*) = 0$ et $c_2(x^*) = 0$).

On s'attendrait à ce qu'une direction de descente réalisable d satisfasse :

$$\begin{cases} \nabla c_i(x^*)^T d \geq 0 & \text{pour } i = 1, 2 \quad (\text{réalisable}), \\ \nabla f(x^*)^T d < 0 & (\text{descente}). \end{cases} \quad (9.21)$$

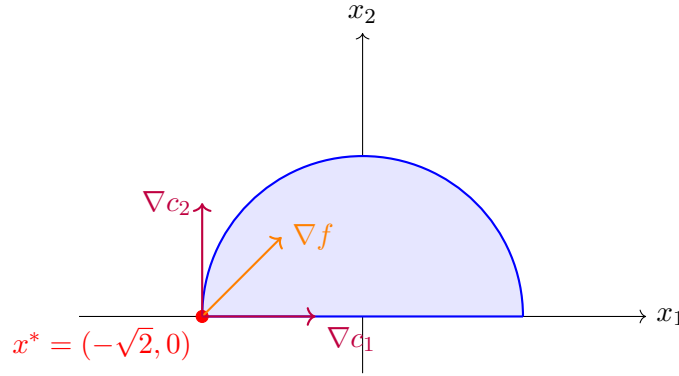


FIGURE 9.8 – Exemple avec deux contraintes actives. Les deux contraintes (disque et positivité) sont actives en x^* .

Modification du Lagrangien Pour ce problème à deux contraintes, on définit le Lagrangien :

$$\mathcal{L}(x, \lambda) = f(x) - \lambda_1 c_1(x) - \lambda_2 c_2(x). \quad (9.22)$$

Les conditions deviennent :

- $\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$ avec $\lambda^* \geq 0$ (i.e., $\lambda_1^* \geq 0$ et $\lambda_2^* \geq 0$).
- Conditions de complémentarité : $\lambda_1^* c_1(x^*) = 0$ et $\lambda_2^* c_2(x^*) = 0$.

Vérifions ces conditions pour notre point $x^* = (-\sqrt{2}, 0)$. On cherche λ_1^*, λ_2^* tels que :

$$\begin{aligned}\nabla f(x^*) - \lambda_1^* \nabla c_1(x^*) - \lambda_2^* \nabla c_2(x^*) &= 0 \\ \begin{pmatrix} 1 \\ 1 \end{pmatrix} - \lambda_1^* \begin{pmatrix} 2\sqrt{2} \\ 0 \end{pmatrix} - \lambda_2^* \begin{pmatrix} 0 \\ 1 \end{pmatrix} &= \begin{pmatrix} 0 \\ 0 \end{pmatrix}.\end{aligned}$$

Ce système donne :

$$\begin{cases} 1 - 2\sqrt{2}\lambda_1^* = 0 \implies \lambda_1^* = \frac{1}{2\sqrt{2}} > 0, \\ 1 - \lambda_2^* = 0 \implies \lambda_2^* = 1 > 0. \end{cases} \quad (9.23)$$

Les multiplicateurs sont positifs, ce qui est cohérent avec le fait que les deux contraintes sont actives et limitent la descente.

Analyse d'un autre point candidat Considérons le point $x = (\sqrt{2}, 0)$. Ce point est réalisable (sur la frontière). Vérifions si les conditions d'optimalité sont satisfaites.

$$\begin{aligned}\nabla f(x) &= \begin{pmatrix} 1 \\ 1 \end{pmatrix}, \\ \nabla c_1(x) &= \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix}, \\ \nabla c_2(x) &= \begin{pmatrix} 0 \\ 1 \end{pmatrix}.\end{aligned}$$

On cherche λ_1^*, λ_2^* tels que :

$$\begin{pmatrix} 1 \\ 1 \end{pmatrix} - \lambda_1^* \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix} - \lambda_2^* \begin{pmatrix} 0 \\ 1 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}. \quad (9.24)$$

Cela donne le système :

$$\begin{cases} 1 + 2\sqrt{2}\lambda_1^* = 0 \implies \lambda_1^* = -\frac{1}{2\sqrt{2}} < 0, \\ 1 - \lambda_2^* = 0 \implies \lambda_2^* = 1 \geq 0. \end{cases} \quad (9.25)$$

Ici, on trouve $\lambda_1^* < 0$. La condition KKT $\lambda^* \geq 0$ n'est pas respectée. **Conclusion** : Le point $x = (\sqrt{2}, 0)$ n'est pas un minimum local (en effet, on pourrait augmenter x_1 pour diminuer f tout en restant dans le domaine).

9.3 Cône tangent et qualifications des contraintes

Dans cette section, nous étudierons les conditions sous lesquelles les points de vue analytique et géométrique coïncident ("conditions under which analytical and geometrical points of view coincide").

9.3.1 Définitions

Définition 9.6 (Suite réalisable). Une suite $(z_k)_k$ est dite **suite réalisable approximant le point réalisable** x si :

- $z_k \rightarrow x$ quand $k \rightarrow \infty$,
- $z_k \in \Omega$ pour k assez grand.

Définition 9.7 (Vecteur tangent). Un vecteur $d \in \mathbb{R}^n$ est dit **tangent** (ou vecteur tangent) à Ω au point réalisable x s'il existe une suite réalisable $(z_k)_k$ approximant x et une suite de scalaires positifs $(t_k)_k$ avec $t_k \rightarrow 0$ tels que :

$$\lim_{k \rightarrow \infty} \frac{z_k - x}{t_k} = d. \quad (9.26)$$

L'ensemble des vecteurs tangents à Ω en x est noté $T_\Omega(x)$ et est appelé **cône tangent** à Ω en x .

Remarque 9.8. $T_\Omega(x)$ est bien un cône :

- $0 \in T_\Omega(x)$ (prendre la suite constante $z_k = x$).

- Si $d \in T_\Omega(x)$, alors pour tout $\alpha > 0$, $\alpha d \in T_\Omega(x)$.
En effet, il suffit de remplacer la suite $(t_k)_k$ par $(t'_k)_k$ définie par $t'_k = \alpha^{-1}t_k$. Comme $t_k \rightarrow 0$, on a aussi $t'_k \rightarrow 0$, et

$$\lim_{k \rightarrow \infty} \frac{z_k - x}{t'_k} = \lim_{k \rightarrow \infty} \alpha \frac{z_k - x}{t_k} = \alpha d.$$

Définition 9.9 (Conditions réalisables linéarisées). Étant donné un point réalisable x et l'ensemble des contraintes actives $\mathcal{A}(x)$, l'ensemble des **directions réalisables linéarisées** $\mathcal{F}(x)$ est défini par :

$$\mathcal{F}(x) = \left\{ d \in \mathbb{R}^n \mid \begin{array}{ll} d^T \nabla c_i(x) = 0 & \forall i \in \mathcal{E}, \\ d^T \nabla c_i(x) \geq 0 & \forall i \in \mathcal{I} \cap \mathcal{A}(x) \end{array} \right\}. \quad (9.27)$$

Remarque 9.10. $\mathcal{F}(x)$ est aussi un cône.

Exemple 9.11 (Cône tangent pour une contrainte d'égalité). Considérons le problème :

$$\min_{x \in \mathbb{R}^2} x_1 + x_2 \quad \text{sous la contrainte} \quad x_1^2 + x_2^2 - 2 = 0. \quad (9.28)$$

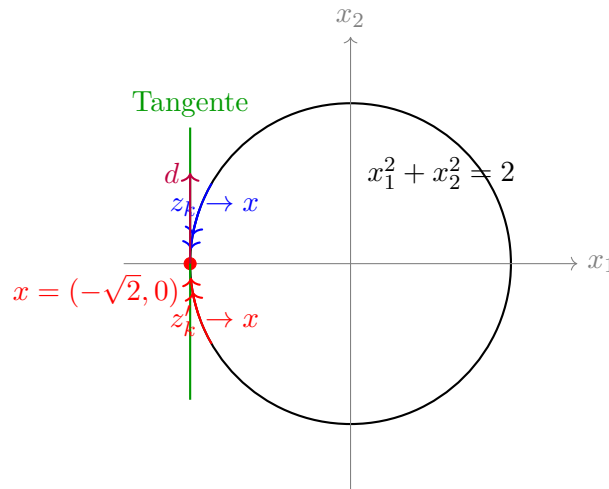
Soit $x = (-\sqrt{2}, 0)$. Les **suites réalisables** $(z_k)_k$ approchant x doivent se trouver sur le cercle. Considérons les suites réalisables :

$$z_k = \left(-\sqrt{2 - \frac{1}{k^2}}, \frac{1}{k} \right) \quad \text{ou} \quad z'_k = \left(-\sqrt{2 - \frac{1}{k^2}}, -\frac{1}{k} \right). \quad (9.29)$$

Vérifions qu'elles sont sur le cercle : $\left(-\sqrt{2 - 1/k^2} \right)^2 + (\pm 1/k)^2 = 2 - 1/k^2 + 1/k^2 = 2$.

Avec $t_k = \|z_k - x\|$, on trouve que les vecteurs $(0, 1)$ et $(0, -1)$ sont tangents. Plus généralement, l'ensemble des vecteurs tangents est la droite verticale :

$$T_\Omega(x) = \{(0, d_2) \mid d_2 \in \mathbb{R}\}. \quad (9.30)$$



Calculons maintenant $\mathcal{F}(x)$ en $x = (-\sqrt{2}, 0)$. On a $\mathcal{E} = \{1\}$ et $\mathcal{I} = \emptyset$, donc $\mathcal{A}(x) = \{1\}$. La contrainte $c_1(x) = x_1^2 + x_2^2 - 2$ a pour gradient :

$$\nabla c_1(x) = \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} \implies \nabla c_1(x) = \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix}. \quad (9.31)$$

L'ensemble des directions réalisables linéarisées est :

$$\mathcal{F}(x) = \left\{ d = \begin{pmatrix} d_1 \\ d_2 \end{pmatrix} \in \mathbb{R}^2 \mid d^T \nabla c_1(x) = 0 \right\}. \quad (9.32)$$

La condition $d^T \nabla c_1(x) = 0$ donne :

$$\begin{pmatrix} d_1 \\ d_2 \end{pmatrix}^T \begin{pmatrix} -2\sqrt{2} \\ 0 \end{pmatrix} = -2\sqrt{2}d_1 = 0 \implies d_1 = 0. \quad (9.33)$$

Donc :

$$\mathcal{F}(x) = \left\{ (0, d_2)^T \mid d_2 \in \mathbb{R} \right\} = T_\Omega(x). \quad (9.34)$$

Remarque 9.12 (Cas où $\mathcal{F}(x) \neq T_\Omega(x)$). • Si on change la contrainte pour $(x_1^2 + x_2^2 - 2)^2 = 0$, on peut montrer que $\mathcal{F}(x) = \mathbb{R}^2 \neq T_\Omega(x)$.

En effet, le gradient de cette nouvelle contrainte en x s'annule :

$$\nabla \left[(x_1^2 + x_2^2 - 2)^2 \right] = 2(x_1^2 + x_2^2 - 2) \cdot \begin{pmatrix} 2x_1 \\ 2x_2 \end{pmatrix} = 0 \text{ sur le cercle.}$$

Donc la condition $d^T \nabla c(x) = 0$ est toujours satisfaite, d'où $\mathcal{F}(x) = \mathbb{R}^2$.

- Si on change la contrainte pour $2 - x_1^2 - x_2^2 \geq 0$ (inégalité), alors :

$$T_\Omega(x) = \left\{ (d_1, d_2)^T \in \mathbb{R}^2 \mid d_1 \geq 0 \right\} = \mathcal{F}(x). \quad (9.35)$$

9.3.2 Qualification des contraintes (LICQ)

Définition 9.13 (LICQ – Linear Independence Constraint Qualification). Étant donné un point réalisable $x \in \Omega$ et l'ensemble actif $\mathcal{A}(x)$, on dit que la condition **LICQ** (Linear Independence Constraint Qualification) est satisfaite en x si l'ensemble des gradients des contraintes actives

$$\{\nabla c_i(x) \mid i \in \mathcal{A}(x)\} \quad (9.36)$$

est **linéairement indépendant**.

Proposition 9.14. Sous la condition LICQ, le cône tangent $T_\Omega(x)$ et l'ensemble des directions réalisables linéarisées $\mathcal{F}(x)$ coïncident :

$$T_\Omega(x) = \mathcal{F}(x). \quad (9.37)$$

Lemme 9.15 (Cas des contraintes linéaires). Supposons qu'en un point $x^* \in \Omega$, toutes les contraintes actives c_i (pour $i \in \mathcal{A}(x^*)$) soient des fonctions **linéaires** (ou affines). Alors :

$$T_\Omega(x^*) = \mathcal{F}(x^*). \quad (9.38)$$

Remarque 9.16. Ce lemme est un cas particulier de la proposition précédente. Les contraintes linéaires satisfont automatiquement LICQ si leurs gradients sont linéairement indépendants. De plus, pour les contraintes affines, l'approximation linéaire est exacte, ce qui explique l'égalité $T_\Omega = \mathcal{F}$.

9.4 Conditions d'optimalité

Dans cette section, nous formalisons les conditions d'optimalité pour les problèmes d'optimisation sous contraintes. Pour les preuves détaillées, nous renvoyons à Nocedal et Wright [10].

9.4.1 La fonction Lagrangienne

Définition 9.17 (Fonction Lagrangienne). Pour le problème d'optimisation

$$\min_{x \in \mathbb{R}^n} f(x) \quad \text{sous les contraintes} \quad c_i(x) \begin{cases} = 0 & \text{si } i \in \mathcal{E}, \\ \geq 0 & \text{si } i \in \mathcal{I}, \end{cases} \quad (9.39)$$

la **fonction Lagrangienne** est définie par :

$$\mathcal{L}(x, \lambda) = f(x) - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i c_i(x), \quad (9.40)$$

où $\lambda = (\lambda_i)_{i \in \mathcal{E} \cup \mathcal{I}}$ est le vecteur des **multiplicateurs de Lagrange**.

Remarque 9.18. La convention de signe peut varier selon les auteurs. Notre choix (signe moins devant les λ_i) implique que pour les contraintes d'inégalité, les multiplicateurs optimaux seront positifs $\lambda_i^* \geq 0$, ce qui correspond à l'interprétation économique standard.

9.4.2 Conditions KKT

Théorème 9.19 (Conditions de Karush-Kuhn-Tucker (KKT)). Soit x^* un minimum local du problème d'optimisation. Si LICQ est satisfaite en x^* , alors il existe un vecteur de multiplicateurs $\lambda^* = (\lambda_i^*)_{i \in \mathcal{E} \cup \mathcal{I}}$ tel que :

1. **Stationnarité :**

$$\nabla_x \mathcal{L}(x^*, \lambda^*) = \nabla f(x^*) - \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*) = 0. \quad (9.41)$$

2. **Réalisabilité primale :**

$$c_i(x^*) = 0 \quad \forall i \in \mathcal{E}, \quad c_i(x^*) \geq 0 \quad \forall i \in \mathcal{I}. \quad (9.42)$$

3. **Réalisabilité duale :**

$$\lambda_i^* \geq 0 \quad \forall i \in \mathcal{I}. \quad (9.43)$$

(Pas de contrainte de signe pour λ_i^* avec $i \in \mathcal{E}$.)

4. **Complémentarité :**

$$\lambda_i^* c_i(x^*) = 0 \quad \forall i \in \mathcal{I}. \quad (9.44)$$

Remarque 9.20 (Interprétation de la complémentarité). La condition de complémentarité $\lambda_i^* c_i(x^*) = 0$ signifie que :

- Si une contrainte d'inégalité $i \in \mathcal{I}$ n'est pas active ($c_i(x^*) > 0$), alors le multiplicateur correspondant doit être nul ($\lambda_i^* = 0$).
- Un multiplicateur strictement positif ($\lambda_i^* > 0$) ne peut apparaître que si la contrainte est active ($c_i(x^*) = 0$).

Intuitivement, seules les contraintes “serrées” exercent une “force” sur l'optimum.

9.4.3 Conditions du second ordre

Hypothèse. Dans cette section III.2, nous supposons que f et les c_i sont **deux fois continûment différentiables**.

Définition 9.21 (Cône critique). Étant donné $\mathcal{A}(x^*)$ et des multiplicateurs de Lagrange λ^* satisfaisant les conditions KKT, le **cône critique** $\mathcal{C}(x^*, \lambda^*)$ est défini par :

$$\mathcal{C}(x^*, \lambda^*) = \left\{ w \in \mathcal{F}(x^*) \mid \nabla c_i(x^*)^T w = 0 \text{ pour tout } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ avec } \lambda_i^* > 0 \right\}. \quad (9.45)$$

Remarque 9.22. Le cône critique est un sous-ensemble de l'ensemble des directions réalisables linéarisées $\mathcal{F}(x^*)$. Il contient les directions “critiques” qui sont orthogonales aux gradients des contraintes d'inégalité qui sont à la fois actives **et** qui ont un multiplicateur strictement positif.

Intuitivement, ce sont les directions le long desquelles la contrainte “compte vraiment” car elle exerce une force non nulle sur l'optimum.

Remarque 9.23 (Équivalence pour le cône critique). De manière équivalente, on peut caractériser le cône critique par :

$$w \in \mathcal{C}(x^*, \lambda^*) \iff \begin{cases} \nabla c_i(x^*)^T w = 0 & \text{pour tout } i \in \mathcal{E}, \\ \nabla c_i(x^*)^T w \geq 0 & \text{pour tout } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ avec } \lambda_i^* = 0, \\ \nabla c_i(x^*)^T w = 0 & \text{pour tout } i \in \mathcal{I} \cap \mathcal{A}(x^*) \text{ avec } \lambda_i^* > 0. \end{cases} \quad (9.46)$$

Autrement dit, les directions du cône critique “adhèrent” aux contraintes d'égalité et aux contraintes d'inégalité actives lorsqu'on perturbe légèrement la fonction objectif.

Propriété fondamentale. On a la propriété importante suivante :

$$w \in \mathcal{C}(x^*, \lambda^*) \implies \lambda_i^* \nabla c_i(x^*)^T w = 0 \quad \forall i \in \mathcal{E} \cup \mathcal{I}. \quad (9.47)$$

En effet :

- Pour $i \in \mathcal{E}$, on a $\nabla c_i(x^*)^T w = 0$ par définition de $\mathcal{F}(x^*)$.
- Pour $i \in \mathcal{I} \cap \mathcal{A}(x^*)$ avec $\lambda_i^* > 0$, on a $\nabla c_i(x^*)^T w = 0$ par définition de $\mathcal{C}(x^*, \lambda^*)$.
- Pour $i \in \mathcal{I} \cap \mathcal{A}(x^*)$ avec $\lambda_i^* = 0$, le produit $\lambda_i^* \nabla c_i(x^*)^T w = 0$.
- Pour $i \in \mathcal{I} \setminus \mathcal{A}(x^*)$ (contraintes inactives), on a $\lambda_i^* = 0$ par complémentarité.

Conséquence. Puisque $\nabla_x \mathcal{L}(x^*, \lambda^*) = 0$ par la condition KKT de stationnarité, on a :

$$w \in \mathcal{C}(x^*, \lambda^*) \implies w^T \nabla f(x^*) = w^T \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*) = \sum_{i \in \mathcal{E} \cup \mathcal{I}} \lambda_i^* \nabla c_i(x^*)^T w = 0. \quad (9.48)$$

Ainsi, le cône critique rassemble les directions le long desquelles, à partir de l'information du premier ordre seulement, on ne peut pas déterminer si f va augmenter ou diminuer.

Théorème 9.24 (Conditions nécessaires du second ordre). Soit x^* un minimum local. Supposons que LICQ soit satisfaite en x^* . Soit λ^* le vecteur des multiplicateurs de Lagrange satisfaisant les conditions KKT. Alors, pour tout $w \in \mathcal{C}(x^*, \lambda^*)$:

$$w^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w \geq 0. \quad (9.49)$$

Autrement dit, la Hessienne du Lagrangien est **semi-définie positive** sur le cône critique.

Théorème 9.25 (Conditions suffisantes du second ordre). Soit x^* un point réalisable satisfaisant les conditions KKT avec multiplicateurs λ^* . Supposons que pour tout $w \in \mathcal{C}(x^*, \lambda^*) \setminus \{0\}$:

$$w^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w > 0. \quad (9.50)$$

Alors x^* est un **minimum local strict** du problème d'optimisation.

Remarque 9.26. La différence entre les conditions nécessaires et suffisantes réside dans l'inégalité :

- **Nécessaire** : ≥ 0 (semi-définie positive)
- **Suffisante** : > 0 strictement pour $w \neq 0$ (définie positive sur $\mathcal{C}$)

Cette marge permet de garantir que x^* est un minimum local strict, et non un simple point-selle ou un point d'inflexion.

Exemple 9.27 (Application des conditions KKT). Considérons le problème de minimisation suivant :

$$\min_{x \in \mathbb{R}^2} f(x) = x_1 + x_2 \quad \text{sous la contrainte} \quad c_1(x) = 2 - x_1^2 - x_2^2 \geq 0. \quad (9.51)$$

On a $\mathcal{I} = \{1\}$ et $\mathcal{E} = \emptyset$.

Le Lagrangien. La fonction Lagrangienne est :

$$\mathcal{L}(x, \lambda) = x_1 + x_2 - \lambda(2 - x_1^2 - x_2^2). \quad (9.52)$$

Conditions KKT. Les conditions KKT s'écrivent :

1. **Stationnarité** :

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = 1 + 2\lambda x_1 = 0, \\ \frac{\partial \mathcal{L}}{\partial x_2} = 1 + 2\lambda x_2 = 0. \end{cases} \quad (9.53)$$

2. **Réalisabilité primale** :

$$2 - x_1^2 - x_2^2 \geq 0. \quad (9.54)$$

3. **Réalisabilité duale** :

$$\lambda \geq 0. \quad (9.55)$$

4. **Complémentarité** :

$$\lambda(2 - x_1^2 - x_2^2) = 0. \quad (9.56)$$

Résolution. De (1) et (2), on tire $x_1 = x_2 = -\frac{1}{2\lambda}$ (si $\lambda \neq 0$).

Cas 2 : $\lambda \neq 0$. Par la condition de complémentarité (4), on a $c_1(x) = 0$, donc la contrainte est active :

$$2 - x_1^2 - x_2^2 = 0 \iff x_1^2 + x_2^2 = 2. \quad (9.57)$$

De (1) et (2), on a $x_1 = x_2 = -\frac{1}{2\lambda}$. En substituant dans l'équation du cercle :

$$2x_1^2 = 2 \implies x_1^2 = 1 \implies x_1 = 1 \quad \text{ou} \quad x_1 = -1. \quad (9.58)$$

Analysons les deux possibilités :

- **Si $x_1 = 1$** : Alors $x_2 = 1$. L'équation (1) donne $1 + 2\lambda(1) = 0 \implies \lambda = -1/2$. Or, la condition de réalisabilité duale (3) impose $\lambda \geq 0$. Donc ce cas est **impossible** (contradiction).
- **Si $x_1 = -1$** : Alors $x_2 = -1$. L'équation (1) donne $1 + 2\lambda(-1) = 0 \implies \lambda = 1/2$. On a bien $\lambda > 0$, ce qui est valide.

Ainsi, seul le couple $(x^*, \lambda^*) = ((-1, -1), 1/2)$ satisfait les conditions KKT.

Vérification.

- $x^* = (-1, -1)$ est réalisable : $2 - 1 - 1 = 0 \geq 0$. ✓
- $\lambda^* = 1/2 > 0$. ✓
- Complémentarité : $\lambda^* \cdot c_1(x^*) = \frac{1}{2} \cdot 0 = 0$. ✓

Le point $x^* = (-1, -1)$ avec $\lambda^* = 1/2$ satisfait les conditions KKT.

Vérification des conditions du second ordre. La Hessienne du Lagrangien est :

$$\nabla_{xx}^2 \mathcal{L}(x, \lambda) = 2\lambda \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} = 2\lambda I_2. \quad (9.59)$$

En x^* avec $\lambda^* = 1/2$:

$$\nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) = I_2, \quad (9.60)$$

qui est définie positive. Donc pour tout $w \neq 0$ dans le cône critique :

$$w^T \nabla_{xx}^2 \mathcal{L}(x^*, \lambda^*) w = \|w\|^2 > 0. \quad (9.61)$$

Par le théorème des conditions suffisantes, $x^* = (-1, -1)$ est un **minimum local strict**.

9.5 Dualité

La théorie de la dualité permet de construire un nouveau problème à partir des fonctions et des données du problème original. Ce problème dual peut s'avérer plus facile à résoudre numériquement.

Dans cette section, nous considérons uniquement des contraintes d'inégalité et nous supposons que f et les fonctions $-c_i$ sont convexes.

9.5.1 Problème Primal

Le problème primal est défini par :

$$(P) \quad \min_{x \in \mathbb{R}^n} f(x) \quad \text{sujet à} \quad c(x) \geq 0, \quad (9.62)$$

où $c(x) = (c_1(x), \dots, c_m(x))^T$.

9.5.2 Fonction Duale et Problème Dual

Définition 9.28 (Fonction objective duale). On définit le Lagrangien par $\mathcal{L}(x, \lambda) = f(x) - \lambda^T c(x)$. La **fonction objective duale** $q : \mathbb{R}^m \rightarrow \mathbb{R}$ est définie par :

$$q(\lambda) = \inf_x \mathcal{L}(x, \lambda). \quad (9.63)$$

Le domaine de q , noté $\mathcal{D}$, est l'ensemble des valeurs de λ pour lesquelles cet infimum est fini (c'est-à-dire $> -\infty$) :

$$\mathcal{D} = \{\lambda \in \mathbb{R}^m \mid q(\lambda) > -\infty\}. \quad (9.64)$$

Si f est convexe et les contraintes c_i sont concaves (c-à-d $-c_i$ convexes), alors le calcul de $q(\lambda)$ est un problème d'optimisation convexe (souvent faisable).

Définition 9.29 (Problème Dual). Le **problème dual** consiste à maximiser la fonction duale sur les multiplicateurs positifs :

$$(D) \quad \max_{\lambda \in \mathbb{R}^m} q(\lambda) \quad \text{sujet à} \quad \lambda \geq 0. \quad (9.65)$$

9.5.3 Exemple

Soit le problème primal suivant :

$$\min_{x \in \mathbb{R}^2} \frac{1}{2}(x_1^2 + x_2^2) \quad \text{sujet à} \quad x_1 - 1 \geq 0. \quad (9.66)$$

Ici $f(x) = \frac{1}{2}(x_1^2 + x_2^2)$ et $c_1(x) = x_1 - 1$.

Calcul de la fonction duale. Le Lagrangien est :

$$\mathcal{L}(x, \lambda) = \frac{1}{2}(x_1^2 + x_2^2) - \lambda(x_1 - 1). \quad (9.67)$$

Lorsque λ est fixé, $\mathcal{L}(\cdot, \lambda)$ est une fonction convexe par rapport à x (somme de fonctions convexes). L'infimum est atteint lorsque le gradient par rapport à x s'annule :

$$\begin{cases} \frac{\partial \mathcal{L}}{\partial x_1} = x_1 - \lambda = 0 \implies x_1 = \lambda, \\ \frac{\partial \mathcal{L}}{\partial x_2} = x_2 = 0. \end{cases} \quad (9.68)$$

En substituant ces valeurs dans l'expression du Lagrangien, on obtient $q(\lambda)$:

$$q(\lambda) = \mathcal{L}((\lambda, 0), \lambda) \quad (9.69)$$

$$= \frac{1}{2}(\lambda^2 + 0^2) - \lambda(\lambda - 1) \quad (9.70)$$

$$= \frac{1}{2}\lambda^2 - \lambda^2 + \lambda \quad (9.71)$$

$$= -\frac{1}{2}\lambda^2 + \lambda. \quad (9.72)$$

Résolution du problème dual. Le problème dual est donc :

$$\max_{\lambda \geq 0} \left(-\frac{1}{2}\lambda^2 + \lambda \right). \quad (9.73)$$

La fonction $h(\lambda) = -\frac{1}{2}\lambda^2 + \lambda$ est une parabole concave (ouverte vers le bas). Son maximum sans contrainte est atteint en $h'(\lambda) = -\lambda + 1 = 0 \implies \lambda = 1$. Comme $\lambda = 1$ satisfait la contrainte $\lambda \geq 0$, la solution optimale duale est $\lambda^* = 1$.

Cela nous permet de retrouver la solution primale : $x_1^* = \lambda^* = 1$ et $x_2^* = 0$. Le point $x^* = (1, 0)$ est bien la projection de l'origine sur le demi-plan $x_1 \geq 1$.

9.5.4 Propriétés et Théorèmes de Dualité

Théorème 9.30 (Propriétés de la fonction duale). La fonction objective duale q est **concave** et son domaine $\mathcal{D}$ est un ensemble **convexe**. Ceci est vrai indépendamment de la convexité de f ou des contraintes c_i .

Théorème 9.31 (Dualité Faible – Weak Duality). *Soit $\bar{x}$ un point réalisable pour le problème primal et $\bar{\lambda} \geq 0$ un point réalisable pour le problème dual. Alors :*

$$q(\bar{\lambda}) \leq f(\bar{x}). \quad (9.74)$$

*Ainsi, la valeur de la fonction duale pour tout $\lambda \geq 0$ fournit une **borne inférieure** (lower bound) pour la solution du problème primal.*

Théorème 9.32 (Lien KKT et Dualité). *Supposons que $\bar{x}$ soit une solution du problème primal, que f et les fonctions $-c_i$ soient convexes, et qu'elles soient continûment différentiables en $\bar{x}$.*

*Alors, tout vecteur $\bar{\lambda}$ pour lequel le couple $(\bar{x}, \bar{\lambda})$ satisfait les conditions KKT est une **solution du problème dual**.*

Remarque 9.33. Les conditions KKT pour le problème primal s'écrivent :

$$\begin{aligned} \nabla f(\bar{x}) - \sum_{i=1}^m \bar{\lambda}_i \nabla c_i(\bar{x}) &= 0, \\ c(\bar{x}) &\geq 0, \\ \bar{\lambda} &\geq 0, \\ \bar{\lambda}_i c_i(\bar{x}) &= 0 \quad \text{pour } i = 1, \dots, m. \end{aligned}$$

Théorème 9.34 (Réciproque partielle – Partial Converse). *Supposons que f et les fonctions $-c_i$ soient convexes et continûment différentiables sur $\mathbb{R}^n$. Supposons que $\bar{x}$ soit une solution du problème primal satisfaisant LICQ. Supposons que $\hat{\lambda}$ soit une solution du problème dual et que l'infimum $\inf_x \mathcal{L}(x, \hat{\lambda})$ soit atteint en $\hat{x}$. Supposons enfin que $\mathcal{L}(\cdot, \hat{\lambda})$ soit **strictement convexe**.*

Alors $\bar{x} = \hat{x}$ (c'est-à-dire que $\hat{x}$ est l'unique solution du problème primal) et $f(\bar{x}) = \mathcal{L}(\hat{x}, \hat{\lambda})$.

Remarque 9.35. Dans l'exemple précédent, pour la solution duale $\hat{\lambda} = 1$, l'infimum de $\mathcal{L}(x, 1)$ est atteint en $x_1 = 1, x_2 = 0$, qui est la solution du problème primal.

9.5.5 Exemple : Dualité en Programmation Linéaire

Considérons le programme linéaire primal suivant :

$$(P) \quad \min_{x \in \mathbb{R}^n} c^T x \quad \text{sujet à} \quad Ax - b \geq 0. \quad (9.75)$$

Ici, la fonction objective est linéaire (donc convexe) et les contraintes sont affines. La fonction objective duale est :

$$q(\lambda) = \inf_{x \in \mathbb{R}^n} [c^T x - \lambda^T (Ax - b)] \quad (9.76)$$

$$= \inf_{x \in \mathbb{R}^n} [(c - A^T \lambda)^T x + \lambda^T b]. \quad (9.77)$$

L'infimum de cette fonction affine par rapport à x vaut $-\infty$, sauf si le coefficient de x est nul.

$$\inf_x (c - A^T \lambda)^T x = \begin{cases} 0 & \text{si } c - A^T \lambda = 0, \\ -\infty & \text{sinon.} \end{cases} \quad (9.78)$$

Donc, pour être dans le domaine $\mathcal{D}$ (où $q > -\infty$), nous devons imposer $c = A^T \lambda$. Dans ce cas, $q(\lambda) = \lambda^T b$.

Le problème dual s'écrit alors :

$$(D) \quad \max_{\lambda} \lambda^T b \quad \text{sujet à} \quad c = A^T \lambda, \quad \lambda \geq 0. \quad (9.79)$$

Cela illustre comment le dual d'un problème de minimisation linéaire est un problème de maximisation linéaire.

Chapitre 10

Programmation linéaire : la méthode du simplexe

10.1 Optimalité et dualité

10.1.1 Optimalité du problème primal

Considérons le problème de programmation linéaire (PL) sous sa forme standard :

$$\min_{x \in \mathbb{R}^n} c^T x \quad \text{sujet à} \quad Ax = b, \quad x \geq 0, \quad (10.1)$$

où A est une matrice $m \times n$ de rang m , $c \in \mathbb{R}^n$ est le vecteur de coût et $b \in \mathbb{R}^m$.

Une étape clé est d'identifier une **solution de base réalisable**. On partitionne la matrice A en $A = [B \mid N]$ où B est une matrice $m \times m$ inversible (la base) et N contient les $n - m$ autres colonnes. De même, on partitionne $x = (x_B, x_N)$ et $c = (c_B, c_N)$. L'équation $Ax = b$ devient $Bx_B + Nx_N = b$, soit $x_B = B^{-1}b - B^{-1}Nx_N$.

En fixant $x_N = 0$, on obtient la solution de base $x_B = B^{-1}b$. Si $x_B \geq 0$, la solution est dite réalisable.

Le coût réduit ("reduced cost") est défini par le vecteur s_N :

$$s_N = c_N - (B^{-1}N)^T c_B = c_N - N^T \lambda, \quad (10.2)$$

avec $\lambda = B^{-T} c_B$ le vecteur des multiplicateurs (ou variables duales).

Théorème 10.1 (Condition d'optimalité). *Une solution de base réalisable $(x_B, 0)$ est optimale si et seulement si le coût réduit est positif ou nul, c'est-à-dire $s_N \geq 0$ (pour toutes ses composantes).*

10.1.2 Le problème dual

À tout problème primal (P) correspond un problème dual (D). Pour la forme standard ci-dessus, le dual est :

$$\max_{\lambda \in \mathbb{R}^m} b^T \lambda \quad \text{sujet à} \quad A^T \lambda \leq c. \quad (10.3)$$

Les relations fondamentales entre primal et dual sont :

- **Dualité faible** : Pour tout x réalisable pour (P) et tout λ réalisable pour (D), on a $b^T \lambda \leq c^T x$.
- **Dualité forte** : Si (P) admet une solution optimale x^* , alors (D) admet une solution optimale λ^* et leurs valeurs sont égales : $c^T x^* = b^T \lambda^*$.

10.2 Géométrie de l'ensemble réalisable

L'ensemble des solutions réalisables $\mathcal{F} = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$ est un polyèdre convexe (intersection d'un sous-espace affine et de l'orthant positif).

Proposition 10.2. *Les sommets (ou points extrêmes) de ce polyèdre correspondent exactement aux solutions de base réalisables du problème linéaire.*

La méthode du simplexe consiste géométriquement à se déplacer d'un sommet à un sommet adjacent du polyèdre, en suivant une arête qui améliore (diminue) la fonction objectif. Puisque le nombre de sommets est fini (au plus $\binom{n}{m}$), l'algorithme termine en un nombre fini d'étapes (sous réserve de précautions contre le cyclage, par exemple avec la règle de Bland).

10.3 Aperçu de la méthode du simplexe

Algorithm 17: One step of the Simplex method

- 1 Given $\mathcal{B}, \mathcal{N}, x_B = B^{-1}b \geq 0, x_N = 0$;
 - 2 Solve $B^\top \lambda = c_B$ for λ ;
 - 3 Compute $s_N = c_N - N^\top \lambda$;
 ▷ Pricing
 - 4 **if** $s_N \geq 0$ **then**
 - 5 | **stop** ▷ optimal point found
 - 6 Select $q \in \mathcal{N}$ with $s_q < 0$ as the entering index;
 - 7 Solve $Bd = A_q$ for d ;
 - 8 **if** $d \leq 0$ **then**
 - 9 | **stop** ▷ problem is unbounded
 - 10 Calculate $x_q^+ = \min_{i|d_i > 0} \left(\frac{(x_B)_i}{d_i} \right)$, use p to denote the minimizing i ;
 - 11 Update $x_B^+ = x_B - dx_q^+, x_N^+ = (0, \dots, 0, x_q^+, 0, \dots, 0)^\top$;
 - 12 Change $\mathcal{B}$ by adding q and removing the basic variable corresponding to column p of B ;
-

Chapitre 11

Programmation quadratique

11.1 Introduction

Un programme quadratique (QP) général s'écrit sous la forme :

$$\min_{x \in \mathbb{R}^n} q(x) = \frac{1}{2}x^T Gx + c^T x, \quad (11.1)$$

sujet aux contraintes :

$$a_i^T x = b_i, \quad i \in \mathcal{E}, \quad (11.2)$$

$$a_i^T x \geq b_i, \quad i \in \mathcal{I}, \quad (11.3)$$

où G est une matrice symétrique $n \times n$, $c \in \mathbb{R}^n$, $a_i \in \mathbb{R}^n$ et $b_i \in \mathbb{R}$.

Remarque 11.1. Pour les QP, une quantité finie de calculs est nécessaire pour les résoudre ou pour déclarer qu'ils sont irréalisables ("infeasible").

- On parle de **QP convexe** lorsque G est semi-définie positive.
- On parle de **QP strictement convexe** lorsque G est définie positive.

11.2 QP avec contraintes d'égalité

Considérons les problèmes de la forme :

$$\min_{x \in \mathbb{R}^n} \frac{1}{2}x^T Gx + c^T x, \quad \text{sujet à } Ax = b, \quad (11.4)$$

où $A \in \mathbb{R}^{m \times n}$ (avec $m < n$) est la Jacobienne des contraintes. Ses lignes sont les vecteurs a_i . Le vecteur $b \in \mathbb{R}^m$ rassemble les scalaires b_i . On suppose que la matrice A est de **rang plein** ("full rank").

11.2.1 Propriétés

Les conditions d'optimalité du premier ordre (KKT) s'écrivent :

$$\begin{cases} Gx^* + c - A^T \lambda^* = 0 \\ Ax^* = b \end{cases} \quad (11.5)$$

Cela est équivalent au système matriciel suivant :

$$\begin{pmatrix} G & -A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} x^* \\ \lambda^* \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix} \quad (11.6)$$

Remarque 11.2. Si l'on réécrit x^* comme $x + p$, où x est un candidat pour la solution et p est un pas ("step"), le système devient :

$$\begin{pmatrix} G & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = \begin{pmatrix} g \\ h \end{pmatrix} \quad (11.7)$$

où $g = c + Gx$ est le gradient de la fonction objectif au point x , $h = Ax - b$ est le résidu des contraintes, et $p = x^* - x$.

La matrice $\begin{pmatrix} G & A^T \\ A & 0 \end{pmatrix}$ est appelée la **matrice KKT**.

Soit Z une matrice de taille $n \times (n - m)$ dont les colonnes forment une base de l'espace nul ("null-space") de A . C'est-à-dire que $AZ = 0$ et A est de rang plein.

Lemme 11.3. *Supposons que A soit de rang plein (sur les lignes). Supposons que la matrice Hessienne réduite $Z^T G Z$ soit définie positive. Alors la matrice KKT $\begin{pmatrix} G & A^T \\ A & 0 \end{pmatrix}$ est non-singulière.*

Démonstration. Soient les vecteurs w et v tels que :

$$\begin{pmatrix} G & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} w \\ v \end{pmatrix} = 0 \quad (11.8)$$

La deuxième ligne nous donne $Aw = 0$. Ainsi, w appartient à l'espace nul de A , donc il existe un vecteur u tel que $w = Zu$.

Multiplions la première ligne par w^T à gauche :

$$\begin{aligned} 0 &= \begin{pmatrix} w \\ v \end{pmatrix}^T \begin{pmatrix} G & A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} w \\ v \end{pmatrix} \\ &= w^T G w + w^T A^T v + v^T A w \end{aligned}$$

Puisque $Aw = 0$, on a $v^T A w = 0$. De même, $w^T A^T v = (Aw)^T v = 0$. On obtient donc $0 = w^T G w$. En remplaçant w par Zu , on a :

$$0 = (Zu)^T G (Zu) = u^T (Z^T G Z) u \quad (11.9)$$

Puisque $Z^T G Z$ est définie positive, cela implique que $u = 0$, et par conséquent $w = Zu = 0$.

La première ligne devient alors $G \cdot 0 + A^T v = 0$, soit $A^T v = 0$. Comme A est de rang plein (sur les lignes), A^T est de rang plein (sur les colonnes), ce qui implique que $v = 0$.

Ainsi, le seul vecteur solution est le vecteur nul $\begin{pmatrix} w \\ v \end{pmatrix} = 0$, ce qui prouve que la matrice est non-singulière. □

Nous allons prouver que sous les mêmes conditions, x^* est un minimiseur local strict, et même une solution globale.

Théorème 11.4. *Supposons que A soit de rang plein (sur les lignes). Supposons que la matrice Hessienne réduite $Z^T G Z$ soit définie positive. Alors le vecteur x^* satisfaisant*

$$\begin{pmatrix} G & -A^T \\ A & 0 \end{pmatrix} \begin{pmatrix} x^* \\ \lambda^* \end{pmatrix} = \begin{pmatrix} -c \\ b \end{pmatrix} \quad (11.10)$$

est l'unique solution globale.

Démonstration. Soit x un autre point réalisable : $Ax = b$. Soit $p = x^* - x$. Puisque $Ax = b$ et $Ax^* = b$, nous avons

$$Ap = A(x^* - x) = 0. \quad (11.11)$$

Substituons dans $q(x)$:

$$\begin{aligned} q(x) &= q(x^* - p) = \frac{1}{2}(x^* - p)^T G(x^* - p) + c^T(x^* - p) \\ &= \frac{1}{2}p^T Gp - p^T Gx^* + \frac{1}{2}(x^*)^T Gx^* - c^T p + c^T x^* \\ &= \frac{1}{2}p^T Gp - p^T Gx^* - c^T p + q(x^*) \quad (*) \end{aligned}$$

Utilisons maintenant la première ligne du système KKT :

$$Gx^* + c - A^T \lambda^* = 0 \implies Gx^* + c = A^T \lambda^* \quad (11.12)$$

Multiplions à gauche par p^T :

$$p^T(Gx^* + c) = p^T A^T \lambda^* = (Ap)^T \lambda^* \quad (11.13)$$

Or, nous avons vu que $Ap = 0$, donc le terme de droite est nul. Ainsi :

$$p^T Gx^* + p^T c = 0 \implies -p^T Gx^* - c^T p = 0 \quad (11.14)$$

En substituant cela dans l'équation (*), nous obtenons :

$$q(x) = q(x^*) + \frac{1}{2}p^T Gp \quad (11.15)$$

Puisque p est dans l'espace nul de A , il existe un vecteur $u \in \mathbb{R}^{n-m}$ tel que $p = Zu$. Alors :

$$p^T Gp = (Zu)^T G(Zu) = u^T (Z^T GZ)u \quad (11.16)$$

Comme la matrice Hessienne réduite $Z^T GZ$ est supposée définie positive, on a $u^T Z^T GZ u > 0$ pour tout $u \neq 0$. Or $u \neq 0 \iff p \neq 0 \iff x \neq x^*$.

Par conséquent, pour tout $x \neq x^*$ réalisable,

$$q(x) = q(x^*) + \frac{1}{2}u^T Z^T GZ u > q(x^*) \quad (11.17)$$

Ce qui prouve que x^* est l'unique minimiseur global. $\square$

11.2.2 Résolution directe du système KKT

Remarque 11.5. On peut prouver (Forsgren et al.) que la matrice KKT est toujours indéfinie lorsque $m \geq 1$. Autrement dit, il existe des vecteurs x et y tels que $x^T K x > 0$ et $y^T K y < 0$ (où K désigne la matrice KKT).

Factorisation du système KKT complet

Nous ne pouvons pas utiliser la factorisation de Cholesky à cause de la nature indéfinie de la matrice. Une décomposition LU est faisable, mais elle ignore la symétrie du système.

Ainsi, nous utilisons la **factorisation symétrique indéfinie** pour K (matrice symétrique) :

$$PKP^T = LBL^T, \quad (11.18)$$

où :

- P est une matrice de permutation ;
- L est une matrice triangulaire inférieure unitaire ("unit lower triangular") ;
- B est une matrice diagonale par blocs, avec des blocs de taille 1×1 ou 2×2 .

Pour résoudre le système KKT $K \begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = \begin{pmatrix} g \\ h \end{pmatrix}$:

1. Résoudre $Lz = P^T \begin{pmatrix} g \\ h \end{pmatrix}$ pour z (descente).
2. Résoudre $B\hat{z} = z$ pour $\hat{z}$ (résolution bloc-diagonale).
3. Résoudre $L^T \bar{z} = \hat{z}$ pour $\bar{z}$ (remontée).
4. Poser $\begin{pmatrix} -p \\ \lambda^* \end{pmatrix} = P\bar{z}$.

Méthode du complément de Schur

Cette méthode est applicable si G est **définie positive** (et donc inversible). Partons du système KKT :

$$\begin{cases} -Gp + A^T \lambda^* = g \\ Ap = h \end{cases} \quad (11.19)$$

Multiplions la première équation par AG^{-1} :

$$\begin{aligned} -AG^{-1}Gp + AG^{-1}A^T \lambda^* &= AG^{-1}g \\ -Ap + (AG^{-1}A^T) \lambda^* &= AG^{-1}g \end{aligned}$$

En utilisant la deuxième équation ($Ap = h$), on obtient :

$$(AG^{-1}A^T) \lambda^* = AG^{-1}g - h \quad (11.20)$$

Le système $(AG^{-1}A^T) \lambda^* = AG^{-1}g - h$ est défini positif. On peut le résoudre pour trouver λ^* . Ensuite, on utilise la première ligne du système KKT pour trouver p :

$$Gp = A^T \lambda^* - g \implies p = G^{-1}(A^T \lambda^* - g). \quad (11.21)$$

Considérations numériques : Cette méthode nécessite des opérations avec G^{-1} et la factorisation de la matrice $AG^{-1}A^T$. Elle est donc adaptée lorsque :

- G est bien conditionnée et facile à inverser (exemple : diagonale).
- G^{-1} est explicitement connue.
- m (le nombre de contraintes) est petit.

Méthode de l'espace nul

La non-singularité de G n'est pas requise ici ; nous avons seulement besoin des hypothèses du lemme (A de rang plein, $Z^T G Z$ définie positive). Supposons que nous partitionnons le vecteur p comme suit :

$$p = Yp_Y + Zp_Z \quad (11.22)$$

où Z est la matrice $n \times (n - m)$ de l'espace nul de A , et Y est une matrice $n \times m$ telle que la matrice $[Y \mid Z]$ soit non-singulière. Pour simplification, on choisit souvent Y tel que $AY = I$ (si possible) ou $A[Y \mid Z]$ triangulaire.

Substituons p dans la deuxième équation du système KKT ($Ap = h$) :

$$A(Yp_Y + Zp_Z) = h \implies AYp_Y + AZp_Z = h \quad (11.23)$$

Puisque Z est dans l'espace nul, $AZ = 0$. Ainsi :

$$(AY)p_Y = h \quad (11.24)$$

Comme A est de rang m et $[Y \mid Z]$ est non-singulière, la matrice AY est non-singulière de taille $m \times m$. On peut donc résoudre ce système pour trouver p_Y . (Si on choisit Y tel que $AY = I$, alors $p_Y = h$).

Ensuite, substituons p dans la première équation du système KKT ($-Gp + A^T\lambda^* = g$) :

$$-G(Yp_Y + Zp_Z) + A^T\lambda^* = g \quad (11.25)$$

Multiplions à gauche par Z^T :

$$\begin{aligned} -Z^T G Y p_Y - Z^T G Z p_Z + Z^T A^T \lambda^* &= Z^T g \\ -Z^T G Y p_Y - Z^T G Z p_Z + (AZ)^T \lambda^* &= Z^T g \end{aligned}$$

Comme $AZ = 0$, le terme impliquant λ^* disparaît :

$$-(Z^T G Z)p_Z = Z^T g + Z^T G Y p_Y \quad (11.26)$$

Le système $(Z^T G Z)p_Z = -(Z^T g + Z^T G Y p_Y)$ permet de trouver p_Z , car la matrice réduite $Z^T G Z$ est supposée définie positive (donc inversible).

Enfin, pour trouver λ^* , on multiplie la première équation par Y^T :

$$(AY)^T \lambda^* = Y^T(g + Gp) \quad (11.27)$$

Ce système linéaire permet de déterminer λ^* .

Remarques :

- Cette méthode est efficace lorsque $n - m$ est petit.
- Le calcul de Z est la difficulté principale ; il n'est pas unique. De mauvais choix peuvent résulter en un mauvais conditionnement de $Z^T G Z$.

Exemple : Soit $G = \begin{pmatrix} 6 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 4 \end{pmatrix}$, $c = \begin{pmatrix} -8 \\ -3 \\ -3 \end{pmatrix}$. Contraintes : $\begin{cases} x_1 + x_3 = 3 \\ x_2 + x_3 = 0 \end{cases}$. D'où $A = \begin{pmatrix} 1 & 0 & 1 \\ 0 & 1 & 1 \end{pmatrix}$ et $b = \begin{pmatrix} 3 \\ 0 \end{pmatrix}$. On peut choisir $Z = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}$ (vérifiez $AZ = 0$) et $Y = \begin{pmatrix} 2/3 & -1/3 \\ -1/3 & 2/3 \\ 1/3 & 1/3 \end{pmatrix}$ tel que $AY = I_2$.

Exemple (suite) : Soit $x = \begin{pmatrix} -1 \\ -2 \\ 2 \end{pmatrix}$. On calcule $h = Ax - b = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$ et $g = c + Gx = \begin{pmatrix} -7/2 \\ -2 \\ 2 \end{pmatrix}$ (note : valeurs de l'exemple du cours).

1. **Trouver** p_Y : Résoudre $(AY)p_Y = -h$. Ici $p_Y = -h = \begin{pmatrix} 0 \\ 0 \end{pmatrix}$.
2. **Trouver** p_Z : Résoudre $(Z^T G Z)p_Z = -(Z^T g + Z^T G Y p_Y)$. Calculons les termes : $Z^T G Z = \begin{pmatrix} -1 & -1 & 1 \end{pmatrix} \begin{pmatrix} 6 & 2 & 1 \\ 2 & 5 & 2 \\ 1 & 2 & 4 \end{pmatrix} \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix} = 13$ (scalaire). $Z^T g = -13$. Le terme $Z^T G Y p_Y$ est nul car $p_Y = 0$. L'équation devient : $13p_Z = -(-13) \implies p_Z = 1$.

3. **Reconstruire** $p : p = Yp_Y + Zp_Z = Y \cdot 0 + Z \cdot 1 = \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix}$.

4. **Solution** : $x^* = x + p = \begin{pmatrix} -1 \\ -2 \\ 2 \end{pmatrix} + \begin{pmatrix} -1 \\ -1 \\ 1 \end{pmatrix} = \begin{pmatrix} -2 \\ -3 \\ 3 \end{pmatrix}$.

11.2.3 Résolution itérative du système KKT

Pour les systèmes de grande taille, les méthodes directes deviennent trop coûteuses. On préfère alors les méthodes itératives, comme le Gradient Conjugué (CG). Cependant, l'utilisation directe du CG sur le système KKT complet est instable car la matrice KKT n'est pas définie positive (elle est indéfinie).

L'idée est d'appliquer le CG sur le **système réduit**, qui lui est défini positif (sous l'hypothèse que $Z^T G Z$ l'est).

Substituons $x^* = Yp_Y + Zp_Z$ dans le problème. p_Z est solution du problème de minimisation sans contrainte :

$$\min_{p_Z \in \mathbb{R}^{n-m}} \frac{1}{2} p_Z^T (Z^T G Z) p_Z + p_Z^T c_Z \quad (11.28)$$

où $c_Z = Z^T G Y p_Y + Z^T c$ (en notant que le terme linéaire dépend aussi de p_Y , déterminé par les contraintes).

Ainsi, p_Z vérifie l'équation normale :

$$(Z^T G Z) p_Z = -c_Z \quad (11.29)$$

Puisque $Z^T G Z$ est définie positive, on peut utiliser le Gradient Conjugué (CG) pour trouver p_Z .

Préconditionnement : En pratique, on utilise un préconditionneur W_{ZZ} pour accélérer la convergence. Un bon préconditionneur devrait regrouper les valeurs propres de $W_{ZZ}^{-1/2} (Z^T G Z) W_{ZZ}^{-1/2}$. Idéalement, on choisirait $W_{ZZ} = Z^T G Z$, mais c'est trop coûteux. De bons choix sont de la forme $W_{ZZ} = Z^T H Z$, où H est une approximation symétrique de G telle que $Z^T H Z$ soit définie positive.

L'algorithme suivant (Algorithme 17 dans certaines références) décrit cette procédure :

Algorithm 18: Preconditioned Conjugate Gradient for reduced systems

- 1 Choose an initial point x_Z (correspondant à notre p_Z);
 - 2 Compute $r_Z = Z^T G Z x_Z + c_Z$, $g_Z = W_{ZZ}^{-1} r_Z$, and $d_Z = -g_Z$;
 - 3 **repeat**
 - 4 $\alpha \leftarrow \frac{r_Z^T g_Z}{d_Z^T Z^T G Z d_Z}$;
 - 5 $x_Z \leftarrow x_Z + \alpha d_Z$;
 - 6 $r_Z^+ \leftarrow r_Z + \alpha Z^T G Z d_Z$;
 - 7 $g_Z^+ \leftarrow W_{ZZ}^{-1} r_Z^+$;
 - 8 $\beta \leftarrow \frac{(r_Z^+)^T g_Z^+}{r_Z^T g_Z}$;
 - 9 $d_Z \leftarrow -g_Z^+ + \beta d_Z$;
 - 10 $g_Z \leftarrow g_Z^+$;
 - 11 $r_Z \leftarrow r_Z^+$;
 - 12 **until** a termination test is satisfied;
-

11.3 Problèmes avec contraintes d'inégalité

11.3.1 Propriétés

11.3.2 La méthode du gradient projeté

Calcul du point de Cauchy

Minimisation dans le sous-espace

Résumé

Algorithm 19: Gradient projection method for Quadratic Programming

```

1 Compute a feasible starting point  $x^0$ ;
2 for  $k = 0, 1, \dots$  do
3   if  $x^k$  satisfies the KKT conditions for the bound-constrained problem then
4     | stop with solution  $x^* = x^k$ ;
5   Set  $x = x^k$  and find the Cauchy point  $x^c$ ;
6   Find an approximate solution  $x^+$  of


$$\min_x q(x) = \frac{1}{2}x^\top Gx + x^\top c$$

      subject to  $x_i = x_i^c, \quad i \in \mathcal{A}(x^c)$ 
                  $l_i \leq x_i \leq u_i, \quad i \notin \mathcal{A}(x^c)$ 

      such that  $q(x^+) \leq q(x^c)$  and  $x^+$  is feasible;
7    $x^{k+1} \leftarrow x^+$ ;
```

Chapitre 12

Méthodes de pénalité et Lagrangien augmenté

12.1 La méthode de pénalité quadratique

12.1.1 Motivation

Algorithm 20: Quadratic penalty method

```
1 Given  $\mu_0 > 0$ , a nonnegative sequence  $(\tau_k)_k$  with  $\tau_k \rightarrow 0$ , and a starting point  $x_0^s$ ;  
2 for  $k = 0, 1, \dots$  do  
3   Find an approximate minimizer  $x_k$  of  $Q(\cdot; \mu_k)$ , starting at  $x_k^s$  and terminating when  
    $\|\nabla_x Q(x; \mu_k)\| \leq \tau_k$ ;  
4   if final convergence test satisfied then  
5     stop with approximate solution  $x_k$ ;  
6   Choose new penalty parameter  $\mu_{k+1} > \mu_k$ ;  
7   Choose new starting point  $x_{k+1}^s$ ;
```

12.1.2 Convergence de la méthode (avec contraintes d'égalité)

12.2 Fonctions de pénalité non lisses

12.2.1 Propriétés

Algorithm 21: Classical ℓ^1 penalty method

```
1 Given  $\mu_0 > 0$ , tolerance  $\tau > 0$ , starting point  $x_0^s$ ;  
2 for  $k = 0, 1, \dots$  do  
3   Find an approximate minimizer  $x_k$  of  $\phi_1(x; \mu_k)$ , starting at  $x_k^s$ ;  
4   if  $\|h(x_k)\| \leq \tau$  then  
5     stop with approximate solution  $x_k$ ;  
6   Choose new penalty parameter  $\mu_{k+1} > \mu_k$ ;  
7   Choose new starting point  $x_{k+1}^s$ ;
```

12.2.2 Méthode de pénalité ℓ^1 pratique

12.3 Méthode du Lagrangien augmenté

12.3.1 Propriétés du cas avec contraintes d'égalité

Algorithm 22: Augmented Lagrangian method for equality constraints

- 1 Given $\mu_0 > 0$, tolerance $\tau_0 > 0$, starting points x_0^s and λ^0 ;
 - 2 **for** $k = 0, 1, \dots$ **do**
 - 3 Find an approximate minimizer x_k of $\mathcal{L}_A(\cdot, \lambda^k; \mu_k)$, starting at x_k^s , and terminating
 when $\|\nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k)\| \leq \tau_k$;
 - 4 **if** *a convergence test for the equality-constrained problem is satisfied* **then**
 - 5 | **stop** with approximate solution x_k ;
 - 6 Update Lagrange multipliers using

$$\lambda_i^{k+1} = \lambda_i^k - \mu_k c_i(x_k), \text{ for all } i \in \mathcal{E};$$
 Choose new penalty parameter $\mu_{k+1} \geq \mu_k$;
 - 7 Set starting point for the next iteration to $x_{k+1}^s = x_k$;
 - 8 Select tolerance τ_{k+1} ;
-

12.3.2 Méthodes pratiques du Lagrangien augmenté

Formulation avec contraintes de bornes

Algorithm 23: Bound-constrained Lagrangian method

```

1 Choose an initial point  $x_0$  and initial multipliers  $\lambda^0$ ;
2 Choose convergence tolerances  $\eta_*$  and  $\omega_*$ ;
3 Set  $\mu_0 = 10$ ,  $\omega_0 = \frac{1}{\mu_0}$ , and  $\eta_0 = \frac{1}{\mu_0^{0.1}}$ ;
4 for  $k = 0, 1, \dots$  do
5     Find an approximate solution  $x_k$  of the problem

                
$$\min_x \mathcal{L}_A(x, \lambda^k; \mu_k) \text{ subject to } l \leq x \leq u$$


        such that

                
$$\|x_k - P(x_k - \nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k), l, u)\| \leq \omega_k;$$


        if  $\|c(x_k)\| \leq \eta_k$  then
            ▷ test for convergence
6         if  $\|c(x_k)\| \leq \eta_*$  and  $\|x_k - P(x_k - \nabla_x \mathcal{L}_A(x_k, \lambda^k; \mu_k), l, u)\| \leq \omega_*$  then
7             | stop with approximate solution  $x_k$ ;
            ▷ update multipliers, tighten tolerances
8              $\lambda^{k+1} \leftarrow \lambda^k - \mu_k c(x_k)$ ;
9              $\mu_{k+1} \leftarrow \mu_k$ ;
10             $\eta_{k+1} \leftarrow \frac{\eta_k}{\mu_{k+1}^{0.9}}$ ;
11             $\omega_{k+1} \leftarrow \frac{\omega_k}{\mu_{k+1}}$ ;
12 else
            ▷ increase penalty parameters, tighten tolerances
13             $\lambda^{k+1} \leftarrow \lambda^k$ ;
14             $\mu_{k+1} \leftarrow 100\mu_k$ ;
15             $\eta_{k+1} \leftarrow \frac{1}{\mu_{k+1}^{0.1}}$ ;
16             $\omega_{k+1} \leftarrow \frac{1}{\mu_{k+1}}$ ;

```

Formulation avec contraintes linéaires

Formulation sans contraintes

Bibliographie

- [1] Reeves Fletcher and Colin M Reeves. Function minimization by conjugate gradients. *The computer journal*, 7(2) :149–154, 1964.
- [2] Philip E Gill, Walter Murray, and Margaret H Wright. *Practical optimization*. SIAM, 2019.
- [3] Magnus R Hestenes, Eduard Stiefel, et al. Methods of conjugate gradients for solving linear systems. *Journal of research of the National Bureau of Standards*, 49(6) :409–436, 1952.
- [4] Carl Tim Kelley. Detection and remediation of stagnation in the Nelder–Mead algorithm using a sufficient decrease condition. *SIAM journal on optimization*, 10(1) :43–55, 1999.
- [5] JC Lagarias, JA Reeds, MH Wright, and PE Wright. Convergence properties of the Nelder–Mead algorithm in low dimensions. *AT&T Bell Laboratories, Murray Hill, NJ, Technical Report*, 1995.
- [6] Nicolas Langrené, Xavier Warin, and Pierre Gruet. Fast Gaussian process inference by exact Matérn kernel decomposition. *arXiv preprint arXiv :2508.01864*, 2025.
- [7] David G Luenberger. *Introduction to Linear and Nonlinear Programming*. Addison-Wesley, 1984.
- [8] Jorge J Moré and Danny C Sorensen. Newton’s method. Technical report, Argonne National Lab.(ANL), Argonne, IL (United States), 1982.
- [9] Jorge Nocedal. Updating quasi-Newton matrices with limited storage. *Mathematics of computation*, 35(151) :773–782, 1980.
- [10] Jorge Nocedal and Stephen J Wright. *Numerical optimization*. Springer, 2006.